

Sail Extensions: The Host Implementation

Plain Sail, the prototype, and the complete historical host patch

Querygraph

2026-10-01

Contents

Host implementation: plain Sail and the extension prototype	4
What the comparison actually says	4
Preserve what current upstream already changed	4
Discovery stays above execution	5
Registry and codec changes have different jobs	5
Envelopes and foreign inputs need host boundaries	5
Placement and retry must remain distinguishable	6
Memory ownership changes admission semantics	6
Lifecycle corrections extend beyond plugin loading	6
Keep graph helpers distinct from a generic SDK	7
A phased route to upstream review	7
Complete host integration patch	8
01. Cargo.lock	8
02. Cargo.toml	15
03. crates/sail-common-datafusion/Cargo.toml	16
04. crates/sail-common-datafusion/src/connect_extension.rs	16
05. crates/sail-common-datafusion/src/connect_extension/tests.rs	20
06. crates/sail-common-datafusion/src/driver_extension.rs	23
07. crates/sail-common-datafusion/src/driver_extension/tests.rs	26
08. crates/sail-common-datafusion/src/geometry.rs	27
09. crates/sail-common-datafusion/src/lib.rs	28
10. crates/sail-common-datafusion/src/native_resource.rs	29
11. crates/sail-common-datafusion/src/native_scalar.rs	32
12. crates/sail-common-datafusion/src/session/lifecycle.rs	34
13. crates/sail-common-datafusion/src/session/mod.rs	35
14. crates/sail-common-datafusion/src/spec/plan.rs	35
15. crates/sail-execution/src/driver/job_scheduler/core.rs	35
16. crates/sail-execution/src/driver/job_scheduler/native_retry_tests.rs	36
17. crates/sail-execution/src/driver/job_scheduler/options.rs	38
18. crates/sail-execution/src/job_graph/planner.rs	38
19. crates/sail-execution/src/proto/codec.rs	39
20. crates/sail-execution/src/proto/converter.rs	39
21. crates/sail-execution/src/proto/mod.rs	41
22. crates/sail-execution/src/proto/native_expr.rs	41
23. crates/sail-execution/src/task_runner/actor/handler.rs	46
24. crates/sail-execution/src/worker_manager/kubernetes.rs	46
25. crates/sail-execution/src/worker_manager/mod.rs	47
26. crates/sail-execution/src/worker_manager/process.rs	47
27. crates/sail-execution/src/worker_manager/process_command.rs	49
28. crates/sail-function/src/scalar/array/spark_array.rs	50
29. crates/sail-native-resource-ffi/Cargo.toml	52
30. crates/sail-native-resource-ffi/src/lib.rs	52
31. crates/sail-plan/src/function/mod.rs	54
32. crates/sail-plan/src/resolver/expression/geometry.rs	55
33. crates/sail-plan/src/resolver/expression/mod.rs	57
34. crates/sail-plan/src/resolver/query/extension.rs	57
35. crates/sail-plan/src/resolver/query/mod.rs	58
36. crates/sail-session/Cargo.toml	59

37. crates/sail-session/build.rs	59
38. crates/sail-session/proto/gf/utls/v1/utls.proto	60
39. crates/sail-session/src/extensions/driver.rs	60
40. crates/sail-session/src/extensions/graph_utils/cleanup_tests.rs	64
41. crates/sail-session/src/extensions/graph_utils/functions.rs	66
42. crates/sail-session/src/extensions/graph_utils/local.rs	67
43. crates/sail-session/src/extensions/graph_utils/local_tests.rs	69
44. crates/sail-session/src/extensions/graph_utils/mod.rs	71
45. crates/sail-session/src/extensions/graph_utils/plan.rs	72
46. crates/sail-session/src/extensions/graph_utils/storage.rs	75
47. crates/sail-session/src/extensions/graph_utils/tests.rs	80
48. crates/sail-session/src/extensions/manifest.rs	84
49. crates/sail-session/src/extensions/mod.rs	86
50. crates/sail-session/src/extensions/package_identity.py	92
51. crates/sail-session/src/extensions/plan.rs	92
52. crates/sail-session/src/extensions/plan/tests.rs	94
53. crates/sail-session/src/extensions/python_owner.rs	95
54. crates/sail-session/src/extensions/resource_tests.rs	96
55. crates/sail-session/src/lib.rs	99
56. crates/sail-session/src/runtime.rs	99
57. crates/sail-session/src/runtime/memory.rs	100
58. crates/sail-session/src/session_factory/job_runner.rs	102
59. crates/sail-session/src/session_factory/server.rs	103
60. crates/sail-session/src/session_factory/worker.rs	104
61. crates/sail-session/src/session_manager/actor/core.rs	104
62. crates/sail-session/src/session_manager/actor/handler.rs	105
63. crates/sail-session/src/session_manager/actor/mod.rs	107
64. crates/sail-session/src/session_manager/actor/options.rs	107
65. crates/sail-session/src/session_manager/cleanup.rs	108
66. crates/sail-session/src/session_manager/mod.rs	109
67. crates/sail-spark-connect/Cargo.toml	110
68. crates/sail-spark-connect/proto/sail/extension/v1/extension.proto	110
69. crates/sail-spark-connect/src/entrypoint.rs	110
70. crates/sail-spark-connect/src/error.rs	111
71. crates/sail-spark-connect/src/executor.rs	111
72. crates/sail-spark-connect/src/executor/tests.rs	113
73. crates/sail-spark-connect/src/proto/extension.rs	114
74. crates/sail-spark-connect/src/proto/extension/tests.rs	117
75. crates/sail-spark-connect/src/proto/extension/wire.rs	120
76. crates/sail-spark-connect/src/proto/mod.rs	122
77. crates/sail-spark-connect/src/proto/plan.rs	123
78. crates/sail-spark-connect/src/server.rs	123
79. crates/sail-spark-connect/src/service/plan_analyzer.rs	124
80. crates/sail-spark-connect/src/service/plan_executor.rs	124
81. crates/sail-spark-connect/src/session.rs	125
82. crates/sail-spark-connect/src/session_manager.rs	127

Host implementation: plain Sail and the extension prototype

The [extension review guide](#) describes the boundary an extension author would use. This companion explains the changes inside Sail that support the prototype, which responsibilities belong in the host, and which implementation decisions remain open. Its comparison is deliberately between named source revisions, not between an old fork and an assumed version of upstream.

PLAIN means upstream lakehq/sail at 4d31e15b350c975aed95c23e6cc7e7c51c59fe52. **PROTOTYPE** means querygraph/sail at bd8ce9ae8839477e2c08a0475ab7900b115c5366, the reviewed sail-extensions revision. Their common ancestor is a85d912d72ae03a6d97b6a3fd151f5752da636c6. The historical host patch runs from that ancestor to PROTOTYPE. It records what the fork introduced; it is not a patch prepared against PLAIN.

The current-source comparison below was made by reading both revisions. No merge, build or runtime experiment was performed for this document. Source structure can establish that a mechanism exists and where it lives; it cannot qualify a rebased combination or prove a deployment's behavior.

What the comparison actually says

PLAIN already has Spark Connect, DataFusion execution, catalogs, session factories, distributed scheduling and extension mechanisms used internally by Sail. It is not an engine without extensibility. The prototype adds a particular route for independently packaged native extensions, including a constrained stateful relation path.

Concern	PLAIN at 4d31e15b	PROTOTYPE at bd8ce9ae	Proposed decision destination
Embedding	Custom session factory is already public	Uses session construction to install extensions	Existing embedding seam; not a new prerequisite
Discovery	No pysail.extensions loader	Flag-gated wheel discovery and per-session binding	Session-layer integration
Native scalar codec	Existing explicit UDF codec and expression converter	Package identity and metadata-bearing field preservation	Generic codec invariants, then loader integration
Connect relations	Extension relation returns unsupported	Bounded envelope, registry and handler dispatch	Protocol and planner contract
Foreign inputs	No prototype-specific adapter	Original host context retained; partitions gathered	Host execution invariant; gathering remains a policy
Native placement	No driver-extension ownership node	Driver-owned references and one-attempt native regions	Scheduler implementation with explicit effect policy
Admission	Default runtime creation constructs a pool	Explicit shared domain plus prepaid native leases	Resource policy and small ownership ABI
Teardown	Existing driver/session shutdown	Terminal interruption, draining hooks and lease wait	Generic lifecycle fixes plus native policy
GraphUtils	No compiled-in graph utility adapter	Owned graph staging and helper functions	Separately scoped domain support

These destinations are review proposals, not claims of upstream acceptance. The [eight modules](#) separate the public invariants from the particular types used to implement them.

Preserve what current upstream already changed

The session-factory hook from #2630 is already in PLAIN. `create_spark_session_manager_with_factory` accepts an embedder's factory, and the normal constructor delegates to it. An embedder can wrap Sail's normal configuration rather than replace its semantics. It should not be presented as work still needed to enable the prototype. [PLAIN session manager](#).

Upstream has also advanced since the common ancestor. Its Spark session initialization now seeds DataFusion's execution timezone from the resolved Spark session timezone. Its planner prepares asyn-

chronous functions; its UDF codec includes the Jev async wrapper. Its job-graph planner materializes shared file-scan build sides so a single-partition build can be scanned once and broadcast. These are concrete changes in paths the prototype also touches, not hypothetical future conflicts. [PLAIN timezone initialization](#), [query planner](#), [job-graph planner](#).

A future upstream change must retain those behaviors while adding the narrowly chosen extension mechanism. Replacing a current file with its fork version would not be such an integration. The historical patch is useful review material precisely because its baseline remains explicit; applicability and combined regression coverage are separate work.

Discovery stays above execution

PROTOTYPE discovers `pysail.extensions` entry points when the flag is enabled, reads manifests, validates exact API/DataFusion/Arrow declarations, checks collisions, and binds fresh session state. Native objects cross named DataFusion capsules. The loader retains package code for process lifetime and retains session owners where native objects require them. Execution crates do not need to discover Python packages themselves. [PROTOTYPE loader](#).

PLAIN has a session-factory seam but not this package protocol. The proposed addition is therefore a loader and registration policy, not the invention of session customization. Name precedence, collision errors, partial-load failure and worker installation requirements belong in the user-visible contract; Python ownership wrappers and registry storage are implementation details.

This direction matches the upstream maintainer's preference for Python distribution and DataFusion FFI, with independently scheduled releases and no public Sail Rust trait. Exact version declarations and package fingerprints currently define a narrow qualification boundary. They do not prove arbitrary capsule layouts safe or establish a stable ABI range. Neither wheel installation nor content identity replaces testing the supported artifact pair. [Maintainer discussion](#); [compatibility module](#).

Registry and codec changes have different jobs

PLAIN already resolves functions and serializes many built-in and Python UDF variants. Its current decoder remains an explicit codec, not the prototype's package-identity registry. PROTOTYPE adds `OwnedScalar`, retaining the native owner and identifying the package/configuration and function. Workers resolve that identity from installed code; pointers never travel in the plan. [PLAIN codec](#), [PROTOTYPE native registry](#).

There is a second concern: preserving complete return fields, including their metadata, while expressions cross the distributed codec. PROTOTYPE's converter handles native scalars and metadata-bearing scalar, literal and cast expressions. That invariant matters even when an ordinary builtin sits between native calls; a package loader alone cannot preserve information the host codec discards. [PROTOTYPE expression handling](#).

Review these changes separately: generic field-preservation tests can stand without a loader, while package registration needs collision and identity tests. Any port must preserve PLAIN's newer UDF behavior, including async wrappers. Source inspection here does not establish that the historical wrapper composes with every current function kind.

Envelopes and foreign inputs need host boundaries

PLAIN explicitly rejects `Relation.extension`. PROTOTYPE decodes either an allowed bare payload or a Sail envelope containing a typed payload and input plans. It enforces envelope and payload sizes, input arity, type-URL length and nesting limits before dispatching through a relation registry. The handler returns a provider, not eagerly computed result rows. [PLAIN dispatch](#), [PROTOTYPE decoder](#).

The input adapter addresses a narrower runtime problem. DataFusion's foreign context reconstruction does not retain all Sail services, so `HostInputExec` executes and polls its child under the original host context

and runtime. It also installs `CoalescePartitionsExec`, exposing one gathered partition. That gathering is the prototype's initial input policy, not an inherent requirement of native interoperability. [PROTOTYPE adapter](#).

The upstream contract should preserve execution context, schemas, ownership and truthful distribution properties. A later partition-preserving adapter must satisfy those invariants explicitly. It should not be prevented by declaring single-partition gathering permanent. Nor should the current adapter be read as propagation of every host service into arbitrary foreign operators.

Placement and retry must remain distinguishable

PLAIN's scheduler uses the configured attempt limit for its task regions. PROTOTYPE identifies regions containing `DriverExtensionExec` and limits them to one attempt. A bound plan is referenced by owner and plan identifiers; decoding validates liveness, ownership, input arity and schemas. This prevents a worker from treating a driver-local reference as its own native state. [PLAIN attempt policy](#), [PROTOTYPE attempt policy](#).

The conservative rule protects opaque operations whose mutation may have committed before an acknowledgement was lost. It includes reads because the host lacks a general effect classification. It is not cross-request exactly-once execution. Its cost is giving up otherwise useful retry opportunities.

Stateless foreign worker operators are a possible next design, before replicated or recoverable native state. Replay-safe reads are another separate choice. Neither requires exposing scheduler internals as a public SDK. The review decision is whether this initial driver policy is acceptable, and what evidence would justify an additional placement or replay class. [Placement module](#).

Memory ownership changes admission semantics

In PLAIN's default runtime factory, creating a runtime constructs a memory pool from configuration; its mutator can subsequently customize the builder. PROTOTYPE adds an explicitly owned admission domain. With extensions enabled, the standard manager supplies its domain to sessions and in-process workers. A separately started worker factory owns another domain. [PLAIN runtime factory](#), [PROTOTYPE domain](#).

The resource bridge prepays a non-spillable quota from that host pool and passes an opaque lease to the extension. Participating state and exported buffers must retain it until final storage release. The manifest currently restricts this bridge to driver placement; it does not automatically admit worker scalar allocations. Idle prepaid capacity stays reserved.

These are process-local admission domains, not a cgroup or machine limit. Independent worker pools do not jointly enforce one physical-memory budget. Pool configuration, process count and headroom for unaccounted runtime, Python and transport allocations all matter. Even a correct reservation ledger is not an RSS measurement.

The ownership invariant merits a small ABI review; prepayment and manager-wide sharing merit a separate policy review. Hierarchical admission is a legitimate alternative to prepayment. The unsafe case is independent promises against the same capacity, not the mere existence of multiple pools. [Memory module](#).

Lifecycle corrections extend beyond plugin loading

PLAIN shuts down drivers and the gateway through the session manager. PROTOTYPE adds protocol-resource draining, rejection of late session work, native-release tracking and cleanup tasks for sessions already removed from the live map. Its executor gains a terminal interrupted state so reattachment can observe interruption without retaining execution buffers. Python-owner destruction acquires the GIL to avoid indefinitely deferred decrefs. [PLAIN shutdown](#), [PROTOTYPE shutdown](#).

Several corrections alter ordinary Sail paths without requiring the extension flag. They should be proposed and tested as such, rather than described as entirely isolated experimental code. The source includes focused regression fixtures, but this document does not report a fresh run against PLAIN.

The native-release wait is unbounded. Cooperative cancellation and retaining quota for surviving owners are the safety requirements; a timeout cannot pretend those owners have stopped. Upstream must choose between continued charged ownership after a reported shutdown failure, a stronger isolation model, or termination of the owning process. That operational decision remains open and must accompany any promise about bounded shutdown. [Lifecycle module](#).

Keep graph helpers distinct from a generic SDK

PROTOTYPE also compiles GraphUtils into `sail-session`: graph-client staging, owned run cleanup and helper functions. It deliberately uses the host's object store and credentials rather than passing them into a wheel. Its configured root and request type are domain-facing facilities, not necessities for every extension. [PROTOTYPE GraphUtils](#).

This distinction prevents the physical location of code from deciding its architectural status. A helper can use generic host hooks while remaining a separate integration. Sedona's metadata handling similarly demonstrates useful requirements without making every geometry-specific decision part of the core contract. Review generic invariants with small fixtures first, then review the domain adapters that exercise them.

A phased route to upstream review

1. **Rebase the reasoning before the code.** Compare each proposed change with PLAIN, preserve newer upstream behavior, and identify the smallest current reproduction or missing interface. The historical patch is an inventory, not a merge verdict.
2. **Separate general corrections.** Propose lifecycle and field-preservation changes with regressions that do not require the full loader. State which ordinary execution paths change and test them on the proposed current tree.
3. **Review bounded host interfaces.** Settle envelope validation, handler dispatch, ownership and host-context guarantees. Review the resource ABI separately from admission-domain policy and shutdown deadlines.
4. **Integrate package loading and worker identity.** Define collision rules, qualified versions, failure before partial registration, and package installation expectations. Exercise real driver/worker codec round trips.
5. **Add policy and domain integrations deliberately.** Qualify the initial driver-only relation policy, then assess stateless worker execution and replay-safe classes independently. Keep GraphUtils and recoverable graph state outside the minimum SDK proposal.

Every proposed commit needs its own current-source tests; combinations need a combined gate. Nothing here claims that the historical patch applies cleanly, that all prototype choices should be upstreamed, or that source inspection qualifies a cluster deployment. The goal is a reviewable host contract whose implementation can evolve without forcing extensions to depend on Sail's private session, scheduler or pool machinery.

Complete host integration patch

This appendix contains every changed line, with unified context, under `crates/` and the workspace Cargo manifests/lockfile between the fork's actual upstream base and the reviewed prototype. It includes generic integration, domain-specific GraphUtils helpers, tests and dependency changes; inclusion is not a proposal that every file belongs in the minimum extension API.

Patch base: `a85d912d72ae03a6d97b6a3fd151f5752da636c6`. Patch result: `bd8ce9ae8839477e2c08a0475ab7900b115c5366`. Present plain-Sail comparison: `4d31e15b350c975aed95c23e6cc7e7c51c59fe52`.

This is a complete historical integration patch, not a patch against present plain Sail. It has not been rebased or qualified against that newer revision. The source bundle also contains the complete affected files at each of those three revisions when the file exists. Tests in this appendix are source evidence; this document build does not rerun their historical runtime gates.

01. Cargo.lock

```
1 diff --git a/Cargo.lock b/Cargo.lock
2 index 60a069a1da595a0e0cea905daf9f97dc7f8a6a04..0fac74dc746d3eacb4c77d4c3b4306f4e2602f
   de 100644
3 --- a/Cargo.lock
4 +++ b/Cargo.lock
5 @@ -133,7 +133,7 @@ version = "1.1.5"
6   source = "registry+https://github.com/rust-lang/crates.io-index"
7   checksum = "40c48f72fd53cd289104fc64099abca73db4166ad86ea0b4341abe65af83dad"
8   dependencies = [
9     - "windows-sys 0.60.2",
10    + "windows-sys 0.61.2",
11   ]
12
13   [[package]]
14 @@ -144,7 +144,7 @@ checksum = "291e6a250ff86cd4a820112fb8898808a366d8f9f58ce16d1f5383
   53ad55747d"
15   dependencies = [
16     "anstyle",
17     "once_cell_polyfill",
18     - "windows-sys 0.60.2",
19     + "windows-sys 0.61.2",
20   ]
21
22   [[package]]
23 @@ -220,9 +220,9 @@ checksum = "d3fb67a6e08acf24fdeccbac2cb6ac4305825bd1f117462e0e6f2f
   193345ad56"
24
25   [[package]]
26   name = "arrow"
27   -version = "59.2.0"
28   +version = "59.3.0"
29   source = "registry+https://github.com/rust-lang/crates.io-index"
30   -checksum = "61d285d16bce7d0be61912f7928342b673067b6b7d7ef6cc179258ba7de1fecf"
31   +checksum = "7c14b3d39f306bc28fd639d59f06e17a0f377d0021e1b7e9054e4d6fedc98774"
32   dependencies = [
33     "arrow-arith",
34     "arrow-array",
35 @@ -241,9 +241,9 @@ dependencies = [
36
37   [[package]]
38   name = "arrow-arith"
39   -version = "59.2.0"
40   +version = "59.3.0"
41   source = "registry+https://github.com/rust-lang/crates.io-index"
42   -checksum = "757ef1836251e88222542a7da2623bc1c9cb9e20afefa6db2c41e79991cd91d4"
43   +checksum = "ce2961626677665b2195eb59242af4c7befe7b8737ca2050295389362380104e"
44   dependencies = [
45     "arrow-array",
46     "arrow-buffer",
47 @@ -255,9 +255,9 @@ dependencies = [
48
49   [[package]]
50   name = "arrow-array"
51   -version = "59.2.0"
52   +version = "59.3.0"
```

```

53 source = "registry+https://github.com/rust-lang/crates.io-index"
54 -checksum = "bc9a4a4b2b5ecd0e04df03471661cb61f28bed3c7fd50994715129b01b2edb97"
55 +checksum = "1e5f6adefdf587d7a31db5d2266189624b526730cd3627f9ff9fedae97ad584"
56 dependencies = [
57     "ahash",
58     "arrow-buffer",
59 @@ -275,9 +275,9 @@ dependencies = [
60
61 [[package]]
62 name = "arrow-avro"
63 -version = "59.2.0"
64 +version = "59.3.0"
65 source = "registry+https://github.com/rust-lang/crates.io-index"
66 -checksum = "9fb45cd6bd2b25c0965793b83200eaca82214273a8030fbbc2d783e4c7c65a61"
67 +checksum = "9145b685d586482102e4fe3d40349fe00086c0b219d51cae9daae4698849ea"
68 dependencies = [
69     "arrow-array",
70     "arrow-buffer",
71 @@ -299,9 +299,9 @@ dependencies = [
72
73 [[package]]
74 name = "arrow-buffer"
75 -version = "59.2.0"
76 +version = "59.3.0"
77 source = "registry+https://github.com/rust-lang/crates.io-index"
78 -checksum = "c12b576ef18c1deb80925a248b25ad84f419198d791b8e293fc6aaa60441fe90"
79 +checksum = "097d193003ce7995d5d087089069ec2a6e0187faf5a6f8c9f38af2645d987182"
80 dependencies = [
81     "bytes",
82     "half",
83 @@ -311,9 +311,9 @@ dependencies = [
84
85 [[package]]
86 name = "arrow-cast"
87 -version = "59.2.0"
88 +version = "59.3.0"
89 source = "registry+https://github.com/rust-lang/crates.io-index"
90 -checksum = "68338a9096a5dc9bc11927c58c43a8526d96bf6abd2012ef6c0c9f505991cc79"
91 +checksum = "635c9c635668ad26adf76cce8fb276c4be7cf06e63bd516de7da514f9680ee53"
92 dependencies = [
93     "arrow-array",
94     "arrow-buffer",
95 @@ -333,9 +333,9 @@ dependencies = [
96
97 [[package]]
98 name = "arrow-csv"
99 -version = "59.2.0"
100 +version = "59.3.0"
101 source = "registry+https://github.com/rust-lang/crates.io-index"
102 -checksum = "25011b52b346407d497ef0030e12b45e4f2d0cc279efc09c4f3d09106db30e36"
103 +checksum = "4c2ebf8d631e79b02c16cf5ae860561272c26024ec88fce389a56aadd558e86"
104 dependencies = [
105     "arrow-array",
106     "arrow-cast",
107 @@ -348,9 +348,9 @@ dependencies = [
108
109 [[package]]
110 name = "arrow-data"
111 -version = "59.2.0"
112 +version = "59.3.0"
113 source = "registry+https://github.com/rust-lang/crates.io-index"
114 -checksum = "723fe4aeed7604e00b9883a465af4ff0a0e6c44c03e41a68c3d1cbc403e0e44d"
115 +checksum = "9ba2f832eaecca24b8f26143dba750e42ee4ab51cf7d65e701ca9607cfda9f358"
116 dependencies = [
117     "arrow-buffer",
118     "arrow-schema",
119 @@ -361,9 +361,9 @@ dependencies = [
120
121 [[package]]
122 name = "arrow-flight"
123 -version = "59.2.0"
124 +version = "59.3.0"
125 source = "registry+https://github.com/rust-lang/crates.io-index"
126 -checksum = "2bebfacc9d71f0728f6774164e4d4254b5e504d2b46812d0512d8290ec119a64"
127 +checksum = "be0e6d452fff35cb4a3ef1a719d03fdfe653e590e8012b57d3057a316e295402"
128 dependencies = [
129     "arrow-arith",
130     "arrow-array",

```

```

131 @@ -388,9 +388,9 @@ dependencies = [
132
133   [[package]]
134   name = "arrow-ipc"
135   -version = "59.2.0"
136   +version = "59.3.0"
137   source = "registry+https://github.com/rust-lang/crates.io-index"
138   -checksum = "149437b14371f5b9ec60f5ddc751483ae99d7a7072653c0075e5e469156eea7b"
139   +checksum = "dcc41681ea80f521df14c36725b74d4c60702c47f0793af2be469c04527e2599"
140   dependencies = [
141     "arrow-array",
142     "arrow-buffer",
143   @@ -404,9 +404,9 @@ dependencies = [
144
145   [[package]]
146   name = "arrow-json"
147   -version = "59.2.0"
148   +version = "59.3.0"
149   source = "registry+https://github.com/rust-lang/crates.io-index"
150   -checksum = "f18b9123ccfec418a663f821c9a034af339711678c11ffe00d3ec07da5ff9f7e"
151   +checksum = "a2f57d7a81969f24ccf80809587b76c09897e6f829d2d65a5976bfb3218851f1"
152   dependencies = [
153     "arrow-array",
154     "arrow-buffer",
155   @@ -429,9 +429,9 @@ dependencies = [
156
157   [[package]]
158   name = "arrow-ord"
159   -version = "59.2.0"
160   +version = "59.3.0"
161   source = "registry+https://github.com/rust-lang/crates.io-index"
162   -checksum = "e6c08dff0686cf23ca4f562803f191ccbeb726dbae6309cd4b4aaf65e0f2c979"
163   +checksum = "2c900759f3bd8354fd4196bc4403eee846894dc2adf66b4225472006a0bf18c5"
164   dependencies = [
165     "arrow-array",
166     "arrow-buffer",
167   @@ -442,9 +442,9 @@ dependencies = [
168
169   [[package]]
170   name = "arrow-pyarrow"
171   -version = "59.2.0"
172   +version = "59.3.0"
173   source = "registry+https://github.com/rust-lang/crates.io-index"
174   -checksum = "c196ecc25b3a8dcbcl842f2619cee653dcfa2fb8b56a291bc0481c3cf5c3821"
175   +checksum = "801aed7e607dbcc60ed6359025383b27a490327d4282ba87a9a3b78e25959c9a"
176   dependencies = [
177     "arrow-array",
178     "arrow-data",
179   @@ -454,9 +454,9 @@ dependencies = [
180
181   [[package]]
182   name = "arrow-row"
183   -version = "59.2.0"
184   +version = "59.3.0"
185   source = "registry+https://github.com/rust-lang/crates.io-index"
186   -checksum = "bbec439386df71ad570e6758a94611322b9e9dc8db83b5527321f0b4c9119c2"
187   +checksum = "f4c6425032e28266e3fc4ff680805e57e670d6ea92473043f3e65b7ed6ac79f2"
188   dependencies = [
189     "arrow-array",
190     "arrow-buffer",
191   @@ -467,9 +467,9 @@ dependencies = [
192
193   [[package]]
194   name = "arrow-schema"
195   -version = "59.2.0"
196   +version = "59.3.0"
197   source = "registry+https://github.com/rust-lang/crates.io-index"
198   -checksum = "e6fed2ca0dleade57e811cbe73b98ad50cc08a1183e13b2d2aa43a7df593f40e"
199   +checksum = "10fab8d4563491417ba801fab29d205104d20d4bdf37bda6cd1cf425cff598cd"
200   dependencies = [
201     "bitflags 2.13.2",
202     "serde",
203   @@ -479,9 +479,9 @@ dependencies = [
204
205   [[package]]
206   name = "arrow-select"
207   -version = "59.2.0"
208   +version = "59.3.0"

```

```

209 source = "registry+https://github.com/rust-lang/crates.io-index"
210 -checksum = "466b19cf75130b891dc1b23a84b343c714c62c64c9c62e365c76aa0ff90a53fb"
211 +checksum = "fc58569193c2525915f3cc6310edba3792f1200f65d6e9ed330aa33e691493b8"
212 dependencies = [
213     "ahash",
214     "arrow-array",
215 @@ -493,9 +493,9 @@ dependencies = [
216
217     [[package]]
218     name = "arrow-string"
219     -version = "59.2.0"
220     +version = "59.3.0"
221     source = "registry+https://github.com/rust-lang/crates.io-index"
222     -checksum = "c838a25bb3691e919e0f617616ac51a4ff8517a952e29ca133cf0c22b2ce65b1"
223     +checksum = "2e0813f3c35c1cfea65e14c20a953440f7783c088b7ad2d0db162ccdeefcec14"
224     dependencies = [
225         "arrow-array",
226         "arrow-buffer",
227 @@ -542,6 +542,12 @@ dependencies = [
228         "tokio",
229     ]
230
231 +[[package]]
232 +name = "async-ffi"
233 +version = "0.5.1"
234 +source = "registry+https://github.com/rust-lang/crates.io-index"
235 +checksum = "39cd9de47399986d5b216c6bef9434dfff1689ab61ba8d1e2720dc5fe5c84083"
236 +
237 [[package]]
238 name = "async-lock"
239 version = "3.4.2"
240 @@ -2372,6 +2378,39 @@ dependencies = [
241     "itertools 0.15.0",
242 ]
243
244 +[[package]]
245 +name = "datafusion-ffi"
246 +version = "55.1.0"
247 +source = "registry+https://github.com/rust-lang/crates.io-index"
248 +checksum = "77a9e527ab3fd2cealc216efb81c40684a54490489d4ee6cd5719b17f8442f35"
249 +dependencies = [
250 +    "arrow",
251 +    "arrow-schema",
252 +    "async-ffi",
253 +    "async-trait",
254 +    "chrono",
255 +    "datafusion-catalog",
256 +    "datafusion-common",
257 +    "datafusion-datasource",
258 +    "datafusion-execution",
259 +    "datafusion-expr",
260 +    "datafusion-functions-aggregate-common",
261 +    "datafusion-physical-expr",
262 +    "datafusion-physical-expr-common",
263 +    "datafusion-physical-optimizer",
264 +    "datafusion-physical-plan",
265 +    "datafusion-proto",
266 +    "datafusion-proto-common",
267 +    "datafusion-session",
268 +    "futures",
269 +    "libloading 0.9.0",
270 +    "log",
271 +    "prost",
272 +    "semver",
273 +    "stabby",
274 +    "tokio",
275 +]
276 +
277 [[package]]
278 name = "datafusion-functions"
279 version = "55.1.0"
280 @@ -2915,7 +2954,7 @@ dependencies = [
281     "libc",
282     "option-ext",
283     "redox_users",
284     - "windows-sys 0.60.2",
285     + "windows-sys 0.61.2",
286 ]

```

```

287
288 [[package]]
289 @@ -3081,7 +3120,7 @@ source = "registry+https://github.com/rust-lang/crates.io-index"
290 checksum = "39cab71617ae0d63f51a36d69f866391735b51691dbda63cf6f96d042b63efeb"
291 dependencies = [
292     "libc",
293     - "windows-sys 0.52.0",
294     + "windows-sys 0.61.2",
295 ]
296
297 [[package]]
298 @@ -4599,7 +4638,7 @@ checksum = "3640c1c38b8e4e43584d8df18be5fc6b0aa314ce6ebf51b53313
d4306cca8e46"
299 dependencies = [
300     "hermit-abi",
301     "libc",
302     - "windows-sys 0.52.0",
303     + "windows-sys 0.61.2",
304 ]
305
306 [[package]]
307 @@ -5453,7 +5492,7 @@ version = "0.50.3"
308 source = "registry+https://github.com/rust-lang/crates.io-index"
309 checksum = "7957b9740744892f114936ab4a57b3f487491bbeafaf8083688b16841a4240e5"
310 dependencies = [
311     - "windows-sys 0.60.2",
312     + "windows-sys 0.61.2",
313 ]
314
315 [[package]]
316 @@ -5825,9 +5864,9 @@ dependencies = [
317
318 [[package]]
319 name = "parquet"
320 -version = "59.2.0"
321 +version = "59.3.0"
322 source = "registry+https://github.com/rust-lang/crates.io-index"
323 -checksum = "7065842956a20c2a536924ce8e4d9955f7422451511b9eb7500d7bfe5077e59c"
324 +checksum = "ff322f54b1a0f9288e614ed1f2d329b380af5476420db19f46ffb865e1163d73"
325 dependencies = [
326     "ahash",
327     "arrow-array",
328 @@ -5859,9 +5898,9 @@ dependencies = [
329
330 [[package]]
331 name = "parquet-variant"
332 -version = "59.2.0"
333 +version = "59.3.0"
334 source = "registry+https://github.com/rust-lang/crates.io-index"
335 -checksum = "3f7e5fff3ed0c07514a7fb8bee3f2ea5a53f36939410ecac4a466620213539a8"
336 +checksum = "3950502c5d383bc98ac83424c50ad06214cfb8a8554ef6f19413e71894079288"
337 dependencies = [
338     "arrow",
339     "arrow-schema",
340 @@ -5875,9 +5914,9 @@ dependencies = [
341
342 [[package]]
343 name = "parquet-variant-compute"
344 -version = "59.2.0"
345 +version = "59.3.0"
346 source = "registry+https://github.com/rust-lang/crates.io-index"
347 -checksum = "ba4d3de89dab8d1aaaf601ae8d71bd07ea88cfca9efc1df5815b982c30f631e1"
348 +checksum = "28c4fc004d60a76d727fe4a383b48e8c4d629384b9d9f0cc359164ec2cbd3326"
349 dependencies = [
350     "arrow",
351     "arrow-schema",
352 @@ -5892,9 +5931,9 @@ dependencies = [
353
354 [[package]]
355 name = "parquet-variant-json"
356 -version = "59.2.0"
357 +version = "59.3.0"
358 source = "registry+https://github.com/rust-lang/crates.io-index"
359 -checksum = "fb19dfelbd24c17addd761ba4f7000f615e2fa12525871c7baa835dbb3d7f147"
360 +checksum = "216b1aeca3ee2e8f429ade4dfef24e233d008ee91ce745fb273c39fc5936b85"
361 dependencies = [
362     "arrow-schema",
363     "base64 0.23.1",

```

```

364 @@ -6728,7 +6767,7 @@ dependencies = [
365     "once_cell",
366     "socket2",
367     "tracing",
368     - "windows-sys 0.52.0",
369     + "windows-sys 0.61.2",
370 ]
371
372 [[package]]
373 @@ -7281,7 +7320,7 @@ dependencies = [
374     "errno",
375     "libc",
376     "linux-raw-sys 0.12.1",
377     - "windows-sys 0.52.0",
378     + "windows-sys 0.61.2",
379 ]
380
381 [[package]]
382 @@ -7615,6 +7654,7 @@ dependencies = [
383     "pin-project-lite",
384     "prost",
385     "sail-common",
386     + "sail-native-resource-ffi",
387     "serde",
388     "serde_arrow",
389     "serde_json",
390 @@ -7935,6 +7975,10 @@ dependencies = [
391     "sail-function",
392 ]
393
394 +[[package]]
395 +name = "sail-native-resource-ffi"
396 +version = "0.1.0"
397 +
398 [[package]]
399 name = "sail-object-store"
400 version = "0.7.1"
401 @@ -8100,18 +8144,23 @@ dependencies = [
402     name = "sail-session"
403     version = "0.7.1"
404     dependencies = [
405     + "arrow-schema",
406     "async-trait",
407     "chrono",
408     "datafusion",
409     "datafusion-common",
410     "datafusion-datasource",
411     "datafusion-expr",
412     + "datafusion-ffi",
413     "datafusion-physical-expr",
414     "fastrace",
415     "futures",
416     "indexmap 2.14.2",
417     "log",
418     "object_store",
419     + "prost",
420     + "prost-build",
421     + "pyo3",
422     "readonly",
423     "sail-cache",
424     "sail-catalog",
425     @@ -8138,10 +8187,13 @@ dependencies = [
426     "sail-system-store",
427     "sail-telemetry",
428     "secrecy",
429     + "serde",
430     + "serde_json",
431     "tempfile",
432     "thiserror 2.0.20",
433     "tokio",
434     "tonic",
435     + "url",
436     "uuid",
437 ]
438
439 @@ -8181,6 +8233,7 @@ dependencies = [
440     "sail-telemetry",
441     "serde",

```

```

442 "serde_json",
443 + "stacker",
444 "syn 2.0.119",
445 "thiserror 2.0.20",
446 "tokio",
447 @@ -8437,7 +8490,7 @@ source = "registry+https://github.com/rust-lang/crates.io-index"
448 checksum = "5b55fb86dfd3a2f5f76ea78310a88f96c4ea21a3031f8d212443d56123fd0521"
449 dependencies = [
450 "libc",
451 - "windows-sys 0.52.0",
452 + "windows-sys 0.61.2",
453 ]
454
455 [[package]]
456 @@ -8668,6 +8721,12 @@ dependencies = [
457 "digest 0.11.3",
458 ]
459
460 +[[package]]
461 +name = "sha2-const-stable"
462 +version = "0.1.0"
463 +source = "registry+https://github.com/rust-lang/crates.io-index"
464 +checksum = "5f179d4e11094a893b82fff208f74d448a7512f99f5a0acbd5c679b705f83ed9"
465 +
466 [[package]]
467 name = "sharded-slab"
468 version = "0.1.7"
469 @@ -8745,7 +8804,7 @@ source = "registry+https://github.com/rust-lang/crates.io-index"
470 checksum = "c3d1e2c7f27f8d4cb10542a02c49005dbd6e93095799d6f3be745fae9f8fedd4"
471 dependencies = [
472 "libc",
473 - "windows-sys 0.60.2",
474 + "windows-sys 0.61.2",
475 ]
476
477 [[package]]
478 @@ -8818,6 +8877,40 @@ dependencies = [
479 "syn 2.0.119",
480 ]
481
482 +[[package]]
483 +name = "stabby"
484 +version = "72.1.16"
485 +source = "registry+https://github.com/rust-lang/crates.io-index"
486 +checksum = "3d53d2428934c46277fafd2d41e39357595aa1e47954c75db2b14ed90632f3cc"
487 +dependencies = [
488 + "rustversion",
489 + "stabby-abi",
490 +]
491 +
492 +[[package]]
493 +name = "stabby-abi"
494 +version = "72.1.16"
495 +source = "registry+https://github.com/rust-lang/crates.io-index"
496 +checksum = "f375eae680bb54203ee5e47d4cd2ae7b79c0a79ed90919279f38f500ad53f190"
497 +dependencies = [
498 + "rustc_version",
499 + "rustversion",
500 + "sha2-const-stable",
501 + "stabby-macros",
502 +]
503 +
504 +[[package]]
505 +name = "stabby-macros"
506 +version = "72.1.16"
507 +source = "registry+https://github.com/rust-lang/crates.io-index"
508 +checksum = "ea664671a576c5f7e32fee291ac123d82af5e92b0689beb355347c00c76eef1"
509 +dependencies = [
510 + "proc-macro-crate",
511 + "proc-macro2",
512 + "quote",
513 + "syn 2.0.119",
514 +]
515 +
516 [[package]]
517 name = "stable_deref_trait"
518 version = "1.2.1"
519 @@ -8834,7 +8927,7 @@ dependencies = [

```

```

520     "cfg-if",
521     "libc",
522     "psm",
523 - "windows-sys 0.60.2",
524 + "windows-sys 0.61.2",
525 ]
526
527 [[package]]
528 @@ -8998,10 +9091,10 @@ source = "registry+https://github.com/rust-lang/crates.io-index"
529     checksum = "32497e9a4c7b38532efcdebeef879707aa9f794296a4f0244f6f69e9bc8574bd"
530     dependencies = [
531         "fastrand",
532 - "getrandom 0.3.4",
533 + "getrandom 0.4.3",
534         "once_cell",
535         "rustix 1.1.4",
536 - "windows-sys 0.52.0",
537 + "windows-sys 0.61.2",
538     ]
539
540 [[package]]
541 @@ -9542,7 +9635,7 @@ version = "2.1.4"
542     source = "registry+https://github.com/rust-lang/crates.io-index"
543     checksum = "5283634e518fe9e82c7b20520bb4bc209009fd16c82077c802f811ecbb0117a"
544     dependencies = [
545 - "rand 0.9.5",
546 + "rand 0.10.3",
547     ]
548
549 [[package]]
550 @@ -9997,7 +10090,7 @@ version = "0.1.11"
551     source = "registry+https://github.com/rust-lang/crates.io-index"
552     checksum = "c2a7b1c03c876122aa43f3020e6c3c3ee5c05081c9a00739faf7503aeba10d22"
553     dependencies = [
554 - "windows-sys 0.52.0",
555 + "windows-sys 0.61.2",
556     ]
557
558 [[package]]

```

02. Cargo.toml

```

1 diff --git a/Cargo.toml b/Cargo.toml
2 index b2865f9af30d792109e9b2d91bfe8c803c15ba79..bc6e5aff18d50475eaa55ae64597da8f11a69e
   fa 100644
3 --- a/Cargo.toml
4 +++ b/Cargo.toml
5 @@ -23,6 +23,7 @@ dbg_macro = "deny"
6     todo = "deny"
7
8     [workspace.dependencies]
9 +stacker = "0.1.25"
10     thiserror = { version = "2.0.18" }
11     tokio = { version = "1.52.3", features = ["full"] }
12     tokio-util = { version = "0.7.19", features = ["rt"] }
13 @@ -162,7 +163,7 @@ prost-types = "0.14"
14     # The `axum` version must match the one used in `tonic` (replace `RELEASE` with the r
   elease we are using):
15     # https://github.com/hyperium/tonic/blob/vRELEASE/tonic/Cargo.toml
16     axum = "0.8.9"
17 -datafusion = { version = "55.1.0", default-features = false, features = [
18 +datafusion = { version = "55.1.0", default-features = false, features = [
19     "nested_expressions",
20     "crypto_expressions",
21     "datetime_expressions",
22 @@ -176,32 +177,33 @@ datafusion = { version = "55.1.0", default-features = false, fea
   tures = [
23     "avro",
24     "recursive_protection",
25 ] }
26 -datafusion-common = { version = "55.1.0", features = ["object_store"] }
27 -datafusion-datasource = { version = "55.1.0" }
28 -datafusion-datasource-avro = { version = "55.1.0" }
29 -datafusion-datasource-json = { version = "55.1.0" }

```

```

30 -datafusion-datasource-parquet = { version = "55.1.0" }
31 -datafusion-expr = { version = "55.1.0", default-features = false }
32 -datafusion-expr-common = { version = "55.1.0" }
33 -datafusion-proto = { version = "55.1.0" }
34 -datafusion-functions = { version = "55.1.0" }
35 -datafusion-functions-nested = { version = "55.1.0", default-features = false }
36 -datafusion-physical-expr = { version = "55.1.0" }
37 -datafusion-session = { version = "55.1.0" }
38 -datafusion-spark = { version = "55.1.0", features = ["core"] }
39 +datafusion-common = { version = "55.1.0", features = ["object_store"] }
40 +datafusion-datasource = { version = "55.1.0" }
41 +datafusion-datasource-avro = { version = "55.1.0" }
42 +datafusion-datasource-json = { version = "55.1.0" }
43 +datafusion-datasource-parquet = { version = "55.1.0" }
44 +datafusion-expr = { version = "55.1.0", default-features = false }
45 +datafusion-expr-common = { version = "55.1.0" }
46 +datafusion-ffi = { version = "55.1.0" }
47 +datafusion-proto = { version = "55.1.0" }
48 +datafusion-functions = { version = "55.1.0" }
49 +datafusion-functions-nested = { version = "55.1.0", default-features = false }
50 +datafusion-physical-expr = { version = "55.1.0" }
51 +datafusion-session = { version = "55.1.0" }
52 +datafusion-spark = { version = "55.1.0", features = ["core"] }
53   pyo3 = { version = "0.29.0", features = ["serde"] }
54   jiter = { version = "0.15.0", default-features = false }
55 -arrow = { version = "59.2.0", features = ["chrono-tz"] }
56 -arrow-array = { version = "59.2.0", features = ["ffi"] }
57 -arrow-buffer = { version = "59.2.0" }
58 -arrow-data = { version = "59.2.0" }
59 -arrow-schema = { version = "59.2.0", features = ["serde"] }
60 -arrow-flight = { version = "59.2.0", features = ["flight-sql-experimental"] }
61 -arrow-pyarrow = { version = "59.2.0" }
62 -parquet = { version = "59.2.0" }
63 -parquet-variant = { version = "59.2.0" }
64 -parquet-variant-compute = { version = "59.2.0" }
65 -parquet-variant-json = { version = "59.2.0" }
66 +arrow = { version = "59.3.0", features = ["chrono-tz"] }
67 +arrow-array = { version = "59.3.0", features = ["ffi"] }
68 +arrow-buffer = { version = "59.3.0" }
69 +arrow-data = { version = "59.3.0" }
70 +arrow-schema = { version = "59.3.0", features = ["serde"] }
71 +arrow-flight = { version = "59.3.0", features = ["flight-sql-experimental"] }
72 +arrow-pyarrow = { version = "59.3.0" }
73 +parquet = { version = "59.3.0" }
74 +parquet-variant = { version = "59.3.0" }
75 +parquet-variant-compute = { version = "59.3.0" }
76 +parquet-variant-json = { version = "59.3.0" }
77   serde_arrow = { version = "0.14.2", features = ["arrow-59"] }
78   # The `object_store` version must match the one used in DataFusion.
79   object_store = { version = "0.13.2", features = ["aws", "gcp", "azure", "http"] }

```

03. crates/sail-common-datafusion/Cargo.toml

```

1 diff --git a/crates/sail-common-datafusion/Cargo.toml b/crates/sail-common-datafusion/
  Cargo.toml
2 index 3e5373a5690077a161d2221d8b81fddca9618ed1..85c1fd71a06c7d21ab228c91920591f2c7349f
  5e 100644
3 --- a/crates/sail-common-datafusion/Cargo.toml
4 +++ b/crates/sail-common-datafusion/Cargo.toml
5 @@ -8,6 +8,7 @@ workspace = true
6
7 [dependencies]
8   sail-common = { path = "../sail-common" }
9 +sail-native-resource-ffi = { path = "../sail-native-resource-ffi" }
10
11   chrono = { workspace = true }
12   datafusion = { workspace = true }

```

04. crates/sail-common-datafusion/src/connect_extension.rs

```

1 diff --git a/crates/sail-common-datafusion/src/connect_extension.rs b/crates/sail-comm
  on-datafusion/src/connect_extension.rs
2 new file mode 100644

```

```

3 index 0000000000000000000000000000000000000000..89941ca59257ee97f667f7443a1a9b8bb0f110
4 6a
5 --- /dev/null
6 +++ b/crates/sail-common-datafusion/src/connect_extension.rs
7 @@ -0,0 +1,257 @@
8 +#!/ Internal host adapters for the experimental local Connect extension API.
9 +#!/
10 +#!/ This Rust interface stays inside Sail. Native packages cross the independently
11 +#!/ version-checked DataFusion FFI boundary in the package loader, not this trait.
12 +
13 +use std::collections::BTreeMap;
14 +use std::fmt::{Debug, Formatter};
15 +use std::pin::Pin;
16 +use std::sync::Arc;
17 +use std::task::{Context, Poll};
18 +
19 +use datafusion::arrow::datatypes::SchemaRef;
20 +use datafusion::arrow::record_batch::RecordBatch;
21 +use datafusion::catalog::TableProvider;
22 +use datafusion::execution::{RecordBatchStream, SendableRecordBatchStream, TaskContext};
23 +
24 +use datafusion::physical_expr::PhysicalExpr;
25 +use datafusion::physical_plan::coalesce_partitions::CoalescePartitionsExec;
26 +use datafusion::physical_plan::{
27 +    DisplayAs, DisplayFormatType, ExecutionPlan, PlanProperties, ReplaceChildrenOptions,
28 +};
29 +use datafusion_common::tree_node::TreeNodeRecursion;
30 +use datafusion_common::{Result, exec_err, plan_err};
31 +use futures::Stream;
32 +use tokio::runtime::Handle;
33 +
34 +use crate::extension::SessionExtension;
35 +
36 +/// Planning must only describe execution: it must not consume input or mutate
37 +/// extension state. AnalyzePlan can invoke this method without executing a query.
38 +pub trait ConnectRelationHandler: Send + Sync + 'static {
39 +    fn plan(
40 +        &self,
41 +        payload: &[u8],
42 +        inputs: Vec<Arc<dyn ExecutionPlan>>,
43 +    ) -> Result<Arc<dyn TableProvider>>;
44 +}
45 +
46 +struct RegisteredHandler {
47 +    accepts_bare: bool,
48 +    min_inputs: usize,
49 +    max_inputs: usize,
50 +    handler: Arc<dyn ConnectRelationHandler>,
51 +}
52 +
53 +/// One registry is bound to one server session. Package discovery may be shared;
54 +/// handlers and the graph state they own must be instantiated for this session.
55 +#[derive(Default)]
56 +pub struct ConnectExtensionRegistry {
57 +    handlers: BTreeMap<String, RegisteredHandler>,
58 +}
59 +
60 +impl ConnectExtensionRegistry {
61 +    pub fn new() -> Self {
62 +        Self::default()
63 +    }
64 +
65 +    pub fn register(
66 +        &mut self,
67 +        type_url: String,
68 +        accepts_bare: bool,
69 +        min_inputs: usize,
70 +        max_inputs: usize,
71 +        handler: Arc<dyn ConnectRelationHandler>,
72 +    ) -> Result<> {
73 +        if type_url.is_empty() || type_url.len() > 512 {
74 +            return plan_err!("extension type URL must contain between 1 and 512 bytes");
75 +        }
76 +
77 +        if min_inputs > max_inputs || (accepts_bare && min_inputs != 0) {
78 +            return plan_err!("invalid input arity registration for extension {type_url}");
79 +        }
80 +
81 +        self.handlers.insert(type_url, RegisteredHandler {
82 +            accepts_bare,
83 +            min_inputs,
84 +            max_inputs,
85 +            handler,
86 +        });
87 +    }
88 +
89 +    pub fn get(&self, type_url: &str) -> Option<RegisteredHandler> {
90 +        self.handlers.get(type_url).cloned()
91 +    }
92 +
93 +    pub fn iter_handlers(&self) -> impl Iterator<RegisteredHandler> {
94 +        self.handlers.iter().map(|(_, h)| h.clone())
95 +    }
96 +}

```

```

76 +     }
77 +     if self.handlers.contains_key(&type_url) {
78 +         return plan_err!("duplicate Connect extension type URL: {type_url}");
79 +     }
80 +     self.handlers.insert(
81 +         type_url,
82 +         RegisteredHandler {
83 +             accepts_bare,
84 +             min_inputs,
85 +             max_inputs,
86 +             handler,
87 +         },
88 +     );
89 +     Ok(())
90 + }
91 +
92 + /// Validate before planning children, so invalid requests cannot trigger
93 + /// planning callbacks in their nested extensions.
94 + pub fn resolve(
95 +     &self,
96 +     type_url: &str,
97 +     is_envelope: bool,
98 +     input_count: usize,
99 + ) -> Result<Arc<dyn ConnectRelationHandler>> {
100 +     let Some(entry) = self.handlers.get(type_url) else {
101 +         let registered = self.handlers.keys().cloned().collect::<Vec<_>>().join("
102 +         , ");
103 +         return plan_err!(
104 +             "unregistered Connect extension type URL: {type_url}; registered: [{r
105 +             egistered}]");
106 +     };
107 +     if !is_envelope && !entry.accepts_bare {
108 +         return plan_err!(
109 +             "Connect extension {type_url} requires a SailExtensionRequest envelop
110 +             e");
111 +     };
112 +     if !is_envelope && input_count != 0 {
113 +         return plan_err!("bare Connect extension {type_url} cannot have inputs");
114 +     };
115 +     if !(entry.min_inputs..=entry.max_inputs).contains(&input_count) {
116 +         return plan_err!(
117 +             "Connect extension {type_url} expects {}.{} inputs, received {input
118 +             _count}",
119 +             entry.min_inputs,
120 +             entry.max_inputs
121 +         );
122 +     };
123 +     Ok(Arc::clone(&entry.handler))
124 + }
125 +
126 + impl SessionExtension for ConnectExtensionRegistry {
127 +     fn name() -> &'static str {
128 +         "ConnectExtensionRegistry"
129 +     }
130 + }
131 +
132 + /// Executes a previously planned Sail input under its original host context.
133 + /// DataFusion 55's foreign TaskContext reconstruction does not preserve Sail's
134 + /// RuntimeEnv. A foreign consumer therefore receives this wrapper, whose execute
135 + /// ignores the reconstructed context. Coalescing happens on the host and consumes
136 + /// every partition. The host runtime is retained and entered for execution and
137 + /// stream polls: FFI child replacement can discard the FFI wrapper's runtime.
138 + /// This is a local-mode bridge, not a serializable remote node.
139 + pub struct HostInputExec {
140 +     input: Arc<dyn ExecutionPlan>,
141 +     context: Arc<TaskContext>,
142 +     runtime: Handle,
143 + }
144 +
145 + impl HostInputExec {
146 +     pub fn gathered_input(&self) -> Arc<dyn ExecutionPlan> {
147 +         self.input.clone()
148 +     }
149 + }

```

```

150 + pub fn new(input: Arc<dyn ExecutionPlan>, context: Arc<TaskContext>, runtime: Han
dle) -> Self {
151 +     Self {
152 +         input: Arc::new(CoalescePartitionsExec::new(input)),
153 +         context,
154 +         runtime,
155 +     }
156 + }
157 +}
158 +
159 +impl Debug for HostInputExec {
160 +    fn fmt(&self, f: &mut Formatter<'>) -> std::fmt::Result {
161 +        f.debug_struct("HostInputExec")
162 +            .field("input", &self.input)
163 +            .finish_non_exhaustive()
164 +    }
165 +}
166 +
167 +impl DisplayAs for HostInputExec {
168 +    fn fmt_as(&self, _t: DisplayFormatType, f: &mut Formatter<'>) -> std::fmt::Resul
t {
169 +        write!(
170 +            f,
171 +            "HostInputExec: local, host_context=true, host_runtime=true, partitions=1
"
172 +        )
173 +    }
174 +}
175 +
176 +impl ExecutionPlan for HostInputExec {
177 +    fn name(&self) -> &'static str {
178 +        "HostInputExec"
179 +    }
180 +
181 +    fn properties(&self) -> &Arc<PlanProperties> {
182 +        self.input.properties()
183 +    }
184 +
185 +    fn children(&self) -> Vec<&Arc<dyn ExecutionPlan>> {
186 +        vec![&self.input]
187 +    }
188 +
189 +    fn apply_expressions(
190 +        &self,
191 +        f: &mut dyn FnMut(&Arc<dyn PhysicalExpr>) -> Result<TreeNodeRecursion>,
192 +    ) -> Result<TreeNodeRecursion> {
193 +        Ok(TreeNodeRecursion::Continue)
194 +    }
195 +
196 +    #[expect(deprecated)]
197 +    fn replace_children(
198 +        self: Arc<Self>,
199 +        children: Vec<Arc<dyn ExecutionPlan>>,
200 +        _options: ReplaceChildrenOptions,
201 +    ) -> Result<Arc<dyn ExecutionPlan>> {
202 +        self.with_new_children(children)
203 +    }
204 +
205 +    fn with_new_children(
206 +        self: Arc<Self>,
207 +        mut children: Vec<Arc<dyn ExecutionPlan>>,
208 +    ) -> Result<Arc<dyn ExecutionPlan>> {
209 +        if children.len() != 1 {
210 +            return plan_err!("HostInputExec requires exactly one child");
211 +        }
212 +        if children[0].schema() != self.input.schema() {
213 +            return plan_err!("HostInputExec replacement child has a different schema"
);
214 +        }
215 +        // Re-establish the single-partition contract even after foreign child
216 +        // replacement, which cannot carry required_input_distribution over FFI.
217 +        Ok(Arc::new(Self::new(
218 +            children.remove(0),
219 +            Arc::clone(&self.context),
220 +            self.runtime.clone(),
221 +        )))
222 +    }
223 +}

```



```

29 +         Ok(Arc::new(EmptyTable::new(Arc::new(Schema::empty()))))
30 +     }
31 + }
32 +
33 +#[test]
34 +fn registry_refuses_collisions_unknown_urls_and_invalid_shapes() -> Result<()> {
35 +     let mut registry = ConnectExtensionRegistry::new();
36 +     registry.register(
37 +         "type.test/Stage".into(),
38 +         false,
39 +         2,
40 +         2,
41 +         Arc::new(EmptyHandler),
42 +     );
43 +     registry.register("type.test/Read".into(), true, 0, 0, Arc::new(EmptyHandler))?;
44 +     assert!(registry.resolve("type.test/Stage", true, 2).is_ok());
45 +     assert!(registry.resolve("type.test/Read", false, 0).is_ok());
46 +     assert!(matches!(
47 +         registry.resolve("type.test/Stage", false, 0),
48 +         Err(e) if e.to_string().contains("Stage requires a SailExtensionRequest envel
ope")
49 +     ));
50 +     assert!(matches!(
51 +         registry.resolve("type.test/Stage", true, 1),
52 +         Err(e) if e.to_string().contains("expects 2..=2 inputs, received 1")
53 +     ));
54 +     assert!(matches!(
55 +         registry.resolve("type.test/Missing", true, 0),
56 +         Err(e) if e.to_string().contains("type.test/Missing")
57 +         && e.to_string().contains("[type.test/Read, type.test/Stage]")
58 +     ));
59 +     assert!(matches!(
60 +         registry.register("type.test/Stage".into(), true, 0, 0, Arc::new(EmptyHandler
)),
61 +         Err(e) if e.to_string().contains("duplicate")
62 +     ));
63 +     // A failed duplicate registration leaves the original contract intact.
64 +     assert!(registry.resolve("type.test/Stage", true, 2).is_ok());
65 +     assert!(registry.resolve("type.test/Stage", true, 0).is_err());
66 +     assert!(
67 +         registry
68 +             .register("type.test/Bad".into(), true, 1, 1, Arc::new(EmptyHandler))
69 +             .is_err()
70 +     );
71 +     Ok(())
72 + }
73 +
74 +struct ContextProbeExec {
75 +     expected: Arc<TaskContext>,
76 +     properties: Arc<PlanProperties>,
77 +     executions: Arc<AtomicUsize>,
78 +     runtime: tokio::runtime::Id,
79 + }
80 +
81 +impl Debug for ContextProbeExec {
82 +     fn fmt(&self, f: &mut Formatter<'_>) -> std::fmt::Result {
83 +         write!(f, "ContextProbeExec")
84 +     }
85 + }
86 +
87 +impl DisplayAs for ContextProbeExec {
88 +     fn fmt_as(&self, _t: DisplayFormatType, f: &mut Formatter<'_>) -> std::fmt::Resul
t {
89 +         write!(f, "ContextProbeExec")
90 +     }
91 + }
92 +
93 +impl ExecutionPlan for ContextProbeExec {
94 +     fn name(&self) -> &'static str {
95 +         "ContextProbeExec"
96 +     }
97 +
98 +     fn properties(&self) -> &Arc<PlanProperties> {
99 +         &self.properties
100 +     }
101 +
102 +     fn children(&self) -> Vec<&Arc<dyn ExecutionPlan>> {
103 +         vec![]

```

```

104 +     }
105 +
106 +     fn apply_expressions(
107 +         &self,
108 +         _f: &mut dyn FnMut(&Arc<dyn PhysicalExpr>) -> Result<TreeNodeRecursion>,
109 +     ) -> Result<TreeNodeRecursion> {
110 +         Ok(TreeNodeRecursion::Continue)
111 +     }
112 +
113 +     fn with_new_children(
114 +         self: Arc<Self>,
115 +         children: Vec<Arc<dyn ExecutionPlan>>,
116 +     ) -> Result<Arc<dyn ExecutionPlan>> {
117 +         if !children.is_empty() {
118 +             return plan_err!("ContextProbeExec has no children");
119 +         }
120 +         Ok(self)
121 +     }
122 +
123 +     fn execute(
124 +         &self,
125 +         partition: usize,
126 +         context: Arc<TaskContext>,
127 +     ) -> Result<SendableRecordBatchStream> {
128 +         if !Arc::ptr_eq(&context, &self.expected) {
129 +             return exec_err!("Sail input received a foreign task context");
130 +         }
131 +         assert_eq!(Handle::current().id(), self.runtime);
132 +         self.executions.fetch_add(1, Ordering::SeqCst);
133 +         let values = match partition {
134 +             0 => vec![10, 20],
135 +             1 => vec![],
136 +             2 => vec![30],
137 +             _ => return exec_err!("unexpected partition {partition}"),
138 +         };
139 +         let batch = RecordBatch::try_new(self.schema(), vec![Arc::new(Int64Array::from(
140 +             m(values)))]?);
141 +         let runtime = self.runtime;
142 +         Ok(Box::pin(RecordBatchStreamAdapter::new(
143 +             self.schema(),
144 +             futures::stream::once(async move {
145 +                 assert_eq!(Handle::current().id(), runtime);
146 +                 // Constructing a timer needs the correct reactor at poll time,
147 +                 // even when the caller is an ordinary foreign worker thread.
148 +                 tokio::time::sleep(std::time::Duration::from_millis(1)).await;
149 +                 Ok(batch)
150 +             })),
151 +         ))
152 +     }
153 +
154 +     #[tokio::test]
155 +     async fn host_input_is_lazy_consumes_every_partition_and_preserves_context_after_replacement()
156 +     {
157 +         let host = SessionContext::new().task_ctx();
158 +         let foreign = SessionContext::new().task_ctx();
159 +         let executions = Arc::new(AtomicUsize::new(0));
160 +         let schema = Arc::new(Schema::new(vec![Field::new("id", DataType::Int64, false)]));
161 +         let probe = Arc::new(ContextProbeExec {
162 +             expected: Arc::clone(&host),
163 +             properties: Arc::new(PlanProperties::new(
164 +                 EquivalenceProperties::new(schema),
165 +                 Partitioning::UnknownPartitioning(3),
166 +                 EmissionType::Incremental,
167 +                 Boundedness::Bounded,
168 +             )),
169 +             executions: Arc::clone(&executions),
170 +             runtime: Handle::current().id(),
171 +         });
172 +         let input = Arc::new(HostInputExec::new(probe.clone(), host, Handle::current()));
173 +         assert_eq!(executions.load(Ordering::SeqCst), 0);
174 +         assert!(input.execute(1, Arc::clone(&foreign)).is_err());
175 +         let batches = collect(input.execute(0, Arc::clone(&foreign))).await?;
176 +         assert_eq!(batches.iter().map(RecordBatch::num_rows).sum::<usize>(), 3);
177 +         assert_eq!(executions.load(Ordering::SeqCst), 3);
178 +     }

```

```

179 +     #[expect(deprecated)]
180 +     let replaced = input.with_new_children(vec![probe]);
181 +     let batches = collect(replaced.execute(0, foreign?).await?);
182 +     assert_eq!(batches.iter().map(RecordBatch::num_rows).sum::<usize>(), 3);
183 +     assert_eq!(executions.load(Ordering::SeqCst), 6);
184 +     Ok(())
185 + }
186 +
187 + #[test]
188 + fn host_input_preserves_runtime_when_replaced_and_polled_on_a_foreign_thread() -> Result<()> {
189 +     let runtime = tokio::runtime::Builder::new_multi_thread()
190 +         .worker_threads(2)
191 +         .enable_all()
192 +         .build()?;
193 +     let host = SessionContext::new().task_ctx();
194 +     let foreign = SessionContext::new().task_ctx();
195 +     let executions = Arc::new(AtomicUsize::new(0));
196 +     let schema = Arc::new(Schema::new(vec![Field::new("id", DataType::Int64, false)]));
197 +     let probe = Arc::new(ContextProbeExec {
198 +         expected: Arc::clone(&host),
199 +         properties: Arc::new(PlanProperties::new(
200 +             EquivalenceProperties::new(schema),
201 +             Partitioning::UnknownPartitioning(1),
202 +             EmissionType::Incremental,
203 +             Boundedness::Bounded,
204 +         )),
205 +         executions: Arc::clone(&executions),
206 +         runtime: runtime.handle().id(),
207 +     });
208 +     let input = Arc::new(HostInputExec::new(
209 +         probe.clone(),
210 +         host,
211 +         runtime.handle().clone(),
212 +     ));
213 +     // The single-partition coalescer delegates its stream directly. Thus this
214 +     // tests the wrapper's poll guard itself, without a Tokio task hiding it.
215 +     std::thread::spawn(move || -> Result<()> {
216 +         assert!(Handle::try_current().is_err());
217 +         let batches =
218 +             futures::executor::block_on(collect(input.execute(0, Arc::clone(&foreign)
219 +                 )?));
220 +         assert_eq!(batches.iter().map(RecordBatch::num_rows).sum::<usize>(), 2);
221 +         assert!(Handle::try_current().is_err());
222 +         #[expect(deprecated)]
223 +         let replaced = input.with_new_children(vec![probe]);
224 +         let batches = futures::executor::block_on(collect(replaced.execute(0, foreign
225 +             )?));
226 +         assert_eq!(batches.iter().map(RecordBatch::num_rows).sum::<usize>(), 2);
227 +         assert!(Handle::try_current().is_err());
228 +         Ok(())
229 +     })
230 +     .join()
231 +     .map_err(|_| datafusion_common::internal_datafusion_err!("foreign thread panicked"))?;
232 +     assert_eq!(executions.load(Ordering::SeqCst), 2);
233 +     Ok(())
234 + }

```

06. crates/sail-common-datafusion/src/driver_extension.rs

```

1 diff --git a/crates/sail-common-datafusion/src/driver_extension.rs b/crates/sail-commo
2 n-datafusion/src/driver_extension.rs
3 new file mode 100644
4 index 0000000000000000000000000000000000000000..15fa21a959416f1a6b9d96473ad52ddffacf00
5 cf
6 --- /dev/null
7 +++ b/crates/sail-common-datafusion/src/driver_extension.rs
8 @@ -0,0 +1,210 @@
9 +#!/ Session-owned native regions placed on the driver in a distributed job.
10 +use std::collections::HashMap;
11 +use std::fmt::{Debug, Formatter};
12 +use std::sync::{Arc, Mutex, Weak};
13 +

```

```

12 +use datafusion::execution::{SendableRecordBatchStream, TaskContext};
13 +use datafusion::physical_plan::{DisplayAs, DisplayFormatType, ExecutionPlan, PlanProperties};
14 +use datafusion_common::{Result, plan_datafusion_err, plan_err};
15 +use serde::{Deserialize, Serialize};
16 +
17 +use crate::extension::{SessionExtension, SessionExtensionAccessor};
18 +
19 +pub const DRIVER_CODEEC_PREFIX: &[u8] = b"SAIL_DRIVER_EXTENSION_V1\0";
20 +
21 +/// A frozen native plan, including its graph revision or mutation attempt state.
22 +/// Only its declared host inputs are substituted during task preparation.
23 +pub trait DriverExtensionBinding: Debug + Send + Sync {
24 +    /// Release native snapshots after query completion/cancellation. Streams
25 +    /// already executing own their materialized plans until they are dropped.
26 +    fn close(&self) {}
27 +    fn materialize(
28 +        &self,
29 +        inputs: &[Arc<dyn ExecutionPlan>],
30 +        context: Arc<TaskContext>,
31 +    ) -> Result<Arc<dyn ExecutionPlan>>;
32 +}
33 +
34 +#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Eq)]
35 +#[serde(deny_unknown_fields)]
36 +pub struct DriverDescriptor {
37 +    pub owner: String,
38 +    pub plan_id: String,
39 +}
40 +
41 +#[derive(Debug)]
42 +pub struct BoundDriverPlan {
43 +    pub descriptor: DriverDescriptor,
44 +    pub properties: Arc<PlanProperties>,
45 +    pub input_schemas: Vec<arrow_schema::SchemaRef>,
46 +    pub binding: Arc<dyn DriverExtensionBinding>,
47 +}
48 +
49 +/// Weak entries do not extend graph snapshots beyond the lifetime of a query.
50 +/// The job graph owns each live bound plan while its tasks can be decoded.
51 +#[derive(Debug, Default)]
52 +pub struct DriverExtensionRegistry(Mutex<HashMap<String, Weak<BoundDriverPlan>>>);
53 +
54 +impl SessionExtension for DriverExtensionRegistry {
55 +    fn name() -> &'static str {
56 +        "DriverExtensionRegistry"
57 +    }
58 +}
59 +
60 +impl DriverExtensionRegistry {
61 +    pub fn register(&self, plan: &Arc<BoundDriverPlan>) -> Result<()> {
62 +        let mut entries = self
63 +            .lock()
64 +            .map_err(|_| plan_datafusion_err!("driver extension registry poisoned"))?;
65 +
66 +        entries.retain(|_, value| value.strong_count() > 0);
67 +        if entries.contains_key(&plan.descriptor.plan_id) {
68 +            return plan_err!("duplicate driver extension plan identity");
69 +        }
70 +        entries.insert(plan.descriptor.plan_id.clone(), Arc::downgrade(plan));
71 +        Ok(())
72 +    }
73 +
74 +    fn lookup(&self, descriptor: &DriverDescriptor) -> Result<Arc<BoundDriverPlan>> {
75 +        let entries = self
76 +            .lock()
77 +            .map_err(|_| plan_datafusion_err!("driver extension registry poisoned"))?;
78 +
79 +        let plan = entries
80 +            .get(&descriptor.plan_id)
81 +            .and_then(Weak::upgrade)
82 +            .ok_or_else(|| {
83 +                plan_datafusion_err!(
84 +                    "driver extension plan is unavailable in this session or has expired"
85 +                )
86 +            })

```

```

86 +         }}?;
87 +         if plan.descriptor != *descriptor {
88 +             return plan_err!("driver extension owner mismatch");
89 +         }
90 +         Ok(plan)
91 +     }
92 + }
93 +
94 + #[derive(Debug)]
95 + pub struct DriverExtensionExec {
96 +     pub bound: Arc<BoundDriverPlan>,
97 +     inputs: Vec<Arc<dyn ExecutionPlan>>,
98 + }
99 +
100 + impl DriverExtensionExec {
101 +     pub fn new(bound: Arc<BoundDriverPlan>, inputs: Vec<Arc<dyn ExecutionPlan>>) -> Result<Self> {
102 +         if inputs.len() != bound.input_schemas.len()
103 +             || inputs
104 +                 .iter()
105 +                 .zip(&bound.input_schemas)
106 +                 .any(|(input, schema)| input.schema() != *schema)
107 +         {
108 +             return plan_err!("driver extension input arity/schema mismatch");
109 +         }
110 +         Ok(Self { bound, inputs })
111 +     }
112 +
113 +     pub fn encode(&self, buffer: &mut Vec<u8>) -> Result<()> {
114 +         buffer.extend_from_slice(DRIVER_CODEC_PREFIX);
115 +         serde_json::to_writer(buffer, &self.bound.descriptor)
116 +             .map_err(|e| plan_datafusion_err!("driver extension descriptor: {e}"))
117 +     }
118 +
119 +     pub fn decode(
120 +         buffer: &[u8],
121 +         inputs: &[Arc<dyn ExecutionPlan>],
122 +         context: &TaskContext,
123 +     ) -> Result<Arc<dyn ExecutionPlan>> {
124 +         if buffer.len() > 8192 {
125 +             return plan_err!("driver extension descriptor exceeds 8192 bytes");
126 +         }
127 +         let payload = buffer
128 +             .strip_prefix(DRIVER_CODEC_PREFIX)
129 +             .ok_or_else(|| plan_datafusion_err!("unknown driver extension codec owner
130 + /version"))?;
131 +         let descriptor: DriverDescriptor = serde_json::from_slice(payload)
132 +             .map_err(|e| plan_datafusion_err!("driver extension descriptor: {e}"))?;
133 +         let registry = context
134 +             .extension::()
135 +             .map_err(|_| {
136 +                 plan_datafusion_err!("driver-only extension cannot be decoded on a worker")
137 +             })?;
138 +         Ok(Arc::new(Self::new(
139 +             registry.lookup(&descriptor)?,
140 +             inputs.to_vec(),
141 +         )))
142 +     }
143 + }
144 +
145 + impl DisplayAs for DriverExtensionExec {
146 +     fn fmt_as(&self, _: DisplayFormatType, f: &mut Formatter<'_>) -> std::fmt::Result {
147 +         write!(
148 +             f,
149 +             "DriverExtensionExec: owner={}, plan={}, placement=driver, retry=disabled
150 + ",
151 +             self.bound.descriptor.owner, self.bound.descriptor.plan_id
152 +         )
153 +     }
154 + }
155 +
156 + impl ExecutionPlan for DriverExtensionExec {
157 +     fn apply_expressions(
158 +         &self,
159 +         _f: &mut dyn FnMut(
160 +             &Arc<dyn datafusion::physical_expr::PhysicalExpr>,

```

```

159 +     ) -> Result<datafusion_common::tree_node::TreeNodeRecursion>,
160 + ) -> Result<datafusion_common::tree_node::TreeNodeRecursion> {
161 +     Ok(datafusion_common::tree_node::TreeNodeRecursion::Continue)
162 + }
163 + fn name(&self) -> &str {
164 +     "DriverExtensionExec"
165 + }
166 + fn properties(&self) -> &Arc<PlanProperties> {
167 +     &self.bound.properties
168 + }
169 + fn children(&self) -> Vec<&Arc<dyn ExecutionPlan>> {
170 +     self.inputs.iter().collect()
171 + }
172 + fn required_input_distribution(&self) -> Vec<datafusion::physical_expr::Distribution> {
173 +     vec![datafusion::physical_expr::Distribution::SinglePartition; self.inputs.len()]
174 + }
175 + fn with_new_children(
176 +     self: Arc<Self>,
177 +     children: Vec<Arc<dyn ExecutionPlan>>,
178 + ) -> Result<Arc<dyn ExecutionPlan>> {
179 +     Ok(Arc::new(Self::new(self.bound.clone(), children)))
180 + }
181 + fn execute(
182 +     &self,
183 +     partition: usize,
184 +     context: Arc<TaskContext>,
185 + ) -> Result<SendableRecordBatchStream> {
186 +     if partition != 0
187 +     || self
188 +         .inputs
189 +         .iter()
190 +         .any(|input| input.properties().partitioning.partition_count() != 1)
191 +     {
192 +         return plan_err!("driver extension requires one output partition and gathered inputs");
193 +     }
194 +     let plan = self
195 +         .bound
196 +         .binding
197 +         .materialize(&self.inputs, context.clone());
198 +     plan.execute(0, context)
199 + }
200 +}
201 +
202 +pub fn contains_driver_extension(plan: &Arc<dyn ExecutionPlan>) -> bool {
203 +     plan.is::<DriverExtensionExec>() || plan.children().into_iter().any(contains_driver_extension)
204 +}
205 +
206 +pub fn release_driver_extensions(plan: &Arc<dyn ExecutionPlan>) {
207 +     if let Some(native) = plan.downcast_ref::<DriverExtensionExec>() {
208 +         native.bound.binding.close();
209 +     }
210 +     for child in plan.children() {
211 +         release_driver_extensions(child);
212 +     }
213 +}
214 +
215 +#[cfg(test)]
216 +mod tests;

```

07. crates/sail-common-datafusion/src/driver_extension/tests.rs

```

1 diff --git a/crates/sail-common-datafusion/src/driver_extension/tests.rs b/crates/sail-common-datafusion/src/driver_extension/tests.rs
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..c5e7a7c0d9e02f84e394241a003818245a6d6c0f
4 --- /dev/null
5 +++ b/crates/sail-common-datafusion/src/driver_extension/tests.rs
6 @@ -0,0 +1,64 @@
7 +use datafusion::physical_plan::empty::EmptyExec;
8 +use datafusion::prelude::{SessionConfig, SessionContext};

```

```

9 +
10 +use super::*;
11 +
12 +#[derive(Debug)]
13 +struct Binding(Arc<dyn ExecutionPlan>);
14 +impl DriverExtensionBinding for Binding {
15 +    fn materialize(
16 +        &self,
17 +        _ : &[Arc<dyn ExecutionPlan>],
18 +        _ : Arc<TaskContext>,
19 +    ) -> Result<Arc<dyn ExecutionPlan>> {
20 +        Ok(self.0.clone())
21 +    }
22 +}
23 +
24 +#[test]
25 +fn driver_extension_codec_binds_only_live_plans_in_the_original_session() -> Result<()> {
26 +    let registry = Arc::new(DriverExtensionRegistry::default());
27 +    let context =
28 +        SessionContext::new_with_config(SessionConfig::new().with_extension(registry.clone()))
29 +            .task_ctx();
30 +    let empty: Arc<dyn ExecutionPlan> =
31 +        Arc::new(EmptyExec::new(Arc::new(arrow_schema::Schema::empty())));
32 +    let bound = Arc::new(BoundDriverPlan {
33 +        descriptor: DriverDescriptor {
34 +            owner: "fixture@1:digest".into(),
35 +            plan_id: "incarnation:operation".into(),
36 +        },
37 +        properties: empty.properties().clone(),
38 +        input_schemas: vec![],
39 +        binding: Arc::new(Binding(empty.clone())),
40 +    });
41 +    registry.register(&bound)?;
42 +    let exec = DriverExtensionExec::new(bound.clone(), vec![])?;
43 +    let mut bytes = vec![];
44 +    exec.encode(&mut bytes)?;
45 +    let decoded = DriverExtensionExec::decode(&bytes, &[], &context)?;
46 +    assert_eq!(decoded.name(), "DriverExtensionExec");
47 +    assert!(DriverExtensionExec::decode(&bytes, &[empty], &context).is_err());
48 +    assert!(DriverExtensionExec::decode(&bytes, &[], &TaskContext::default()).is_err());
49 +    let other = SessionContext::new_with_config(
50 +        SessionConfig::new().with_extension(Arc::new(DriverExtensionRegistry::default()))),
51 +    )
52 +    .task_ctx();
53 +    assert!(DriverExtensionExec::decode(&bytes, &[], &other).is_err());
54 +    let mut forged = DRIVER_CODEC_PREFIX.to_vec();
55 +    serde_json::to_writer(
56 +        &mut forged,
57 +        &DriverDescriptor {
58 +            owner: "other".into(),
59 +            plan_id: bound.descriptor.plan_id.clone(),
60 +        },
61 +    )
62 +    .map_err(|e| plan_datafusion_err!("{}", e))?;
63 +    assert!(DriverExtensionExec::decode(&forged, &[], &context).is_err());
64 +    drop(decoded);
65 +    drop(exec);
66 +    drop(bound);
67 +    assert!(DriverExtensionExec::decode(&bytes, &[], &context).is_err());
68 +    assert!(DriverExtensionExec::decode(&vec![0; 8193], &[], &context).is_err());
69 +    Ok(())
70 +}

```

08. crates/sail-common-datafusion/src/geometry.rs

```

1 diff --git a/crates/sail-common-datafusion/src/geometry.rs b/crates/sail-common-datafu
2 sion/src/geometry.rs
3 new file mode 100644
4 index 0000000000000000000000000000000000000000..86b06c1b723b2d53cc8c1da667499964ace75f
5 e2
6 --- /dev/null

```

```

5 +++ b/crates/sail-common-datafusion/src/geometry.rs
6 @@ -0,0 +1,65 @@
7 +/// Conservative propagation of GeoArrow logical types through value selectors.
8 +use std::collections::HashMap;
9 +
10 +use arrow_schema::{DataType, FieldRef};
11 +
12 +/// Preserve a geometry type only when every non-NULL alternative has the same
13 +/// storage type and GeoArrow extension metadata. Plain binary is not geometry;
14 +/// neither mixed CRS nor geometry/geography alternatives may be relabelled.
15 +pub fn common_geometry_metadata(fields: &[FieldRef]) -> Option<HashMap<String, String>
16 +>> {
17 +    const NAME: &str = "ARROW:extension:name";
18 +    const METADATA: &str = "ARROW:extension:metadata";
19 +    let mut fields = fields.iter().filter(|field| !field.data_type().is_null());
20 +    let first = fields.next()?;
21 +    if !matches!(
22 +        first.data_type(),
23 +        DataType::Binary | DataType::LargeBinary | DataType::BinaryView
24 +    ) || first.metadata().get(NAME).map(String::as_str) != Some("geoarrow.wkb")
25 +    {
26 +        return None;
27 +    }
28 +    if fields.any(|field| {
29 +        field.data_type() != first.data_type()
30 +        || field.metadata().get(NAME) != first.metadata().get(NAME)
31 +        || field.metadata().get(METADATA) != first.metadata().get(METADATA)
32 +    }) {
33 +        return None;
34 +    }
35 +    Some(
36 +        first
37 +        .metadata()
38 +        .iter()
39 +        .filter(|(key, _)| matches!(key.as_str(), NAME | METADATA))
40 +        .map(|(key, value)| (key.clone(), value.clone()))
41 +        .collect(),
42 +    )
43 +}
44 +#[cfg(test)]
45 +mod tests {
46 +    use std::sync::Arc;
47 +
48 +    use arrow_schema::Field;
49 +
50 +    use super::*;
51 +
52 +    #[test]
53 +    fn geometry_alternatives_require_compatible_types_and_crs() {
54 +        let geometry = Arc::new(Field::new("g", DataType::Binary, true).with_metadata(
55 +            (
56 +                HashMap::from([
57 +                    ("ARROW:extension:name".into(), "geoarrow.wkb".into()),
58 +                    ("ARROW:extension:metadata".into(), "{}".into()),
59 +                ]),
60 +            ));
61 +        let null = Arc::new(Field::new("null", DataType::Null, true));
62 +        assert!(common_geometry_metadata(&[geometry.clone(), null]).is_some());
63 +        let plain = Arc::new(Field::new("binary", DataType::Binary, true));
64 +        assert!(common_geometry_metadata(&[geometry.clone(), plain]).is_none());
65 +        let mut other = geometry.as_ref().clone();
66 +        other.metadata_mut().insert(
67 +            "ARROW:extension:metadata".into(),
68 +            r#"{"edges":"spherical"}"#.into(),
69 +        );
70 +        assert!(common_geometry_metadata(&[geometry, Arc::new(other)]).is_none());
71 +    }
72 +}

```

09. crates/sail-common-datafusion/src/lib.rs

```

1 diff --git a/crates/sail-common-datafusion/src/lib.rs b/crates/sail-common-datafusion/
  src/lib.rs

```



```

44 + pub fn reserve(
45 +     &self,
46 +     pool: &Arc<dyn MemoryPool>,
47 +     extension: &str,
48 +     bytes: usize,
49 + ) -> Result<MemoryLease> {
50 +     reserve_native_quota_inner(pool, extension, bytes, Some(self.0.clone()))
51 + }
52 +
53 + pub async fn wait_for_release(&self) {
54 +     let mut receiver = self.0.subscribe();
55 +     // The sender in self remains alive throughout this wait.
56 +     let _ = receiver.wait_for(|count| *count == 0).await;
57 + }
58 +}
59 +
60 +/// Opt-in lifecycle evidence for integration tests. Normal execution does no
61 +/// filesystem logging. A release event is written by the last lease owner after
62 +/// its reservation is returned, rather than by a cancellation request handler.
63 +struct QuotaAudit {
64 +    path: PathBuf,
65 +    id: u64,
66 +    extension: String,
67 +    bytes: usize,
68 +    pool: Arc<dyn MemoryPool>,
69 +}
70 +
71 +impl QuotaAudit {
72 +    fn write(&self, event: &str) -> Result<()> {
73 +        static WRITER: std::sync::Mutex<()> = std::sync::Mutex::new();
74 +        let _lock = WRITER.lock().map_err(|_| {
75 +            DataFusionError::Execution("native resource audit writer poisoned".into())
76 +        })?;
77 +        let record = serde_json::json!({
78 +            "timestamp_utc": chrono::Utc::now().to_rfc3339(),
79 +            "pid": std::process::id(), "id": self.id, "event": event,
80 +            "extension": self.extension, "bytes": self.bytes,
81 +            "pool_reserved": self.pool.reserved(),
82 +        });
83 +        let mut bytes = serde_json::to_vec(&record)
84 +            .map_err(|error| DataFusionError::Execution(error.to_string()))?;
85 +        bytes.push(b'\n');
86 +        OpenOptions::new()
87 +            .create(true)
88 +            .append(true)
89 +            .open(&self.path)?
90 +            .write_all(&bytes)?;
91 +        Ok(())
92 +    }
93 +}
94 +
95 +impl Drop for NativeQuota {
96 +    fn drop(&mut self) {
97 +        // Release first: the audit is evidence about completed accounting.
98 +        self.reservation.take();
99 +        if let Some(audit) = &self.audit
100 +            && let Err(error) = audit.write("released")
101 +        {
102 +            eprintln!("native resource release audit failed: {error}");
103 +        }
104 +        if let Some(tracker) = &self.tracker {
105 +            tracker.send_modify(|count| *count -= 1);
106 +        }
107 +    }
108 +}
109 +
110 +/// Admit the entire native session cap before any extension state is allocated.
111 +/// The lease is non-spillable; the native subpool must stay within this prepaid
112 +/// cap. This accounts participating native allocations, not total process RSS.
113 +pub fn reserve_native_quota(
114 +    pool: &Arc<dyn MemoryPool>,
115 +    extension: &str,
116 +    bytes: usize,
117 +) -> Result<MemoryLease> {
118 +    reserve_native_quota_inner(pool, extension, bytes, None)
119 +}
120 +

```

```

121 +fn reserve_native_quota_inner(
122 +    pool: &Arc<dyn MemoryPool>,
123 +    extension: &str,
124 +    bytes: usize,
125 +    tracker: Option<watch::Sender<usize>>,
126 +) -> Result<MemoryLease> {
127 +    if bytes == 0 {
128 +        return plan_err!("native extension {extension} requires a positive memory quota");
129 +    }
130 +    let reservation = MemoryConsumer::new(format!("native extension {extension} session quota"))
131 +        .with_can_spill(false)
132 +        .register(pool);
133 +    reservation.try_grow(bytes).map_err(|error| {
134 +        // Spark's error transport unwraps Context variants. Keep the admission
135 +        // boundary in the actual resource error so the client sees it too.
136 +        DataFusionError::ResourcesExhausted(format!(
137 +            "native extension {extension} host memory admission of {bytes} bytes refused: {error}")
138 +        ))
139 +    });
140 +    let audit = std::env::var_os("SAIL_NATIVE_RESOURCE_AUDIT").map(|path| {
141 +        static IDS: AtomicU64 = AtomicU64::new(1);
142 +        QuotaAudit {
143 +            path: path.into(),
144 +            id: IDS.fetch_add(1, Ordering::Relaxed),
145 +            extension: extension.to_owned(),
146 +            bytes,
147 +            pool: pool.clone(),
148 +        }
149 +    });
150 +    if let Some(audit) = &audit {
151 +        audit.write("admitted");
152 +    }
153 +    if let Some(tracker) = &tracker {
154 +        tracker.send_modify(|count| *count += 1);
155 +    }
156 +    Ok(MemoryLease::new(
157 +        Arc::new(NativeQuota {
158 +            reservation: Some(reservation),
159 +            audit,
160 +            tracker,
161 +        }),
162 +        bytes as u64,
163 +    ))
164 +}
165 +
166 +#[cfg(test)]
167 +mod tests {
168 +    use datafusion::execution::memory_pool::GreedyMemoryPool;
169 +    use super::*;
170 +
171 +    #[tokio::test]
172 +    async fn shutdown_barrier_waits_for_the_last_detached_output_owner() -> Result<()> {
173 +        {
174 +            let tracker = NativeResourceTracker::default();
175 +            let pool: Arc<dyn MemoryPool> = Arc::new(GreedyMemoryPool::new(64));
176 +            let lease = tracker.reserve(&pool, "native-output", 64)?;
177 +            let output = lease.clone();
178 +            drop(lease);
179 +            let mut waiting = Box::pin(tracker.wait_for_release());
180 +            assert!(futures::poll!(waiting.as_mut()).is_pending());
181 +            std::thread::spawn(move || drop(output))
182 +                .join()
183 +                .map_err(|_| DataFusionError::Execution("output owner thread panicked".into()))?;
184 +            waiting.await;
185 +            assert_eq!(pool.reserved(), 0);
186 +            Ok(())
187 +        }
188 +
189 +        #[test]
190 +        fn native_quota_and_datafusion_operators_contend_until_the_last_owner_drops() -> Result<()> {
191 +            let pool: Arc<dyn MemoryPool> = Arc::new(GreedyMemoryPool::new(96));

```



```

45 + fn signature(&self) -> &Signature {
46 +     self.udf.signature()
47 + }
48 + fn return_type(&self, types: &[DataType]) -> Result<DataType> {
49 +     self.udf.return_type(types)
50 + }
51 + fn return_field_from_args(&self, args: ReturnFieldArgs) -> Result<FieldRef> {
52 +     self.udf.return_field_from_args(args)
53 + }
54 + fn invoke_with_args(&self, args: ScalarFunctionArgs) -> Result<ColumnarValue> {
55 +     // The owner is held for every call and for as long as this UDF is planned.
56 +     let owner = &self.owner;
57 +     self.udf.invoke_with_args(args)
58 + }
59 + fn coerce_types(&self, types: &[DataType]) -> Result<Vec<DataType>> {
60 +     self.udf.coerce_types(types)
61 + }
62 + fn short_circuits(&self) -> bool {
63 +     self.udf.short_circuits()
64 + }
65 +}
66 +
67 +impl std::fmt::Debug for OwnedScalar {
68 +    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
69 +        f.debug_struct("OwnedScalar")
70 +            .field("name", &self.name)
71 +            .field("identity", &self.identity)
72 +            .finish()
73 +    }
74 +}
75 +
76 +pub const SCALAR_CODECS_PREFIX: &[u8] = b"SAIL_NATIVE_SCALAR_V1\0";
77 +type Functions = HashMap<(String, String), Arc<ScalarUDF>>;
78 +fn functions() -> &'static Mutex<Functions> {
79 +    static FUNCTIONS: OnceLock<Mutex<Functions>> = OnceLock::new();
80 +    FUNCTIONS.get_or_init(Mutex::default)
81 +}
82 +
83 +/// Only immutable, placement=any scalar implementations enter this process registry.
84 +/// Keys include a content digest of package code and canonical configuration.
85 +pub fn retain_scalar(udf: ScalarUDF) -> Result<()> {
86 +    let native = udf
87 +        .inner()
88 +        .downcast_ref::<OwnedScalar>()
89 +        .ok_or_else(|| plan_datafusion_err!("expected native scalar"))?;
90 +    let key = (native.identity.clone(), native.name.clone());
91 +    functions()
92 +        .lock()
93 +        .map_err(|_| plan_datafusion_err!("native scalar registry poisoned"))?
94 +        .entry(key)
95 +        .or_insert_with(|| Arc::new(udf));
96 +    Ok(())
97 +}
98 +
99 +pub fn encode_scalar(udf: &ScalarUDF, output: &mut Vec<u8>) -> Result<bool> {
100 +    let Some(native) = udf.inner().downcast_ref::<OwnedScalar>() else {
101 +        return Ok(false);
102 +    };
103 +    output.extend_from_slice(SCALAR_CODECS_PREFIX);
104 +    let descriptor = serde_json::to_vec(&(native.identity, native.name))
105 +        .map_err(|_| plan_datafusion_err!("native scalar descriptor: {e}"))?;
106 +    output.extend_from_slice(&descriptor);
107 +    Ok(true)
108 +}
109 +
110 +pub fn decode_scalar(name: &str, bytes: &[u8]) -> Result<Arc<ScalarUDF>> {
111 +    if bytes.len() > 8192 {
112 +        return Err(plan_datafusion_err!("native scalar descriptor too large"));
113 +    }
114 +    let payload = bytes
115 +        .strip_prefix(SCALAR_CODECS_PREFIX)
116 +        .ok_or_else(|| plan_datafusion_err!("invalid native scalar codec owner/version"))?;
117 +    let (identity, encoded_name): (String, String) = serde_json::from_slice(payload)
118 +        .map_err(|_| plan_datafusion_err!("native scalar descriptor: {e}"))?;
119 +    if name != encoded_name {
120 +        return Err(plan_datafusion_err!("native scalar name mismatch"));
121 +    }

```



```

27 +     }
28 +
29 +     pub async fn stop(&self) -> Result<()> {
30 +         self.resource.stop().await
31 +     }
32 +}
33 +
34 +impl SessionExtension for SessionLifecycle {
35 +    fn name() -> &'static str {
36 +        "SessionLifecycle"
37 +    }
38 +}

```

13. crates/sail-common-datafusion/src/session/mod.rs

```

1 diff --git a/crates/sail-common-datafusion/src/session/mod.rs b/crates/sail-common-dat
2 afusion/src/session/mod.rs
3 index 2bde515e56dcdca5ef2ed4af8e0464c8799e8a39..23e61281b8659a968e1632901e8a044c1450e8
4 1c 100644
5 --- a/crates/sail-common-datafusion/src/session/mod.rs
6 +++ b/crates/sail-common-datafusion/src/session/mod.rs
7 @@ -1,4 +1,5 @@
8     pub mod activity;
9     pub mod job;
10 +pub mod lifecycle;
11     pub mod plan;
12     pub mod repartition;

```

14. crates/sail-common/src/spec/plan.rs

```

1 diff --git a/crates/sail-common/src/spec/plan.rs b/crates/sail-common/src/spec/plan.rs
2 index 33c14d0d485a6a12d6fecbeaa5ffca3898db741b..33f736ebda67536a01bacaa2bb5939bd72e971
3 82 100644
4 --- a/crates/sail-common/src/spec/plan.rs
5 +++ b/crates/sail-common/src/spec/plan.rs
6 @@ -73,6 +73,13 @@ impl CommandPlan {
7     #[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
8     #[serde(rename_all = "camelCase", rename_all_fields = "camelCase")]
9     pub enum QueryNode {
10 +        /// Experimental, local-mode Spark Connect relation extension.
11 +        Extension {
12 +            payload_type_url: String,
13 +            payload: Vec<u8>,
14 +            inputs: Vec<QueryPlan>,
15 +            is_envelope: bool,
16 +        },
17         Read {
18             #[serde(flatten)]
19             read_type: ReadType,

```

15. crates/sail-execution/src/driver/job_scheduler/core.rs

```

1 diff --git a/crates/sail-execution/src/driver/job_scheduler/core.rs b/crates/sail-exec
2 ution/src/driver/job_scheduler/core.rs
3 index 7161812f21a1062c0e4c34ee4eb9978f8bb5b205..c6b3ab1ec98832cc2dd925776df5211a2f82af
4 b8 100644
5 --- a/crates/sail-execution/src/driver/job_scheduler/core.rs
6 +++ b/crates/sail-execution/src/driver/job_scheduler/core.rs
7 @@ -10,6 +10,9 @@ use datafusion_proto::physical_plan::PhysicalExtensionCodec;
8     use indexmap::{IndexMap, IndexSet};
9     use log::{debug, warn};
10     use sail_common::actor::ActorContext;
11 +use sail_common_datafusion::driver_extension::{
12 +    contains_driver_extension, release_driver_extensions,
13 +};
14     use sail_common_datafusion::error::CommonErrorCause;
15     use sail_python_udf::error::PyErrExtractor;
16     use sail_system_store::SystemEvent;
17 @@ -203,6 +206,18 @@ impl JobScheduler {
18         .any(|x| matches!(x.state, TaskRegionState::Failed))

```

```

17         {
18             let cause = Self::infer_job_failure_cause(job);
19 +             let cause = if job
20 +                 .graph
21 +                 .stages()
22 +                 .iter()
23 +                 .any(|stage| contains_driver_extension(&stage.plan))
24 +             {
25 +                 CommonErrorCause::Execution(format!(
26 +                     "driver extension task failed; automatic retry disabled; an unack
nowledged mutation outcome is indeterminate: {cause}")
27 +                 ))
28 +             } else {
29 +                 cause
30 +             };
31             if let JobState::Running { output, .. } = &job.state {
32                 actions.push(JobAction::FailJobOutput {
33                     handle: output.handle(),
34 @@ -248,11 +263,23 @@ impl JobScheduler {
35
36         fn update_task_regions(job: &mut JobDescriptor, options: &JobSchedulerOptions) {
37             for (r, region) in job.topology.regions.iter().enumerate() {
38 +                 // A lost acknowledgement cannot distinguish a committed native
39 +                 // mutation from one canceled before publication. Never re-execute
40 +                 // a region containing a driver-resident native operation.
41 +                 let max_attempts = if region
42 +                     .tasks
43 +                     .iter()
44 +                     .any(|task| contains_driver_extension(&job.graph.stages()[task.stage]
.plan))
45 +                 {
46 +                     1
47 +                 } else {
48 +                     options.task_max_attempts
49 +                 };
50                 let failed = region.tasks.iter().any(|t| {
51                     let attempts = &job.stages[t.stage].tasks[t.partition].attempts;
52                     if let Some(attempt) = attempts.last()
53                         && matches!(attempt.state, TaskState::Failed | TaskState::Cancele
d)
54 -                         && attempts.len() >= options.task_max_attempts
55 +                         && attempts.len() >= max_attempts
56                     {
57                         return true;
58                     }
59 @@ -625,6 +652,9 @@ impl JobScheduler {
60                 context: job.context.clone(),
61             });
62             job.state.finish_output(outcome);
63 +             for stage in job.graph.stages() {
64 +                 release_driver_extensions(&stage.plan);
65 +             }
66             event_reporter.report(SystemEvent::JobUpdated {
67                 session_id,
68                 job_id: u64::from(job_id),
69 @@ -1107,6 +1137,10 @@ impl StageGroup {
70             }
71         }
72
73 +#[cfg(test)]
74 +#[path = "native_retry_tests.rs"]
75 +mod native_retry_tests;
76 +
77 +#[cfg(test)]
78     mod tests {
79         use std::collections::HashSet;

```

16. crates/sail-execution/src/driver/job_scheduler/native_retry_tests.rs

```

1 diff --git a/crates/sail-execution/src/driver/job_scheduler/native_retry_tests.rs b/cr
ates/sail-execution/src/driver/job_scheduler/native_retry_tests.rs
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..f11eb3c3f3bc04f1026c8db0f8963a6d1dd1da
07
4 --- /dev/null

```

```

5 +++ b/crates/sail-execution/src/driver/job_scheduler/native_retry_tests.rs
6 @@ -0,0 +1,92 @@
7 +use std::sync::atomic::{AtomicUsize, Ordering};
8 +
9 +use datafusion::arrow::datatypes::Schema;
10 +use datafusion::physical_plan::empty::EmptyExec;
11 +use sail_common datafusion::driver_extension::{
12 +     BoundDriverPlan, DriverDescriptor, DriverExtensionBinding, DriverExtensionExec,
13 + };
14 +
15 +use super::*;
16 +use crate::job_graph::JobGraphOptions;
17 +use crate::shuffle::ShuffleCompression;
18 +
19 +#[derive(Debug)]
20 +struct CommitThenLoseAcknowledgement(Arc<AtomicUsize>);
21 +impl DriverExtensionBinding for CommitThenLoseAcknowledgement {
22 +     fn materialize(
23 +         &self,
24 +         _: &[Arc<dyn ExecutionPlan>],
25 +         _: Arc<TaskContext>,
26 +     ) -> datafusion::common::Result<Arc<dyn ExecutionPlan>> {
27 +         self.0.fetch_add(1, Ordering::SeqCst);
28 +         datafusion::common::exec_err!("injected acknowledgement loss after commit")
29 +     }
30 + }
31 +
32 +#[test]
33 +fn driver_mutation_acknowledgement_loss_exhausts_the_region_after_one_attempt()
34 +-> ExecutionResult<()> {
35 +     let commits = Arc::new(AtomicUsize::new(0));
36 +     let empty: Arc<dyn ExecutionPlan> = Arc::new(EmptyExec::new(Arc::new(Schema::empty(
37 +         y()))));
38 +     let native: Arc<dyn ExecutionPlan> = Arc::new(DriverExtensionExec::new(
39 +         Arc::new(BoundDriverPlan {
40 +             descriptor: DriverDescriptor {
41 +                 owner: "fault-fixture@1:code".into(),
42 +                 plan_id: "committed-write".into(),
43 +             },
44 +             properties: empty.properties().clone(),
45 +             input_schemas: vec![],
46 +             binding: Arc::new(CommitThenLoseAcknowledgement(commits.clone())),
47 +         },
48 +         vec![],
49 +     )?);
50 +     let backend = ShuffleBackendKind::Flight {
51 +         compression: ShuffleCompression::None,
52 +     };
53 +     let options = JobSchedulerOptions::for_retry_test(3, backend.clone());
54 +     // Control: an ordinary failed task remains retryable with this configuration.
55 +     for (plan, is_native) in [(empty, false), (native.clone(), true)] {
56 +         let graph = JobGraph::try_new(
57 +             plan,
58 +             JobGraphOptions {
59 +                 shuffle_backend: backend.clone(),
60 +             },
61 +         );
62 +         let mut job =
63 +             JobDescriptor::try_new(graph, JobState::Draining, Arc::new(TaskContext::default()));
64 +         for stage in &mut job.stages {
65 +             for task in &mut stage.tasks {
66 +                 task.attempts.push(TaskAttemptDescriptor {
67 +                     state: TaskState::Failed,
68 +                     messages: vec![],
69 +                     cause: None,
70 +                     job_output_fetched: false,
71 +                 });
72 +             }
73 +         }
74 +         if is_native {
75 +             assert!(native.execute(0, Arc::new(TaskContext::default())).is_err());
76 +             assert_eq!(commits.load(Ordering::SeqCst), 1);
77 +             assert!(
78 +                 job.graph
79 +                     .stages()
80 +                     .iter()
81 +                     .any(|stage| matches!(stage.placement, TaskPlacement::Driver))

```

```

81 +         );
82 +     }
83 +     JobScheduler::update_task_regions(&mut job, &options);
84 +     assert_eq!(
85 +         job.regions
86 +         .iter()
87 +         .any(|region| matches!(region.state, TaskRegionState::Failed)),
88 +         is_native
89 +     );
90 +     assert!(
91 +         job.stages
92 +         .iter()
93 +         .all(|stage| stage.tasks.iter().all(|task| task.attempts.len() == 1))
94 +     );
95 + }
96 + assert_eq!(commits.load(Ordering::SeqCst), 1);
97 + Ok(())
98 +}

```

17. crates/sail-execution/src/driver/job_scheduler/options.rs

```

1 diff --git a/crates/sail-execution/src/driver/job_scheduler/options.rs b/crates/sail-e
2 index c341718cf6d7d073bace6be5889df06fa57377ac..4a67906436ad1f9ad416ba99b59951732418ff
3 de 100644
4 --- a/crates/sail-execution/src/driver/job_scheduler/options.rs
5 +++ b/crates/sail-execution/src/driver/job_scheduler/options.rs
6 @@ -23,3 +23,18 @@ impl From<&DriverOptions> for JobSchedulerOptions {
7     }
8 }
9 +
10 +#[cfg(test)]
11 +impl JobSchedulerOptions {
12 +    pub(super) fn for_retry_test(
13 +        task_max_attempts: usize,
14 +        shuffle_backend: ShuffleBackendKind,
15 +    ) -> Self {
16 +        Self {
17 +            session_id: "fault-fixture".into(),
18 +            task_launch_timeout: Duration::from_secs(30),
19 +            task_max_attempts,
20 +            shuffle_backend,
21 +        }
22 +    }
23 +}

```

18. crates/sail-execution/src/job_graph/planner.rs

```

1 diff --git a/crates/sail-execution/src/job_graph/planner.rs b/crates/sail-execution/sr
2 index 0540ef15deaa10ff5f11c743ca559bf4e09b1510..284da4c5fdb16f1eb63283eee2fe543ff125c5
3 87 100644
4 --- a/crates/sail-execution/src/job_graph/planner.rs
5 +++ b/crates/sail-execution/src/job_graph/planner.rs
6 @@ -490,13 +490,7 @@ fn plan_job_graph_stages(
7     let plan =
8         create_rescale_input(child, coalesce.output_partitions(), graph, scalar_c
9         ontext)?;
10     PlannedSubtree::without_pending_scalar_subquery_expr(plan)
11 } else if subtree.plan.is::<SystemTableExec>()
12 | subtree.plan.is::<CatalogCommandExec>()
13 | subtree.plan.is::<FileDeleteExec>()
14 | subtree.plan.is::<DeltaCommitExec>()
15 | subtree.plan.is::<IcebergCommitExec>()
16 | subtree.plan.is::<RemoteCheckpointCommitExec>()
17 {
18 } else if is_driver_stage_plan(&subtree.plan) {
19     if matches!(driver_stage_handling, DriverStageHandling::PreserveRoot) {
20         subtree.into_planned_subtree()
21     } else {
22 @@ -642,6 +636,7 @@ fn is_driver_stage_plan(plan: &Arc<dyn ExecutionPlan>) -> bool {
23 }

```

```

22
23     plan.is::<SystemTableExec>()
24 +     || plan.is::<sail_common_datafusion::driver_extension::DriverExtensionExec>()
25     || plan.is::<CatalogCommandExec>()
26     || plan.is::<FileDeleteExec>()
27     || plan.is::<DeltaCommitExec>()

```

19. crates/sail-execution/src/proto/codec.rs

```

1 diff --git a/crates/sail-execution/src/proto/codec.rs b/crates/sail-execution/src/proto/codec.rs
2 index 86a93571332dbf305f1c64160815f9e7e56c4e93..1763c695e23cd7283a9cbbc1f02ef8490b8463
3 --- a/crates/sail-execution/src/proto/codec.rs
4 +++ b/crates/sail-execution/src/proto/codec.rs
5 @@ -95,6 +95,8 @@ use sail_common_datafusion::catalog::{
6     CatalogPartitionField, LakehouseExecutionContext, PartitionTransform,
7 };
8 use sail_common_datafusion::datasource::PhysicalSinkMode;
9 use sail_common_datafusion::driver_extension::{DRIVER_CODEC_PREFIX, DriverExtensionEx
10 ec};
11 +use sail_common_datafusion::native_scalar::{decode_scalar, encode_scalar};
12 use sail_common_datafusion::schema_evolution::{
13     SchemaEvolutionCastColumnExpr, SchemaEvolutionDefaultExpr,
14     SchemaEvolutionPhysicalExprAdapterFactoryWithMatching, SchemaEvolutionTimezoneMod
15 e,
16 @@ -352,6 +354,9 @@ impl PhysicalExtensionCodec for RemoteExecutionCodec {
17     ctx: &TaskContext,
18     proto_converter: &dyn PhysicalProtoConverterExtension,
19 ) -> Result<Arc<dyn ExecutionPlan>> {
20 +    if buf.starts_with(DRIVER_CODEC_PREFIX) {
21 +        return DriverExtensionExec::decode(buf, inputs, ctx);
22 +    }
23     let node = ExtendedPhysicalPlanNode::decode(buf)
24     .map_err(|e| plan_datafusion_err!("failed to decode plan: {e}"));
25     let ExtendedPhysicalPlanNode { node_kind } = node;
26 @@ -1904,6 +1909,9 @@ impl PhysicalExtensionCodec for RemoteExecutionCodec {
27     buf: &mut Vec<u8>,
28     proto_converter: &dyn PhysicalProtoConverterExtension,
29 ) -> Result<()> {
30 +    if let Some(native) = node.downcast_ref::<DriverExtensionExec>() {
31 +        return native.encode(buf);
32 +    }
33     let node_kind = if let Some(range) = node.downcast_ref::<RangeExec>() {
34         let schema = try_encode_schema(range.original_schema().as_ref());
35         let projection = self.try_encode_projection(range.projection());
36 @@ -3000,6 +3008,9 @@ impl PhysicalExtensionCodec for RemoteExecutionCodec {
37 }
38
39 fn try_decode_udf(&self, name: &str, buf: &[u8]) -> Result<Arc<ScalarUDF>> {
40 +    if buf.starts_with(sail_common_datafusion::native_scalar::SCALAR_CODEC_PREFIX
41 ) {
42 +        return decode_scalar(name, buf);
43 +    }
44     // TODO: Implement custom registry to avoid codec for built-in functions.
45     // The `match name` below has no session-registry fallback, so every
46     // scalar UDF needs an explicit arm or distributed decode fails with
47 @@ -3472,6 +3483,9 @@ impl PhysicalExtensionCodec for RemoteExecutionCodec {
48 }
49
50 fn try_encode_udf(&self, node: &ScalarUDF, buf: &mut Vec<u8>) -> Result<()> {
51 +    if encode_scalar(node, buf)? {
52 +        return Ok(());
53 +    }
54     // TODO: Implement custom registry to avoid codec for built-in functions
55     let node_inner = node.inner();
56     let udf_kind: UdfKind = if node_inner.is::<ArrayElement>()

```

20. crates/sail-execution/src/proto/converter.rs

```

1 diff --git a/crates/sail-execution/src/proto/converter.rs b/crates/sail-execution/src/
2 proto/converter.rs

```

```

2 index 5dbc2e4edd8a7249ddfa20181f88126b9748ddc2..8682125789aeb55710e4776df1f69f48aa6d68
b5 100644
3 --- a/crates/sail-execution/src/proto/converter.rs
4 +++ b/crates/sail-execution/src/proto/converter.rs
5 @@ -69,29 +69,38 @@ impl PhysicalProtoConverterExtension for RemotePhysicalProtoConver
ter {
6     input_schema: &Schema,
7     ctx: &PhysicalPlanDecodeContext<'_,>,
8     ) -> Result<Arc<dyn PhysicalExpr>> {
9 -     let decoded = match decode_remote_expr_kind(proto)? {
10 -         Some((ExprKind::HigherOrderUdf(node), inputs)) => {
11 -             self.higher_order_proto_to_expr(node, inputs, input_schema, ctx)
12 -         }
13 -         Some((ExprKind::LambdaVariable(node), _)) => {
14 -             let field = try_decode_field_ref(&node.field)?;
15 -             let index = usize::try_from(node.index).map_err(|_| {
16 -                 plan_datafusion_err!(
17 -                     "LambdaVariable index {} does not fit in usize",
18 -                     node.index
19 -                 )
20 -             })?;
21 -             Ok(Arc::new(LambdaVariable::new(index, field)) as Arc<dyn PhysicalExp
r>)
22 -         }
23 -         Some((ExprKind::Lambda(node), inputs)) => {
24 -             let [body] = inputs else {
25 -                 return plan_err!("LambdaExpr expects exactly one input, got {}",
inputs.len());
26 -             };
27 -             let body = self.proto_to_physical_expr(body, input_schema, ctx)?;
28 -             Ok(Arc::new(LambdaExpr::try_new(node.params, body)?) as Arc<dyn Physi
calExpr>)
29 -         }
30 -         _ => self.default_proto_to_physical_expr(proto, input_schema, ctx),
31 -     };
32 +     let decoded = if let Some(expr) =
33 +         super::native_expr::decode(proto, input_schema, ctx, self)?
34 +     {
35 +         expr
36 +     } else {
37 +         match decode_remote_expr_kind(proto)? {
38 +             Some((ExprKind::HigherOrderUdf(node), inputs)) => {
39 +                 self.higher_order_proto_to_expr(node, inputs, input_schema, ctx)
40 +             }
41 +             Some((ExprKind::LambdaVariable(node), _)) => {
42 +                 let field = try_decode_field_ref(&node.field)?;
43 +                 let index = usize::try_from(node.index).map_err(|_| {
44 +                     plan_datafusion_err!(
45 +                         "LambdaVariable index {} does not fit in usize",
46 +                         node.index
47 +                     )
48 +                 })?;
49 +                 Ok(Arc::new(LambdaVariable::new(index, field)) as Arc<dyn Physica
lExpr>)
50 +             }
51 +             Some((ExprKind::Lambda(node), inputs)) => {
52 +                 let [body] = inputs else {
53 +                     return plan_err!(
54 +                         "LambdaExpr expects exactly one input, got {}",
55 +                         inputs.len()
56 +                     );
57 +                 };
58 +                 let body = self.proto_to_physical_expr(body, input_schema, ctx)?;
59 +                 Ok(Arc::new(LambdaExpr::try_new(node.params, body)?) as Arc<dyn P
hysicalExpr>)
60 +             }
61 +             _ => self.default_proto_to_physical_expr(proto, input_schema, ctx),
62 +         };
63 +     };
64
65     let Some(expression_id) = proto.expr_id else {
66         return Ok(decoded);
67 @@ -118,6 +127,9 @@ impl PhysicalProtoConverterExtension for RemotePhysicalProtoConver
ter {
68     expr: &Arc<dyn PhysicalExpr>,
69     codec: &dyn PhysicalExtensionCodec,
70     ) -> Result<PhysicalExprNode> {

```

```

71 +         if let Some(proto) = super::native_expr::encode(expr, codec, self)? {
72 +             return Ok(proto);
73 +         }
74         if let Some(hof) = expr.downcast_ref:::<HigherOrderFunctionExpr>() {
75             return self.higher_order_expr_to_proto(expr, hof, codec);
76         }

```

21. crates/sail-execution/src/proto/mod.rs

```

1 diff --git a/crates/sail-execution/src/proto/mod.rs b/crates/sail-execution/src/proto/
  mod.rs
2 index 49599ef748d665216da9efc52a964c861bbadb14..e1448edc33157b2c4872d445881fafaa7fe230
  75 100644
3 --- a/crates/sail-execution/src/proto/mod.rs
4 +++ b/crates/sail-execution/src/proto/mod.rs
5 @@ -2,6 +2,7 @@ mod codec;
6  mod converter;
7  mod decode;
8  mod encode;
9  +mod native_expr;
10
11 pub use codec::RemoteExecutionCodec;
12 #[cfg(test)]

```

22. crates/sail-execution/src/proto/native_expr.rs

```

1 diff --git a/crates/sail-execution/src/proto/native_expr.rs b/crates/sail-execution/sr
  c/proto/native_expr.rs
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..562865e02aba05fb1c560e3fde7142bbc34807
  a8
4 --- /dev/null
5 +++ b/crates/sail-execution/src/proto/native_expr.rs
6 @@ -0,0 +1,358 @@
7 +/// Native scalar return fields carry extension metadata that DataFusion's
8 +/// standard scalar-expression protobuf currently omits.
9 +use std::sync::Arc;
10 +
11 +use datafusion::arrow::datatypes::Schema;
12 +use datafusion::common::metadata::FieldMetadata;
13 +use datafusion::common::{Result, plan_datafusion_err};
14 +use datafusion::execution::FunctionRegistry;
15 +use datafusion::physical_expr::expressions::{CastExpr, Literal};
16 +use datafusion::physical_expr::{PhysicalExpr, ScalarFunctionExpr};
17 +use datafusion_proto::physical_plan::to_proto::serialize_physical_expr_with_converter
  ;
18 +use datafusion_proto::physical_plan::{
19 +    PhysicalExtensionCodec, PhysicalPlanDecodeContext, PhysicalProtoConverterExtensio
  n,
20 +};
21 +use datafusion_proto::protobuf::{PhysicalExprNode, PhysicalExtensionExprNode, physica
  l_expr_node};
22 +use sail_common_datafusion::native_scalar::OwnedScalar;
23 +use serde::{Deserialize, Serialize};
24 +
25 +use super::decode::try_decode_field_ref;
26 +use super::encode::try_encode_field_ref;
27 +
28 +const PREFIX: &[u8] = b"SAIL_NATIVE_SCALAR_EXPR_V1\0";
29 +const LITERAL_PREFIX: &[u8] = b"SAIL_METADATA_LITERAL_V1\0";
30 +const CAST_PREFIX: &[u8] = b"SAIL_METADATA_CAST_V1\0";
31 +const MAX_DESCRIPTOR: usize = 1024 * 1024;
32 +
33 +#[derive(Serialize, Deserialize)]
34 +#[serde(deny_unknown_fields)]
35 +struct Descriptor {
36 +    name: String,
37 +    udf: Vec<u8>,
38 +    field: Vec<u8>,
39 +    nullable: bool,
40 +}
41 +
42 +pub(super) fn encode(

```

```

43 +     expr: &Arc<dyn PhysicalExpr>,
44 +     codec: &dyn PhysicalExtensionCodec,
45 +     converter: &dyn PhysicalProtoConverterExtension,
46 + ) -> Result<Option<PhysicalExprNode>> {
47 +     if let Some(cast) = expr.downcast_ref::<CastExpr>()
48 +         && cast.has_explicit_metadata()
49 +         && !cast.target_field().metadata().is_empty()
50 +     {
51 +         let bytes = try_encode_field_ref(cast.target_field())?;
52 +         if bytes.len() > MAX_DESCRIPTOR {
53 +             return Err(plan_datafusion_err!("cast field descriptor too large"));
54 +         }
55 +         return Ok(Some(PhysicalExprNode {
56 +             expr_type: Some(physical_expr_node::ExprType::Extension(
57 +                 PhysicalExtensionExprNode {
58 +                     expr: [CAST_PREFIX, bytes.as_slice()].concat(),
59 +                     inputs: vec![serialize_physical_expr_with_converter(
60 +                         expr, codec, converter,
61 +                     )?],
62 +                 },
63 +             )),
64 +             expr_id: expr.expression_id(),
65 +         }));
66 +     }
67 +     if expr.downcast_ref::<Literal>().is_some() {
68 +         let field = expr.return_field(&Schema::empty())?;
69 +         if !field.metadata().is_empty() {
70 +             let bytes = try_encode_field_ref(&field)?;
71 +             if bytes.len() > MAX_DESCRIPTOR {
72 +                 return Err(plan_datafusion_err!("literal field descriptor too large"));
73 +             }
74 +             return Ok(Some(PhysicalExprNode {
75 +                 expr_type: Some(physical_expr_node::ExprType::Extension(
76 +                     PhysicalExtensionExprNode {
77 +                         expr: [LITERAL_PREFIX, bytes.as_slice()].concat(),
78 +                         inputs: vec![serialize_physical_expr_with_converter(
79 +                             expr, codec, converter,
80 +                         )?],
81 +                     },
82 +                 )),
83 +                 expr_id: expr.expression_id(),
84 +             }));
85 +         }
86 +     }
87 +     let Some(scalar) = expr.downcast_ref::<ScalarFunctionExpr>() else {
88 +         return Ok(None);
89 +     };
90 +     let field = expr.return_field(&Schema::empty())?;
91 +     if !scalar.fun().inner().is::<OwnedScalar>() && field.metadata().is_empty() {
92 +         return Ok(None);
93 +     }
94 +     let mut udf = vec![];
95 +     codec.try_encode_udf(scalar.fun(), &mut udf)?;
96 +     let descriptor = Descriptor {
97 +         name: scalar.fun().name().to_owned(),
98 +         udf,
99 +         field: try_encode_field_ref(&field)?,
100 +         nullable: scalar.nullable(),
101 +     };
102 +     let bytes = serde_json::to_vec(&descriptor)
103 +         .map_err(|e| plan_datafusion_err!("native scalar descriptor: {e}"))?;
104 +     if bytes.len() > MAX_DESCRIPTOR {
105 +         return Err(plan_datafusion_err!("native scalar descriptor too large"));
106 +     }
107 +     let inputs = scalar
108 +         .args()
109 +         .iter()
110 +         .map(|arg| converter.physical_expr_to_proto(arg, codec))
111 +         .collect::<Result<Vec<PhysicalExprNode>>>()?;
112 +     Ok(Some(PhysicalExprNode {
113 +         expr_type: Some(physical_expr_node::ExprType::Extension(
114 +             PhysicalExtensionExprNode {
115 +                 expr: [PREFIX, bytes.as_slice()].concat(),
116 +                 inputs,
117 +             },
118 +         )),
119 +         expr_id: expr.expression_id(),

```

```

120 +   )))
121 +}
122 +
123 +pub(super) fn decode(
124 +    proto: &PhysicalExprNode,
125 +    schema: &Schema,
126 +    ctx: &PhysicalPlanDecodeContext<'>,
127 +    converter: &dyn PhysicalProtoConverterExtension,
128 +) -> Result<Option<Arc<dyn PhysicalExpr>>> {
129 +    let Some(physical_expr_node::ExprType::Extension(node)) = &proto.expr_type else {
130 +        return Ok(None);
131 +    };
132 +    if let Some(bytes) = node.expr.strip_prefix(CAST_PREFIX) {
133 +        if bytes.len() > MAX_DESCRIPTOR {
134 +            return Err(plan_datafusion_err!("cast field descriptor too large"));
135 +        }
136 +        let [input] = node.inputs.as_slice() else {
137 +            return Err(plan_datafusion_err!(
138 +                "metadata cast requires one cast input"
139 +            ));
140 +        };
141 +        if !matches!(input.expr_type, Some(physical_expr_node::ExprType::Cast(_))) {
142 +            return Err(plan_datafusion_err!("metadata cast input is not a cast"));
143 +        }
144 +        let decoded = converter.proto_to_physical_expr(input, schema, ctx)?;
145 +        let cast = decoded
146 +            .downcast_ref:::<CastExpr>()
147 +            .ok_or_else(|| plan_datafusion_err!("metadata cast input is not a cast"))
148 +            ?;
149 +        let field = try_decode_field_ref(bytes)?;
150 +        if field.data_type() != cast.cast_type() {
151 +            return Err(plan_datafusion_err!("metadata cast type mismatch"));
152 +        }
153 +        return Ok(Some(Arc::new(CastExpr::new_with_target_field(
154 +            Arc::clone(cast.expr()),
155 +            field,
156 +            Some(cast.cast_options().clone()),
157 +        ))));
158 +    }
159 +    if let Some(bytes) = node.expr.strip_prefix(LITERAL_PREFIX) {
160 +        if bytes.len() > MAX_DESCRIPTOR {
161 +            return Err(plan_datafusion_err!("literal field descriptor too large"));
162 +        }
163 +        let [input] = node.inputs.as_slice() else {
164 +            return Err(plan_datafusion_err!(
165 +                "metadata literal requires one literal input"
166 +            ));
167 +        };
168 +        if !matches!(
169 +            input.expr_type,
170 +            Some(physical_expr_node::ExprType::Literal(_))
171 +        ) {
172 +            return Err(plan_datafusion_err!(
173 +                "metadata literal input is not a literal"
174 +            ));
175 +        }
176 +        let value = converter.proto_to_physical_expr(input, schema, ctx)?;
177 +        let literal = value
178 +            .downcast_ref:::<Literal>()
179 +            .ok_or_else(|| plan_datafusion_err!("metadata literal input is not a literal"))
180 +            ?;
181 +        let field = try_decode_field_ref(bytes)?;
182 +        if field.data_type() != &literal.value().data_type() {
183 +            return Err(plan_datafusion_err!("metadata literal type mismatch"));
184 +        }
185 +        let restored = Literal::new_with_metadata(
186 +            literal.value().clone(),
187 +            Some(FieldMetadata::new_from_field(&field)),
188 +        );
189 +        return Ok(Some(Arc::new(restored)));
190 +    }
191 +    let Some(bytes) = node.expr.strip_prefix(PREFIX) else {
192 +        return Ok(None);
193 +    };
194 +    if bytes.len() > MAX_DESCRIPTOR {
195 +        return Err(plan_datafusion_err!("native scalar descriptor too large"));
196 +    }
197 +    let descriptor: Descriptor = serde_json::from_slice(bytes)

```

```

196 +         .map_err(|e| plan_datafusion_err!("native scalar descriptor: {e}"))?;
197 + // Match DataFusion's standard ScalarUdf decoder: an empty definition means
198 + // a function from the task registry (e.g. get_field), not an empty Sail UDF
199 + // descriptor. Native functions always carry their exact identity bytes.
200 + let udf = if descriptor.udf.is_empty() {
201 +     ctx.task_ctx()
202 +     .udf(&descriptor.name)
203 +     .or_else(|_| ctx.codec().try_decode_udf(&descriptor.name, &[]))?
204 + } else {
205 +     ctx.codec()
206 +     .try_decode_udf(&descriptor.name, &descriptor.udf)?
207 + };
208 + let field = try_decode_field_ref(&descriptor.field)?;
209 + let args = node
210 +     .inputs
211 +     .iter()
212 +     .map(|arg| converter.proto_to_physical_expr(arg, schema, ctx))
213 +     .collect::<Result<_>>()?;
214 + Ok(Some(Arc::new(
215 +     ScalarFunctionExpr::new(
216 +         &descriptor.name,
217 +         udf,
218 +         args,
219 +         field,
220 +         Arc::clone(ctx.task_ctx().session_config().options()),
221 +     )
222 +     .with_nullable(descriptor.nullable),
223 + )))
224 + }
225 +
226 + #[cfg(test)]
227 + mod tests {
228 +     use datafusion::arrow::datatypes::{DataType, Field};
229 +     use datafusion::execution::TaskContext;
230 +     use datafusion::logical_expr::ScalarUDF;
231 +     use datafusion::physical_expr::expressions::Column;
232 +     use sail_common_datafusion::native_scalar::retain_scalar;
233 +
234 +     use super::*;
235 +     use crate::proto::{
236 +         RemoteExecutionCodec, decode_remote_physical_expr, encode_remote_physical_exp
237 +     };
238 +
239 +     #[test]
240 +     fn native_scalar_expression_round_trip_preserves_full_return_field() -> Result<()>
241 +     {
242 +         let task = TaskContext::default();
243 +         let udf = ScalarUDF::new_from_impl(OwnedScalar {
244 +             name: "metadata_fixture".into(),
245 +             identity: "fixture@1:metadata".into(),
246 +             udf: (*datafusion::functions::math::abs()).clone(),
247 +             owner: Arc::new(()),
248 +         });
249 +         retain_scalar(udf.clone())?;
250 +         let field = Arc::new(
251 +             Field::new("geometry", DataType::Float64, false).with_metadata(
252 +                 [
253 +                     ("ARROW:extension:name".into(), "fixture.geometry".into()),
254 +                     ("ARROW:extension:metadata".into(),
255 +                     "{\\"crs\\":\\"EPSG:4326\\"}".into()),
256 +                 ],
257 +             ).into(),
258 +         );
259 +         let schema = Schema::new(vec![Field::new("x", DataType::Float64, false)]);
260 +         let expr: Arc<dyn PhysicalExpr> = Arc::new(
261 +             ScalarFunctionExpr::new(
262 +                 "metadata_fixture",
263 +                 Arc::new(udf),
264 +                 vec![Arc::new(Column::new("x", 0))],
265 +                 field.clone(),
266 +                 Arc::clone(task.session_config().options()),
267 +             ).with_nullable(false),
268 +         );
269 +     }
270 + }

```

```

272 +     let bytes = encode_remote_physical_expr(&RemoteExecutionCodec, &expr)?;
273 +     let decoded = decode_remote_physical_expr(&task, &RemoteExecutionCodec, &bytes, &schema)?;
274 +     assert_eq!(decoded.return_field(&schema)?, field);
275 +     assert!(!decoded.nullable(&schema)?);
276 +     assert_eq!(decoded.children().len(), 1);
277 +     let literal: Arc<dyn PhysicalExpr> = Arc::new(Literal::new_with_metadata(
278 +         datafusion::common::ScalarValue::Float64(Some(2.0)),
279 +         Some(FieldMetadata::new_from_field(&field)),
280 +     ));
281 +     let bytes = encode_remote_physical_expr(&RemoteExecutionCodec, &literal)?;
282 +     let decoded = decode_remote_physical_expr(&task, &RemoteExecutionCodec, &bytes, &schema)?;
283 +     assert_eq!(
284 +         decoded.return_field(&schema)?,
285 +         literal.return_field(&schema)?
286 +     );
287 +     assert_eq!(
288 +         decoded
289 +             .downcast_ref::<Literal>()
290 +             .ok_or_else(|| plan_datafusion_err!("expected restored literal"))?
291 +             .value(),
292 +         &datafusion::common::ScalarValue::Float64(Some(2.0))
293 +     );
294 +     Ok(())
295 + }
296 +
297 + #[test]
298 + fn builtin_geometry_and_explicit_cast_preserve_metadata_on_workers() -> Result<()>
299 + {
300 +     use datafusion::logical_expr::ReturnFieldArgs;
301 +     use sail_function::scalar::geo::st_geomfromwkb::StGeomFromWKB;
302 +
303 +     let task = TaskContext::default();
304 +     let schema = Schema::new(vec![Field::new("wkb", DataType::Binary, true)]);
305 +     let udf = Arc::new(ScalarUDF::from(StGeomFromWKB::new()));
306 +     let field = udf.return_field_from_args(ReturnFieldArgs {
307 +         arg_fields: schema.fields(),
308 +         scalar_arguments: &[None],
309 +     })?;
310 +     let scalar: Arc<dyn PhysicalExpr> = Arc::new(ScalarFunctionExpr::new(
311 +         "st_geomfromwkb",
312 +         udf,
313 +         vec![Arc::new(Column::new("wkb", 0))],
314 +         field.clone(),
315 +         Arc::clone(task.session_config().options()),
316 +     ));
317 +     let cast: Arc<dyn PhysicalExpr> = Arc::new(CastExpr::new_with_target_field(
318 +         Arc::new(Column::new("wkb", 0)),
319 +         field.clone(),
320 +         None,
321 +     ));
322 +     for original in [scalar, cast] {
323 +         let bytes = encode_remote_physical_expr(&RemoteExecutionCodec, &original)?;
324 +         let restored =
325 +             decode_remote_physical_expr(&task, &RemoteExecutionCodec, &bytes, &schema)?;
326 +         assert_eq!(
327 +             restored.return_field(&schema)?,
328 +             original.return_field(&schema)?
329 +         );
330 +         assert_eq!(restored.return_field(&schema)?.metadata(), field.metadata());
331 +     }
332 +     Ok(())
333 + }
334 +
335 + #[test]
336 + fn metadata_bearing_struct_access_uses_the_task_function_registry() -> Result<()>
337 + {
338 +     let context = datafusion::prelude::SessionContext::new();
339 +     let task = context.task_ctx();
340 +     let field =
341 +         Arc::new(Field::new("value", DataType::Utf8, true).with_metadata(
342 +             [("description".into(), "ordinary property metadata".into())].into(),
343 +         ));
344 +     let schema = Schema::new(vec![Field::new(

```

```

343 +         "node",
344 +         DataType::Struct(vec![field.clone().into()],
345 +         true,
346 +     ));
347 +     let expr: Arc<dyn PhysicalExpr> = Arc::new(ScalarFunctionExpr::new(
348 +         "get_field",
349 +         datafusion::functions::core::get_field(),
350 +         vec![
351 +             Arc::new(Column::new("node", 0)),
352 +             Arc::new(Literal::new(datafusion::common::ScalarValue::Utf8(Some(
353 +                 "value".into(),
354 +             )))),
355 +         ],
356 +         field.clone(),
357 +         Arc::clone(task.session_config().options()),
358 +     ));
359 +     let bytes = encode_remote_physical_expr(&RemoteExecutionCodec, &expr)?;
360 +     let decoded = decode_remote_physical_expr(&task, &RemoteExecutionCodec, &bytes, &schema)?;
361 +     assert_eq!(decoded.return_field(&schema)?, field);
362 +     Ok(())
363 + }
364 +}

```

23. crates/sail-execution/src/task_runner/actor/handler.rs

```

1 diff --git a/crates/sail-execution/src/task_runner/actor/handler.rs b/crates/sail-execution/src/task_runner/actor/handler.rs
2 index 20a37c4fc4feca2eca7ef9e91c78b7c286167bec83..7a4d552117e7d853967f922951332bd8fc05cd96 100644
3 --- a/crates/sail-execution/src/task_runner/actor/handler.rs
4 +++ b/crates/sail-execution/src/task_runner/actor/handler.rs
5 @@ -4,7 +4,7 @@ use datafusion::arrow::datatypes::Schema;
6  use datafusion::common::DataFusionError;
7  use datafusion::execution::TaskContext;
8  use datafusion_proto::protobuf::PhysicalPlanNode;
9  -use log::{error, warn};
10 +use log::{debug, error, warn};
11  use prost::Message;
12  use sail_common::actor::{ActorAction, ActorContext};
13  use sail_common_datafusion::error::CommonErrorCause;
14 @@ -125,12 +125,17 @@ impl TaskRunnerActor {
15     });
16     }
17     TaskRunnerPlacement::Worker {
18 +         worker_id,
19         sequence,
20         driver,
21         worker,
22         retry_strategy,
23         ..
24     } => {
25 +         debug!(
26 +             "worker_task_status worker_id={worker_id} job_id={} stage={} partition={} attempt={} status={status}",
27 +             key.job_id, key.stage, key.partition, key.attempt
28 +         );
29         let seq = *sequence;
30         *sequence = match seq.checked_add(1) {
31             Some(s) => s,

```

24. crates/sail-execution/src/worker_manager/kubernetes.rs

```

1 diff --git a/crates/sail-execution/src/worker_manager/kubernetes.rs b/crates/sail-execution/src/worker_manager/kubernetes.rs
2 index c5a43e026df341f0730c3fe9b0ad4693a02b1cc4..fad04a8993b1c050dae1fbae95c28f48481f61e5 100644
3 --- a/crates/sail-execution/src/worker_manager/kubernetes.rs
4 +++ b/crates/sail-execution/src/worker_manager/kubernetes.rs
5 @@ -169,6 +169,11 @@ impl KubernetesWorkerService {
6     }
7     };
8     let mut env = vec![

```

```

9 +         EnvVar {
10 +             name: "SAIL_EXPERIMENTAL_EXTENSIONS".to_string(),
11 +             value: Some(env::var("SAIL_EXPERIMENTAL_EXTENSIONS").unwrap_or_default
12 +             t()),
13 +             value_from: None,
14 +         },
15 +     EnvVar {
16 +         name: "RUST_LOG".to_string(),
17 +         value: Some(env::var("RUST_LOG").unwrap_or("info".to_string())),

```

25. crates/sail-execution/src/worker_manager/mod.rs

```

1 diff --git a/crates/sail-execution/src/worker_manager/mod.rs b/crates/sail-execution/s
rc/worker_manager/mod.rs
2 index 044c53edc4586c2d3bde741f26e4239573895dee..e946f1329d2dd3a0b81574ed0ac0237f422878
c9 100644
3 --- a/crates/sail-execution/src/worker_manager/mod.rs
4 +++ b/crates/sail-execution/src/worker_manager/mod.rs
5 @@ -1,6 +1,8 @@
6 mod kubernetes;
7 mod local;
8 mod options;
9 +mod process;
10 +mod process_command;
11
12 use futures::future::BoxFuture;
13 pub(crate) use options::WorkerLaunchOptions;
14 @@ -35,3 +37,4 @@ pub trait WorkerManager: Send + Sync + 'static {
15
16 pub use kubernetes::{KubernetesWorkerManager, KubernetesWorkerManagerOptions};
17 pub use local::LocalWorkerManager;
18 +pub use process::ProcessWorkerManager;

```

26. crates/sail-execution/src/worker_manager/process.rs

```

1 diff --git a/crates/sail-execution/src/worker_manager/process.rs b/crates/sail-executi
on/src/worker_manager/process.rs
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..4bf0035880b1652fd69ebf16d3970ac6698bc4
2a
4 --- /dev/null
5 +++ b/crates/sail-execution/src/worker_manager/process.rs
6 @@ -0,0 +1,104 @@
7 +//! Experimental local-cluster workers launched as separate Sail executables.
8 +use std::sync::Arc;
9 +use std::time::Duration;
10 +
11 +use futures::future::BoxFuture;
12 +use sail_common::actor::ActorSystem;
13 +use sail_common::config::{CliConfigEnv, ClusterConfigEnv, ExecutionConfigEnv};
14 +use tokio::process::Child;
15 +use tokio::sync::Mutex;
16 +
17 +use super::process_command::{WORKER_COMMAND_ENV, worker_command};
18 +use crate::error::{ExecutionError, ExecutionResult};
19 +use crate::id::WorkerId;
20 +use crate::worker_manager::{WorkerLaunchOptions, WorkerManager};
21 +
22 +#[derive(Default)]
23 +pub struct ProcessWorkerManager {
24 +    children: Arc<Mutex<Vec<Child>>>,
25 +}
26 +
27 +#[tonic::async_trait]
28 +impl WorkerManager for ProcessWorkerManager {
29 +    fn launch_worker(
30 +        &self,
31 +        _system: &mut ActorSystem,
32 +        id: WorkerId,
33 +        options: WorkerLaunchOptions,
34 +    ) -> BoxFuture<'static, ExecutionResult<>>> {
35 +        let children = self.children.clone();
36 +        Box::pin(async move {

```

```

37 +         let configured = std::env::var(WORKER_COMMAND_ENV).ok();
38 +         let mut command = worker_command(configured.as_deref());
39 +         command
40 +             .kill_on_drop(true)
41 +             .env_remove(CliConfigEnv::RUN_PYTHON)
42 +             .env(ClusterConfigEnv::ENABLE_TLS, options.enable_tls.to_string())
43 +             .env(ClusterConfigEnv::SESSION_ID, &options.session_id)
44 +             .env(
45 +                 ClusterConfigEnv::DRIVER_ID,
46 +                 u64::from(options.driver_id).to_string(),
47 +             )
48 +             .env(ClusterConfigEnv::WORKER_ID, u64::from(id).to_string())
49 +             .env(
50 +                 ClusterConfigEnv::DRIVER_EXTERNAL_HOST,
51 +                 &options.driver_external_host,
52 +             )
53 +             .env(
54 +                 ClusterConfigEnv::DRIVER_EXTERNAL_PORT,
55 +                 options.driver_external_port.to_string(),
56 +             )
57 +             .env(ClusterConfigEnv::WORKER_LISTEN_HOST, "127.0.0.1")
58 +             .env("SAIL_CLUSTER__WORKER_LISTEN_PORT", "0")
59 +             .env(ClusterConfigEnv::WORKER_EXTERNAL_HOST, "127.0.0.1")
60 +             .env("SAIL_CLUSTER__WORKER_EXTERNAL_PORT", "0")
61 +             .env(
62 +                 ClusterConfigEnv::WORKER_HEARTBEAT_INTERVAL_SECS,
63 +                 options.worker_heartbeat_interval.as_secs().to_string(),
64 +             )
65 +             .env(
66 +                 ClusterConfigEnv::TASK_STREAM_BUFFER,
67 +                 options.task_stream_buffer.to_string(),
68 +             )
69 +             .env(
70 +                 ClusterConfigEnv::TASK_STREAM_CREATION_TIMEOUT_SECS,
71 +                 options.task_stream_creation_timeout.as_secs().to_string(),
72 +             )
73 +             .env(
74 +                 ExecutionConfigEnv::BATCH_SIZE,
75 +                 options.batch_size.to_string(),
76 +             );
77 +         if let Some(path) = std::env::var_os("SAIL_EXPERIMENTAL_WORKER_PYTHONPATH") {
78 +             command.env("PYTHONPATH", path);
79 +         }
80 +         let child = command
81 +             .spawn()
82 +             .map_err(|e| ExecutionError::InternalError(format!("launch worker {id}
83 +             ): {e}")))?;
84 +         log::info!(
85 +             "extension process worker {id}: pid={:?}, driver_pid={}, session={}",
86 +             child.id(),
87 +             std::process::id(),
88 +             options.session_id
89 +         );
90 +         children.lock().await.push(child);
91 +         Ok(())
92 +     })
93 + }
94 +
95 + async fn stop(&self) -> ExecutionResult<()> {
96 +     for mut child in self.children.lock().await.drain(..) {
97 +         match tokio::time::timeout(Duration::from_secs(3), child.wait()).await {
98 +             Ok(result) => {
99 +                 result.map_err(|e| ExecutionError::InternalError(e.to_string()))?;
100 +             }
101 +             Err(_) => {
102 +                 child
103 +                     .kill()
104 +                     .await
105 +                     .map_err(|e| ExecutionError::InternalError(e.to_string()))?;
106 +             }
107 +         }
108 +     }
109 +     Ok(())
110 + }

```

27. [crates/sail-execution/src/worker_manager/process_command.rs](#)

```

1 diff --git a/crates/sail-execution/src/worker_manager/process_command.rs b/crates/sail-
2 -execution/src/worker_manager/process_command.rs
3 new file mode 100644
4 index 0000000000000000000000000000000000000000..e0d60b3229585804e20b2edd47ed2c22f13966
5 e3
6 --- /dev/null
7 +++ b/crates/sail-execution/src/worker_manager/process_command.rs
8 @@ -0,0 +1,63 @@
9 +#!/ Trusted startup configuration for an external worker launcher.
10 +use tokio::process::Command;
11 +
12 +use crate::error::{ExecutionError, ExecutionResult};
13 +
14 +pub(super) const WORKER_COMMAND_ENV: &str = "SAIL_EXPERIMENTAL_WORKER_COMMAND";
15 +
16 +/// A configured launcher receives the assigned worker/session settings through
17 +/// its environment. Its argv is used exactly, without a shell or appended args.
18 +/// The launcher must remain alive while its worker runs and stop that worker
19 +/// when the launcher closes. This is administrator configuration, never RPC data.
20 +pub(super) fn worker_command(configured: Option<&str>) -> ExecutionResult<Command> {
21 +    if let Some(configured) = configured {
22 +        let argv: Vec<String> = serde_json::from_str(&configured).map_err(|error| {
23 +            ExecutionError::InvalidArgument(format!("{{WORKER_COMMAND_ENV}}: {{error}}"))
24 +        })?;
25 +        let Some(program) = argv.first().filter(|program| !program.is_empty()) else {
26 +            return Err(ExecutionError::InvalidArgument(format!(
27 +                "{{WORKER_COMMAND_ENV}} requires a nonempty executable argv"
28 +            )));
29 +        };
30 +        if argv.iter().any(|argument| argument.contains('\0')) {
31 +            return Err(ExecutionError::InvalidArgument(format!(
32 +                "{{WORKER_COMMAND_ENV}} arguments cannot contain NUL"
33 +            )));
34 +        };
35 +        let mut command = Command::new(program);
36 +        command.args(&argv[1..]);
37 +        Ok(command)
38 +    } else {
39 +        let executable = std::env::current_exe()
40 +            .map_err(|error| ExecutionError::InternalError(error.to_string()))?;
41 +        let mut command = Command::new(executable);
42 +        command.arg("worker");
43 +        Ok(command)
44 +    }
45 +}
46 +
47 +#[cfg(test)]
48 +mod tests {
49 +    use super::*;
50 +
51 +    #[test]
52 +    fn custom_worker_argv_is_literal_and_validated() -> ExecutionResult<()> {
53 +        let command = worker_command(Some(r#"["launcher", "$(not-a-shell)", "two word
54 s"]"#));
55 +        assert_eq!(command.as_std().get_program(), "launcher");
56 +        assert_eq!(
57 +            command.as_std().get_args().collect:::<Vec<_>>(),
58 +            ["$(not-a-shell)", "two words"]
59 +        );
60 +        for invalid in [
61 +            "[", "{", "]", r#"[""#, r#"["bad\u0000arg]"#,
62 +        ] {
63 +            assert!(worker_command(Some(invalid)).is_err(), "{invalid}");
64 +        }
65 +        Ok(())
66 +    }
67 +
68 +    #[tokio::test]
69 +    async fn missing_worker_launcher_is_a_launch_error() -> ExecutionResult<()> {
70 +        let mut command = worker_command(Some(r#"/nonexistent-sail-worker-launcher"
71 "#));
72 +        assert!(command.spawn().is_err());
73 +        Ok(())
74 +    }
75 +}

```

28. crates/sail-function/src/scalar/array/spark_array.rs

```
1 diff --git a/crates/sail-function/src/scalar/array/spark_array.rs b/crates/sail-function/src/scalar/array/spark_array.rs
2 index 6f7d06d735bd82cbbae33c79cddeb34d2882d03e..7b48d34316e471fa868a0b6c224201576b77b8
3 --- a/crates/sail-function/src/scalar/array/spark_array.rs
4 +++ b/crates/sail-function/src/scalar/array/spark_array.rs
5 @@ -9,12 +9,13 @@ use datafusion::arrow::array::{
6     use datafusion::arrow::buffer::OffsetBuffer;
7     use datafusion::arrow::datatypes::{DataType, Field, FieldRef};
8     use datafusion_common::utils::SingleRowListArrayBuilder;
9 -use datafusion_common::{Result, plan_datafusion_err, plan_err};
10 +use datafusion_common::{Result, ScalarValue, plan_datafusion_err, plan_err};
11     use datafusion_expr::type_coercion::binary::comparison_coercion;
12     use datafusion_expr::{
13         ColumnarValue, ReturnFieldArgs, ScalarFunctionArgs, ScalarUDFImpl, Signature, Type
14         eSignature,
15         Volatility,
16     };
17 +use sail_common_datafusion::geometry::common_geometry_metadata;
18     use crate::functions_nested_utils::make_scalar_function;
19
20 @@ -76,9 +77,29 @@ impl ScalarUDFImpl for SparkArray {
21         .cloned()
22         .collect::<Vec<_>>();
23         let contains_null = args.arg_fields.iter().any(|f| f.is_nullable());
24 +        // Coercion can already have changed a NULL literal's storage type to
25 +        // Binary. It remains neutral when choosing the list's geometry type.
26 +        let alternatives = args
27 +            .arg_fields
28 +            .iter()
29 +            .enumerate()
30 +            .map(|(index, field)| {
31 +                if matches!(args.scalar_arguments.get(index), Some(Some(value)) if value.is_null())
32 +                {
33 +                    Arc::new(Field::new("null", DataType::Null, true))
34 +                } else {
35 +                    Arc::clone(field)
36 +                }
37 +            })
38 +            .collect::<Vec<_>>();
39 +        let metadata = common_geometry_metadata(&alternatives).unwrap_or_default();
40         let return_type = match self.return_type(&data_types)? {
41             DataType::List(field) => DataType::List(Arc::new(
42                 field.as_ref().clone().with_nullable(contains_null),
43             +         field
44             +         .as_ref()
45             +         .clone()
46             +         .with_nullable(contains_null)
47             +         .with_metadata(metadata),
48             )),
49             data_type => data_type,
50         };
51 @@ -93,10 +114,36 @@ impl ScalarUDFImpl for SparkArray {
52         DataType::List(field) | DataType::LargeList(field) => field.is_nullable()
53         ,
54         _ => true,
55     };
56 +    let return_type = return_field.data_type().clone();
57 +    let func = make_scalar_function(move |arrays| {
58 +        make_array_inner_with_nullable(arrays, value_nullable)
59 +    });
60 -    func(args.as_slice())
61 +    let result = func(args.as_slice())?;
62 +    if !matches!(&return_type, DataType::List(field) if !field.metadata().is_empty
63 +    y()) {
64 +        return Ok(result);
65 +    }
66 +    // Restore the descriptor after scalar conversion: DataFusion 55's
67 +    // SingleRowListArrayBuilder::with_field copies name/nullability but
68 +    // omits metadata. Both result variants must retain the planned item
69 +    // field, including a scalar list later broadcast to multiple rows.
70 +    let restore = |array: &dyn Array| {
71         array
```

```

70 +         .to_data()
71 +         .into_builder()
72 +         .data_type(return_type.clone())
73 +         .build()
74 +     };
75 +     match result {
76 +         ColumnarValue::Array(array) => {
77 +             Ok(ColumnarValue::Array(make_array(restore(array.as_ref())?)))
78 +         }
79 +         ColumnarValue::Scalar(ScalarValue::List(array)) => {
80 +             Ok(ColumnarValue::Scalar(ScalarValue::List(Arc::new(
81 +                 GenericListArray::<i32>::from(restore(array.as_ref())?),
82 +             )))
83 +         }
84 +         _ => plan_err!("geometry array result must be a list"),
85 +     }
86 + }
87
88     fn aliases(&self) -> &[String] {
89 @@ -297,3 +344,76 @@ fn array_array<0: OffsetSizeTrait>(
90         None,
91     )?))
92 }
93 +
94 +#[cfg(test)]
95 +mod tests {
96 +    use datafusion::arrow::array::BinaryArray;
97 +
98 +    use super::*;
99 +
100 +    #[test]
101 +    fn geometry_array_retains_item_metadata_in_the_actual_arrow_array() -> Result<()>
102 +    {
103 +        let geometry = Arc::new(
104 +            Field::new("g", DataType::Binary, true).with_metadata(
105 +                [
106 +                    ("ARROW:extension:name".into(), "geoarrow.wkb".into()),
107 +                    ("ARROW:extension:metadata".into(), "{}".into()),
108 +                ]
109 +                .into(),
110 +        );
111 +        let null = ScalarValue::Binary(None);
112 +        let fields = vec![
113 +            Arc::new(Field::new("null", DataType::Binary, true)),
114 +            geometry.clone(),
115 +        ];
116 +        let function = SparkArray::new();
117 +        let returned = function.return_field_from_args(ReturnFieldArgs {
118 +            arg_fields: &fields,
119 +            scalar_arguments: &[Some(&null), None],
120 +        })?;
121 +        let DataType::List(item) = returned.data_type() else {
122 +            return plan_err!("expected list result");
123 +        };
124 +        assert_eq!(item.metadata(), geometry.metadata());
125 +        for value in [
126 +            ColumnarValue::Scalar(ScalarValue::Binary(Some(vec![1, 2, 3]))),
127 +            ColumnarValue::Array(Arc::new(BinaryArray::from(vec![Some(&[1, 2, 3][..])
128 +                ; 3]))),
129 +            ColumnarValue::Array(Arc::new(BinaryArray::from(Vec::<Option<&[u8]>>::new(
130 +                )))),
131 +        ] {
132 +            let is_scalar = matches!(value, ColumnarValue::Scalar(_));
133 +            let number_rows = match &value {
134 +                ColumnarValue::Scalar(_) => 3,
135 +                ColumnarValue::Array(array) => array.len(),
136 +            };
137 +            let result = function.invoke_with_args(ScalarFunctionArgs {
138 +                args: vec![ColumnarValue::Scalar(null.clone()), value],
139 +                arg_fields: fields.clone(),
140 +                number_rows,
141 +                return_field: returned.clone(),
142 +                config_options: Arc::new(Default::default()),
143 +            })?;
144 +            assert_eq!(matches!(result, ColumnarValue::Scalar(_)), is_scalar);
145 +            let lengths = if is_scalar {
146 +                vec![0, 1, 3]

```

```

145 + } else {
146 +     vec![number_rows]
147 + };
148 + for length in lengths {
149 +     let array = result.to_array(length)?;
150 +     assert_eq!(array.data_type(), returned.data_type());
151 +     assert_eq!(array.len(), length);
152 + }
153 + }
154 + let plain = Arc::new(Field::new("raw", DataType::Binary, true));
155 + let mixed = function.return_field_from_args(ReturnFieldArgs {
156 +     arg_fields: &[geometry, plain],
157 +     scalar_arguments: &[None, None],
158 + })?;
159 + let DataType::List(item) = mixed.data_type() else {
160 +     return plan_err!("expected list result");
161 + };
162 + assert!(item.metadata().is_empty());
163 + Ok(())
164 + }
165 +}

```

29. crates/sail-native-resource-ffi/Cargo.toml

```

1 diff --git a/crates/sail-native-resource-ffi/Cargo.toml b/crates/sail-native-resource-
  ffi/Cargo.toml
2 new file mode 100644
3 index 00000000000000000000000000000000..b09f272ab275bd1cbd8b00a90303a4ad9517a3
  9d
4 --- /dev/null
5 +++ b/crates/sail-native-resource-ffi/Cargo.toml
6 @@ -0,0 +1,9 @@
7 +[package]
8 +name = "sail-native-resource-ffi"
9 +version = "0.1.0"
10 +edition = "2024"
11 +license = "Apache-2.0"
12 +publish = false
13 +
14 +[lints]
15 +workspace = true

```

30. [crates/sail-native-resource-ffi/src/lib.rs](https://crates.io/crates/sail-native-resource-ffi)

```

1 diff --git a/crates/sail-native-resource-ffi/src/lib.rs b/crates/sail-native-resource-
  ffi/src/lib.rs
2 new file mode 100644
3 index 00000000000000000000000000000000..d15f735d1506f1e8775682829157f781b46266
  db
4 --- /dev/null
5 +++ b/crates/sail-native-resource-ffi/src/lib.rs
6 @@ -0,0 +1,160 @@
7 +/// The small C ABI shared by independently compiled native extensions and Sail.
8 +/// An opaque, prepaid memory lease crosses the boundary; Rust ownership and
9 +/// allocator layouts stay entirely inside the library that issued the lease.
10 +use std::ffi::{CStr, c_void};
11 +use std::fmt::{Debug, Formatter};
12 +use std::ptr::NonNull;
13 +use std::sync::Arc;
14 +
15 +pub const MEMORY_LEASE_CAPSULE: &CStr = c"sail_native_memory_lease_v1";
16 +
17 +#[repr(C)]
18 +struct Header {
19 +    version: u32,
20 +    size: u32,
21 +}
22 +
23 +/// A non-spillable host admission retained until the final clone is released.
24 +///
25 +/// Callbacks must be thread-safe and must never unwind. Every instance owns one
26 +/// reference. A consumer may inspect the header before touching versioned fields.
27 +#[repr(C)]

```

```

28 +pub struct MemoryLease {
29 +    header: Header,
30 +    bytes: u64,
31 +    opaque: *const c_void,
32 +    retain: unsafe extern "C" fn(*const c_void),
33 +    release: unsafe extern "C" fn(*const c_void),
34 +}
35 +
36 +// SAFETY: constructors require a Send + Sync owner; imports promise the same
37 +// thread-safe callback contract. The opaque object is never accessed here.
38 +unsafe impl Send for MemoryLease {}
39 +unsafe impl Sync for MemoryLease {}
40 +
41 +impl MemoryLease {
42 +    /// Export ownership through callbacks compiled in the issuing library.
43 +    pub fn new<T>: Send + Sync + 'static>(owner: Arc<T>, bytes: u64) -> Self {
44 +        unsafe extern "C" fn retain<T>(opaque: *const c_void) {
45 +            // SAFETY: only new() constructs this token, from Arc<T>::into_raw.
46 +            unsafe { Arc::<T>::increment_strong_count(opaque.cast::<T>()) };
47 +        }
48 +        unsafe extern "C" fn release<T>(opaque: *const c_void) {
49 +            // SAFETY: each token owns one reference created by new()/retain().
50 +            unsafe { drop(Arc::<T>::from_raw(opaque.cast::<T>())) };
51 +        }
52 +        Self {
53 +            header: Header {
54 +                version: 1,
55 +                size: std::mem::size_of::<Self>() as u32,
56 +            },
57 +            bytes,
58 +            opaque: Arc::into_raw(owner).cast::<c_void>(),
59 +            retain: retain::<T>,
60 +            release: release::<T>,
61 +        }
62 +    }
63 +
64 +    /// Validate the ABI and quota, then take an independently releasable reference.
65 +    ///
66 +    /// # Safety
67 +    /// `pointer` must address a readable, aligned header. A matching header must
68 +    /// address a live MemoryLease with valid thread-safe, non-unwinding callbacks.
69 +    /// Its issuing library must remain loaded until every imported clone is gone.
70 +    pub unsafe fn import(
71 +        pointer: NonNull<c_void>,
72 +        expected_bytes: u64,
73 +    ) -> Result<Self, &'static str> {
74 +        // SAFETY: the caller guarantees at least the fixed header is readable.
75 +        let header = unsafe { pointer.cast::<Header>().as_ref() };
76 +        if header.version != 1 || header.size as usize != std::mem::size_of::<Self>()
77 +        {
78 +            return Err("native memory lease ABI mismatch");
79 +        }
80 +        // SAFETY: the validated header and caller's capsule contract cover Self.
81 +        let lease = unsafe { pointer.cast::<Self>().as_ref() };
82 +        if lease.bytes != expected_bytes || lease.opaque.is_null() {
83 +            return Err("native memory lease quota mismatch");
84 +        }
85 +        Ok(lease.clone())
86 +    }
87 +
88 +    pub fn bytes(&self) -> u64 {
89 +        self.bytes
90 +    }
91 +}
92 +
93 +impl Clone for MemoryLease {
94 +    fn clone(&self) -> Self {
95 +        // SAFETY: self owns a live reference and callbacks obey the ABI contract.
96 +        unsafe { (self.retain)(self.opaque) };
97 +        Self {
98 +            header: Header {
99 +                version: self.header.version,
100 +                size: self.header.size,
101 +            },
102 +            bytes: self.bytes,
103 +            opaque: self.opaque,
104 +            retain: self.retain,
105 +            release: self.release,

```

```

105 +     }
106 + }
107 +}
108 +
109 +impl Drop for MemoryLease {
110 +    fn drop(&mut self) {
111 +        // SAFETY: this instance owns exactly one reference, relinquished once.
112 +        unsafe { (self.release)(self.opaque) };
113 +    }
114 +}
115 +
116 +impl Debug for MemoryLease {
117 +    fn fmt(&self, f: &mut Formatter<'_>) -> std::fmt::Result {
118 +        f.debug_struct("MemoryLease")
119 +            .field("version", &self.header.version)
120 +            .field("bytes", &self.bytes)
121 +            .finish_non_exhaustive()
122 +    }
123 +}
124 +
125 +#[cfg(test)]
126 +mod tests {
127 +    use std::sync::atomic::{AtomicUsize, Ordering};
128 +
129 +    use super::*;
130 +
131 +    struct Owner(Arc<AtomicUsize>);
132 +    impl Drop for Owner {
133 +        fn drop(&mut self) {
134 +            self.0.fetch_add(1, Ordering::SeqCst);
135 +        }
136 +    }
137 +
138 +    #[test]
139 +    fn imports_validate_before_clone_and_release_only_the_final_owner() -> Result<(),
140 +    &'static str>
141 +    {
142 +        let drops = Arc::new(AtomicUsize::new(0));
143 +        let exported = MemoryLease::new(Arc::new(Owner(drops.clone())), 64);
144 +        let pointer = NonNull::from(&exported).cast::<c_void>();
145 +        // SAFETY: the exported lease remains alive during both imports.
146 +        assert!(unsafe { MemoryLease::import(pointer, 65) }.is_err());
147 +        let imported = unsafe { MemoryLease::import(pointer, 64) }?;
148 +        drop(exported);
149 +        assert_eq!(drops.load(Ordering::SeqCst), 0);
150 +        let output_owner = imported.clone();
151 +        drop(imported);
152 +        assert_eq!(drops.load(Ordering::SeqCst), 0);
153 +        std::thread::spawn(move || drop(output_owner))
154 +            .join()
155 +            .map_err(|_| "release thread failed")?;
156 +        assert_eq!(drops.load(Ordering::SeqCst), 1);
157 +        let wrong = Header {
158 +            version: 2,
159 +            size: 8,
160 +        };
161 +        // SAFETY: a rejected version only reads the fixed header.
162 +        assert!(
163 +            unsafe { MemoryLease::import(NonNull::from(&wrong).cast::<c_void>(), 64)
164 +            }.is_err()
165 +        );
166 +        Ok(())
167 +    }
168 +}

```

31. crates/sail-plan/src/function/mod.rs

```

1 diff --git a/crates/sail-plan/src/function/mod.rs b/crates/sail-plan/src/function/mod.
2 index ba4406d5ae15923ac5d0970023d776e6386b0346..1e2726f9ba054bf4ca36588bcbe255871ad5a7
3 --- a/crates/sail-plan/src/function/mod.rs
4 +++ b/crates/sail-plan/src/function/mod.rs
5 @@ -54,6 +54,11 @@ pub fn is_built_in_generator_function(name: &str) -> bool {
6     BUILT_IN_GENERATOR_FUNCTIONS.contains_key(name)

```

```

7   }
8
9   +/// Includes every built-in function kind for extension collision validation.
10  +pub fn is_built_in_function_name(name: &str) -> bool {
11  +    list_built_in_function_names().contains(&name)
12  +}
13  +
14  fn list_built_in_function_names() -> Vec<&'static str> {
15      let mut names = BUILT_IN_SCALAR_FUNCTIONS
16          .keys()

```

32. crates/sail-plan/src/resolver/expression/geometry.rs

```

1  diff --git a/crates/sail-plan/src/resolver/expression/geometry.rs b/crates/sail-plan/s
rc/resolver/expression/geometry.rs
2  new file mode 100644
3  index 0000000000000000000000000000000000000000..ef86e807096673038bab8f13e57e242819d71a
8d
4  --- /dev/null
5  +++ b/crates/sail-plan/src/resolver/expression/geometry.rs
6  @@ -0,0 +1,155 @@
7  +/// Keep logical geometry types through value-preserving SQL expressions.
8  +use std::sync::Arc;
9  +
10 +use arrow::datatypes::{DataType, Field};
11 +use datafusion_common::tree_node::{Transformed, TreeNode};
12 +use datafusion_common::{DFSchema, Result};
13 +use datafusion_expr::expr::Cast;
14 +use datafusion_expr::{Expr, ExprSchemable};
15 +use sail_common_datafusion::geometry::common_geometry_metadata;
16 +
17 +/// DataFusion 55's CASE, coalesce and array_element infer only storage types.
18 +/// An explicit-field identity cast preserves the selected logical type through
19 +/// both logical simplification (coalesce becomes CASE) and physical planning.
20 +pub(super) fn preserve_geometry(expr: Expr, schema: &DFSchema) -> Result<Expr> {
21 +    // Expressions already repaired while resolving an argument need no second
22 +    // traversal. User type-only casts still strip geometry metadata normally.
23 +    if matches!(&expr, Expr::Cast(cast) if !cast.field.metadata().is_empty()) {
24 +        return Ok(expr);
25 +    }
26 +    let mut expr = expr
27 +        .map_children(|child| preserve_geometry(child, schema).map(Transformed::yes))
28 +        .data;
29 +    let alternatives: Vec<&Expr> = match &expr {
30 +        Expr::Case(case) => case
31 +            .when_then_expr
32 +            .iter()
33 +            .map(|(_, value)| value.as_ref())
34 +            .chain(case.else_expr.iter().map(|value| value.as_ref()))
35 +            .collect(),
36 +        Expr::ScalarFunction(function) if function.func.name() == "coalesce" => {
37 +            function.args.iter().collect()
38 +        }
39 +        Expr::ScalarFunction(function) if function.func.name() == "array_element" =>
40 +        {
41 +            let Some(input) = function.args.first() else {
42 +                return Ok(expr);
43 +            };
44 +            let Ok(data_type) = input.get_type(schema) else {
45 +                return Ok(expr); // An enclosing higher-order lambda may be unresolve
46 +            };
47 +            let field = match data_type {
48 +                DataType::List(field)
49 +                | DataType::LargeList(field)
50 +                | DataType::FixedSizeList(field, _)
51 +                | DataType::ListView(field)
52 +                | DataType::LargeListView(field) => field,
53 +                _ => return Ok(expr),
54 +            };
55 +            if let Some(metadata) = common_geometry_metadata(std::slice::from_ref(&fi
56 +                eld)) {
57 +                let field = Arc::new(
58 +                    field

```

```

57 +             .as_ref()
58 +             .clone()
59 +             .with_nullable(true)
60 +             .with_metadata(metadata),
61 +         );
62 +         return Ok(Expr::Cast(Cast::new_from_field(Box::new(expr), field)));
63 +     }
64 +     return Ok(expr);
65 + }
66 + _ => return Ok(expr),
67 + };
68 + let fields = alternatives
69 +     .iter()
70 +     .map(|value| {
71 +         // A typed NULL cannot change the geometry/CRS selected by other arms.
72 +         if is_null_literal(value) {
73 +             Ok(Arc::new(Field::new("null", DataType::Null, true)))
74 +         } else {
75 +             value.to_field(schema).map(|(_, field)| field)
76 +         }
77 +     })
78 +     .collect::<Result<Vec<_>>>();
79 + let Ok(fields) = fields else {
80 +     return Ok(expr); // Defer free lambda variables to their enclosing resolver.
81 + };
82 + let Some(metadata) = common_geometry_metadata(&fields) else {
83 +     return Ok(expr);
84 + };
85 + // DataFusion 55's coalesce coercion accepts equal Binary storage types,
86 + // but excludes Binary when resolving the union of Binary and Null. Type
87 + // only untyped NULL operands after establishing a common geometry type;
88 + // this neither relabels ordinary binary values nor changes null semantics.
89 + if let Expr::ScalarFunction(function) = &mut expr
90 +     && function.func.name() == "coalesce"
91 +     && let Some(field) = fields.iter().find(|field| !field.data_type().is_null())
92 +     {
93 +         for argument in &mut function.args {
94 +             if argument.get_type(schema)?.is_null() {
95 +                 *argument = Expr::Cast(Cast::new(
96 +                     Box::new(argument.clone()),
97 +                     field.data_type().clone(),
98 +                 ));
99 +             }
100 +         }
101 +     }
102 + let field = Arc::new(
103 +     Field::new("", expr.get_type(schema)?, expr.nullable(schema)?.with_metadata(
104 +         metadata),
105 + );
106 + Ok(Expr::Cast(Cast::new_from_field(Box::new(expr), field)))
107 + +}
108 +fn is_null_literal(expr: &Expr) -> bool {
109 +    match expr {
110 +        Expr::Literal(value, _) => value.is_null(),
111 +        Expr::Cast(cast) => is_null_literal(&cast.expr),
112 +        Expr::TryCast(cast) => is_null_literal(&cast.expr),
113 +        Expr::Alias(alias) => is_null_literal(&alias.expr),
114 +        // ANSI extraction inserts a typed raise_error arm. It produces no
115 +        // value and therefore cannot change the type of a successful result.
116 +        Expr::ScalarFunction(function) => function
117 +            .func
118 +            .inner()
119 +            .is::<sail_function::scalar::misc::raise_error::RaiseError>(
120 +            ),
121 +        _ => false,
122 +    }
123 +}
124 +
125 +#[cfg(test)]
126 +mod tests {
127 +    use arrow::datatypes::Schema;
128 +    use datafusion_expr::{col, lit, when};
129 +
130 +    use super::*;
131 +
132 +    #[test]
133 +    fn selectors_preserve_geometry_but_do_not_type_plain_binary() -> Result<()> {

```

```

134 +         let field = Field::new("g", DataType::Binary, true).with_metadata(
135 +             [
136 +                 ("ARROW:extension:name".into(), "geoarrow.wkb".into()),
137 +                 ("ARROW:extension:metadata".into(), "{}".into()),
138 +             ]
139 +             .into(),
140 +         );
141 +         let schema = DFSchema::try_from(Schema::new(vec![
142 +             field.clone(),
143 +             Field::new("raw", DataType::Binary, true),
144 +         ]))?;
145 +         let case = when(col("g").is_not_null(), col("g")).end()?;
146 +         let fixed = preserve_geometry(case, &schema)?;
147 +         assert_eq!(fixed.to_field(&schema)?.1.metadata(), field.metadata());
148 +         for alternatives in [
149 +             vec![col("g"), lit(datafusion_common::ScalarValue::Null)],
150 +             vec![lit(datafusion_common::ScalarValue::Null), col("g")],
151 +         ] {
152 +             let coalesce = datafusion::functions::expr_fn::coalesce(alternatives);
153 +             let fixed = preserve_geometry(coalesce, &schema)?;
154 +             assert_eq!(fixed.to_field(&schema)?.1.metadata(), field.metadata());
155 +         }
156 +         let mixed = when(col("g").is_not_null(), col("g")).otherwise(col("raw"))?;
157 +         let fixed = preserve_geometry(mixed, &schema)?;
158 +         assert!(fixed.to_field(&schema)?.1.metadata().is_empty());
159 +         Ok(())
160 +     }
161 + }

```

33. crates/sail-plan/src/resolver/expression/mod.rs

```

1 diff --git a/crates/sail-plan/src/resolver/expression/mod.rs b/crates/sail-plan/src/re
2 solver/expression/mod.rs
3 index ebc311789d228ed576facafaaa10d1b012d8bda1..b7af74d60c9c7905745052f8f48acd33bbba49
4 46 100644
5 --- a/crates/sail-plan/src/resolver/expression/mod.rs
6 +++ b/crates/sail-plan/src/resolver/expression/mod.rs
7 @@ -13,6 +13,7 @@ use crate::resolver::state::PlanResolverState;
8 mod attribute;
9 mod cast;
10 mod function;
11 +mod geometry;
12 mod grouping;
13 mod lambda;
14 mod literal;
15 @@ -106,7 +107,7 @@ impl PlanResolver<'_> {
16     ) -> PlanResult<NamedExpr> {
17         use spec::Expr;
18
19         match expr {
20             let mut result = match expr {
21                 Expr::Literal(literal) => self.resolve_expression_literal(literal, state)
22             ,
23                 Expr::UnresolvedAttribute {
24                     name,
25                 } => {
26 @@ -340,7 +341,9 @@ impl PlanResolver<'_> {
27                 Expr::NamedArgument { .. } => Err(PlanError::invalid(
28                     "named argument expression can only be used in UDF arguments",
29                 )),
30             }
31         }?;
32         result.expr = geometry::preserve_geometry(result.expr, schema)?;
33         Ok(result)
34     }
35
36     pub(super) async fn resolve_named_expressions(

```

34. crates/sail-plan/src/resolver/query/extension.rs

```

1 diff --git a/crates/sail-plan/src/resolver/query/extension.rs b/crates/sail-plan/src/r
2 esolver/query/extension.rs
3 new file mode 100644

```



```

3 --- a/crates/sail-plan/src/resolver/query/mod.rs
4 +++ b/crates/sail-plan/src/resolver/query/mod.rs
5 @@ -17,6 +17,7 @@ mod alias;
6     mod column_op;
7     mod cte;
8     mod dedup;
9 +mod extension;
10    mod filter;
11    mod join;
12    mod lateral;
13 @@ -69,6 +70,15 @@ impl PlanResolver<'_> {
14
15        let plan_id = plan.plan_id;
16        let plan = match plan.node {
17 +        QueryNode::Extension {
18 +            payload_type_url,
19 +            payload,
20 +            inputs,
21 +            is_envelope,
22 +        } => {
23 +            self.resolve_query_extension(payload_type_url, payload, inputs, is_envelope, state)
24 +                .await?
25 +        }
26        QueryNode::Read {
27            read_type,
28            is_streaming: _,

```

36. crates/sail-session/Cargo.toml

```

1 diff --git a/crates/sail-session/Cargo.toml b/crates/sail-session/Cargo.toml
2 index c959dda31102b60919cddee8ac9b7b58021e8d66..581934556b4aa70bfa0a3e121d8b1cc50214a2
3 79 100644
4 --- a/crates/sail-session/Cargo.toml
5 +++ b/crates/sail-session/Cargo.toml
6 @@ -41,6 +41,11 @@ datafusion = { workspace = true }
7     datafusion-common = { workspace = true }
8     datafusion-datasource = { workspace = true }
9     datafusion-expr = { workspace = true }
10 +datafusion-ffi = { workspace = true }
11 +arrow-schema = { workspace = true }
12 +pyo3 = { workspace = true }
13 +serde = { workspace = true }
14 +serde_json = { workspace = true }
15     datafusion-physical-expr = { workspace = true }
16     object_store = { workspace = true }
17     secrecy = { workspace = true }
18 @@ -50,6 +55,11 @@ chrono = { workspace = true }
19     indexmap = { workspace = true }
20     futures = { workspace = true }
21     uuid = { workspace = true }
22 +prost = { workspace = true }
23 +url = { workspace = true }
24 +
25 +[build-dependencies]
26 +prost-build = { workspace = true }
27
28 [dev-dependencies]
29 tempfile = { workspace = true }

```

37. crates/sail-session/build.rs

```

1 diff --git a/crates/sail-session/build.rs b/crates/sail-session/build.rs
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..8d7a2311dbfd0143509a6bb8c0e0b6f80ef3e9
4 9f
5 --- /dev/null
6 +++ b/crates/sail-session/build.rs
7 @@ -0,0 +1,6 @@
8 +fn main() -> Result<(), Box<dyn std::error::Error>> {
9 +    let proto = "proto";
10 +    println!("cargo:rerun-if-changed={proto}/gf/utils/v1/utils.proto");

```

```
10 +     prost_build::compile_protos(&[format!("{proto}/gf/utls/v1/utls.proto")], &[prot
11 +     ok(()))
12 + }
```

38. [crates/sail-session/proto/gf/utls/v1/utls.proto](#)

```

1 diff --git a/crates/sail-session/proto/gf/utls/v1/utls.proto b/crates/sail-session/p
  roto/gf/utls/v1/utls.proto
2 new file mode 100644
3 index 00000000000000000000000000000000..d3829bc7aa524f1639b1f5f0e8d5c1b8fd15a2
  73
4 --- /dev/null
5 +++ b/crates/sail-session/proto/gf/utls/v1/utls.proto
6 @@ -0,0 +1,39 @@
7 +syntax = "proto3";
8 +package gf.utls.v1;
9 +
10 +// Zero-input Relation.extension, collected eagerly. This is an experimental
11 +// contract; protocol_version in every receipt is 1. No effects during analysis.
12 +message Request {
13 +  oneof verb {
14 +    Ping ping = 1;
15 +    Exists exists = 2;
16 +    Ls ls = 3;
17 +    Rm rm = 4;
18 +    Mkdir mkdir = 5;
19 +  }
20 +}
21 +message Ping { string client_version = 1; }
22 +message Exists { string path = 1; string token = 2; }
23 +message Ls { string path = 1; uint32 limit = 2; string token = 3; }
24 +message Rm { string path = 1; string token = 2; }
25 +// Empty root uses the configured root. A retry with the same UUID returns the
26 +// same run within this server session; an already released run cannot reopen.
27 +message Mkdir { string root = 1; string request_id = 2; }
28 +
29 +// Response is Arrow, not a protobuf Receipt. All verbs return the same schema:
30 +// kind: utf8 non-null, path: utf8 nullable, size: int64 nullable,
31 +// value: bool nullable, count: int64 nullable, capabilities: utf8 nullable,
32 +// token: utf8 nullable, truncated: bool non-null, engine: utf8 non-null,
33 +// protocol_version: int32 non-null, lease_seconds: int64 non-null.
34 +// Ping: kind=pong, path=root, capabilities=JSON string array.
35 +// Mkdir: kind=mkdir, path=request URI, token=opaque capability.
36 +// Exists: kind=exists, path=request path, value=prefix exists.
37 +// Rm: kind=rm, path=request path, count=objects removed; repeated remove is safe.
38 +// Ls: kind=entry rows (path,size), followed by exactly one kind=ls row with count
39 +// and truncated. limit is 1..1000, default 100; internal ownership markers hidden.
40 +// lease_seconds=0: ownership lasts for this server session. Explicit Rm(run)
41 +// releases it; session expiration/close requests job shutdown, then attempts
42 +// best-effort cleanup with bounded storage-error retries. Detached writers are
43 +// not fully joined; interrupted writes may require operator cleanup of late files.
44 +// A URI under the configured root alone is NOT authority: all FS verbs require
45 +// the matching run token in the same session. Root deletion is always rejected.
```

39. crates/sail-session/src/extensions/driver.rs

```

1 diff --git a/crates/sail-session/src/extensions/driver.rs b/crates/sail-session/src/ex
2 tensions/driver.rs
3 new file mode 100644
4 index 0000000000000000000000000000000000000000..5fb28b6769b689e1d9e3562494040bab48fcd8
5 af
6 --- /dev/null
7 +++ b/crates/sail-session/src/extensions/driver.rs
8 @@ -0,0 +1,240 @@
9 +/// Keep a frozen native region on the driver while exposing its host inputs to
10 +/// Sail's stage planner. Task decoding reuses the original read snapshot/write
11 +/// attempt, and task execution substitutes prepared shuffle inputs across FFI.
12 +use std::fmt::{Debug, Formatter};
13 +use std::sync::{Arc, Mutex};
14 +
15 +use arrow_schema::SchemaRef;
16 +use async_trait::async_trait;

```

```

15 +use datafusion::catalog::{Session, TableProvider};
16 +use datafusion::execution::{SendableRecordBatchStream, TaskContext};
17 +use datafusion::physical_plan::{DisplayAs, DisplayFormatType, ExecutionPlan, PlanProperties};
18 +use datafusion_common::{Result, plan_err};
19 +use datafusion_expr::{Expr, TableType};
20 +use sail_common_datafusion::connect_extension::HostInputExec;
21 +use sail_common_datafusion::driver_extension::{
22 +     BoundDriverPlan, DriverDescriptor, DriverExtensionBinding, DriverExtensionExec,
23 +     DriverExtensionRegistry,
24 +};
25 +
26 +#[derive(Debug)]
27 +pub(super) struct InputPlaceholder {
28 +     pub name: String,
29 +     pub properties: Arc<PlanProperties>,
30 +}
31 +
32 +impl DisplayAs for InputPlaceholder {
33 +     fn fmt_as(&self, _: DisplayFormatType, f: &mut Formatter<'_>) -> std::fmt::Result {
34 +         write!(f, "{}", self.name)
35 +     }
36 +}
37 +
38 +impl ExecutionPlan for InputPlaceholder {
39 +     fn apply_expressions(
40 +         &self,
41 +         _f: &mut dyn FnMut(
42 +             &Arc<dyn datafusion::physical_expr::PhysicalExpr>,
43 +         ) -> Result<datafusion_common::tree_node::TreeNodeRecursion>,
44 +     ) -> Result<datafusion_common::tree_node::TreeNodeRecursion> {
45 +         Ok(datafusion_common::tree_node::TreeNodeRecursion::Continue)
46 +     }
47 +     fn name(&self) -> &str {
48 +         &self.name
49 +     }
50 +     fn properties(&self) -> &Arc<PlanProperties> {
51 +         &self.properties
52 +     }
53 +     fn children(&self) -> Vec<&Arc<dyn ExecutionPlan>> {
54 +         vec![]
55 +     }
56 +     fn with_new_children(
57 +         self: Arc<Self>,
58 +         children: Vec<Arc<dyn ExecutionPlan>>,
59 +     ) -> Result<Arc<dyn ExecutionPlan>> {
60 +         if !children.is_empty() {
61 +             return plan_err!("native input placeholder has no children");
62 +         }
63 +         Ok(self)
64 +     }
65 +     fn execute(&self, _: usize, _: Arc<TaskContext>) -> Result<SendableRecordBatchStream> {
66 +         plan_err!("native input placeholder must be bound before execution")
67 +     }
68 +}
69 +
70 +#[derive(Debug)]
71 +pub(super) struct DriverTableProvider {
72 +     pub inner: Arc<dyn TableProvider>,
73 +     pub inputs: Vec<Arc<dyn ExecutionPlan>>,
74 +     pub names: Vec<String>,
75 +     pub owner: String,
76 +     pub registry: Arc<DriverExtensionRegistry>,
77 +}
78 +
79 +#[async_trait]
80 +impl TableProvider for DriverTableProvider {
81 +     fn schema(&self) -> SchemaRef {
82 +         self.inner.schema()
83 +     }
84 +     fn table_type(&self) -> TableType {
85 +         self.inner.table_type()
86 +     }
87 +     async fn scan(
88 +         &self,
89 +         session: &dyn Session,

```

```

90 +     projection: Option<&Vec<usize>>,
91 +     filters: &[Expr],
92 +     limit: Option<usize>,
93 + ) -> Result<Arc<dyn ExecutionPlan>> {
94 +     let native = self.inner.scan(session, projection, filters, limit).await?;
95 +     if native.properties().partitioning.partition_count() != 1 {
96 +         return plan_err!("driver-only native relation must produce exactly one pa
rtition");
97 +     }
98 +     let bound = Arc::new(BoundDriverPlan {
99 +         descriptor: DriverDescriptor {
100 +             owner: self.owner.clone(),
101 +             plan_id: uuid::Uuid::new_v4().to_string(),
102 +         },
103 +         properties: native.properties().clone(),
104 +         input_schemas: self.inputs.iter().map(|input| input.schema()).collect(),
105 +         binding: Arc::new(NativeBinding {
106 +             resources: Mutex::new(Some(NativeResources {
107 +                 native,
108 +                 _provider: self.inner.clone(),
109 +             })),
110 +             names: self.names.clone(),
111 +         }),
112 +     });
113 +     self.registry.register(&bound)?;
114 +     Ok(Arc::new(DriverExtensionExec::new(
115 +         bound,
116 +         self.inputs.clone(),
117 +     )))
118 + }
119 +}
120 +
121 +#[derive(Debug)]
122 +struct NativeBinding {
123 +     resources: Mutex<Option<NativeResources>>,
124 +     names: Vec<String>,
125 +}
126 +
127 +#[derive(Debug)]
128 +struct NativeResources {
129 +     native: Arc<dyn ExecutionPlan>,
130 +     _provider: Arc<dyn TableProvider>,
131 +}
132 +
133 +impl DriverExtensionBinding for NativeBinding {
134 +     fn close(&self) {
135 +         if let Ok(mut resources) = self.resources.lock() {
136 +             resources.take();
137 +         }
138 +     }
139 +     fn materialize(
140 +         &self,
141 +         inputs: &[Arc<dyn ExecutionPlan>],
142 +         context: Arc<TaskContext>,
143 +     ) -> Result<Arc<dyn ExecutionPlan>> {
144 +         let native = self
145 +             .resources
146 +             .lock()
147 +             .map_err(|_| {
148 +                 datafusion_common::plan_datafusion_err!("native plan lifetime lock po
isoned")
149 +             })?
150 +             .as_ref()
151 +             .ok_or_else(|| {
152 +                 datafusion_common::plan_datafusion_err!("native driver query has alre
ady finished")
153 +             })?
154 +             .native
155 +             .clone();
156 +         let runtime = tokio::runtime::Handle::try_current().map_err(|e| {
157 +             datafusion_common::plan_datafusion_err!("driver native runtime missing: {
e}")
158 +         })?;
159 +         let replacements: Vec<Arc<dyn ExecutionPlan>> = inputs
160 +             .iter()
161 +             .map(|input| {
162 +                 Arc::new(HostInputExec::new(
163 +                     input.clone(),

```

```

164 +         context.clone(),
165 +         runtime.clone(),
166 +     )) as Arc<dyn ExecutionPlan>
167 + }
168 + .collect();
169 + fn substitute(
170 +     plan: Arc<dyn ExecutionPlan>,
171 +     names: &[String],
172 +     replacements: &[Arc<dyn ExecutionPlan>],
173 +     seen: &mut [bool],
174 + ) -> Result<Arc<dyn ExecutionPlan>> {
175 +     if let Some(index) = names.iter().position(|name| name == plan.name()) {
176 +         if plan.schema() != replacements[index].schema() {
177 +             return plan_err!("native input placeholder schema changed");
178 +         }
179 +         seen[index] = true;
180 +         return Ok(replacements[index].clone());
181 +     }
182 +     let children = plan.children();
183 +     if children.is_empty() {
184 +         return Ok(plan);
185 +     }
186 +     let replaced = children
187 +         .iter()
188 +         .map(|child| substitute((*child).clone(), names, replacements, seen))
189 +         .collect::<Result<Vec<_>>>()?>;
190 +     if children
191 +         .iter()
192 +         .zip(&replaced)
193 +         .all(|(before, after)| Arc::ptr_eq(before, after))
194 +     {
195 +         return Ok(plan);
196 +     }
197 +     plan.replace_children(
198 +         replaced,
199 +         datafusion::physical_plan::ReplaceChildrenOptions::new(
200 +             datafusion::physical_plan::execution_plan::ChildrenPropertiesMode
201 +             ::Recompute,
202 +         ),
203 +     )
204 +     let mut seen = vec![false; self.names.len()];
205 +     let plan = substitute(native, &self.names, &replacements, &mut seen)?;
206 +     if seen.iter().any(|value| !value) {
207 +         return plan_err!("native relation lost a declared host input");
208 +     }
209 +     Ok(plan)
210 + }
211 +}
212 +
213 +#[cfg(test)]
214 +mod tests {
215 +    use datafusion::datasource::empty::EmptyTable;
216 +    use datafusion::physical_plan::empty::EmptyExec;
217 +
218 +    use super::*;
219 +
220 +    #[tokio::test]
221 +    async fn driver_binding_releases_archived_snapshot_but_keeps_inflight_plan_alive(
222 +    ) -> Result<()> {
223 +        {
224 +            let schema = Arc::new(arrow_schema::Schema::empty());
225 +            let native: Arc<dyn ExecutionPlan> = Arc::new(EmptyExec::new(schema.clone()))
226 +            ;
227 +            let weak = Arc::downgrade(&native);
228 +            let binding = NativeBinding {
229 +                resources: Mutex::new(Some(NativeResources {
230 +                    native,
231 +                    _provider: Arc::new(EmptyTable::new(schema)),
232 +                })),
233 +                names: vec![],
234 +            };
235 +            let inflight = binding.materialize(&[], Arc::new(TaskContext::default()))?;
236 +            binding.close();
237 +            assert!(weak.upgrade().is_some());
238 +            assert!(
239 +                binding
240 +                    .materialize(&[], Arc::new(TaskContext::default()))

```

```
239 +         .is_err()
240 +     );
241 +     drop(inflight);
242 +     assert!(weak.upgrade().is_none());
243 +     binding.close();
244 +     Ok(())
245 + }
246 +}
```

40. [crates/sail-session/src/extensions/graph_utils/cleanup_tests.rs](#)

```

1 diff --git a/crates/sail-session/src/extensions/graph_utils/cleanup_tests.rs b/crates/
2 sail-session/src/extensions/graph_utils/cleanup_tests.rs
3 new file mode 100644
4 index 00000000000000000000000000000000..f9c0afdf7e5917c63b0595cfa2577a60f3f646
5 c0
6 --- /dev/null
7 +++ b/crates/sail-session/src/extensions/graph_utils/cleanup_tests.rs
8 @@ -0,0 +1,144 @@
9 +use std::sync::Arc;
10 +use std::sync::atomic::{AtomicUsize, Ordering};
11 +
12 +use async_trait::async_trait;
13 +use datafusion::execution::runtime_env::RuntimeEnv;
14 +use datafusion_common::{Result, plan_datafusion_err};
15 +use futures::TryStreamExt;
16 +use futures::stream::BoxStream;
17 +use object_store::memory::InMemory;
18 +use object_store::path::Path;
19 +use object_store::{
20 +    CopyOptions, GetOptions, GetResult, ListResult, MultipartUpload, ObjectMeta, Obje
21 ctStore,
22 +    PutMultipartOptions, PutOptions, PutPayload, PutResult,
23 };
24 +
25 +use super::proto::request::Verb;
26 +use super::proto::{Mkdir, Request};
27 +use super::storage::GraphRuns;
28 +
29 +#[derive(Debug)]
30 +struct FailLists {
31 +    inner: InMemory,
32 +    failures_remaining: AtomicUsize,
33 +    list_calls: AtomicUsize,
34 +}
35 +
36 +impl std::fmt::Display for FailLists {
37 +    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
38 +        f.write_str("FailLists")
39 +    }
40 +}
41 +
42 +#[async_trait]
43 +impl ObjectStore for FailLists {
44 +    async fn put_opts(
45 +        &self,
46 +        path: &Path,
47 +        payload: PutPayload,
48 +        opts: PutOptions,
49 +    ) -> object_store::Result<PutResult> {
50 +        self.inner.put_opts(path, payload, opts).await
51 +    }
52 +
53 +    async fn put_multipart_opts(
54 +        &self,
55 +        path: &Path,
56 +        opts: PutMultipartOptions,
57 +    ) -> object_store::Result<Box<dyn MultipartUpload>> {
58 +        self.inner.put_multipart_opts(path, opts).await
59 +    }
60 +
61 +    async fn get_opts(&self, path: &Path, opts: GetOptions) -> object_store::Result<G
62 etResult> {
63         self.inner.get_opts(path, opts).await
64     }
65 +
66 +    fn delete_stream(
67 +        &self,
68 +        locations: BoxStream<'static, object_store::Result<Path>>,
69 +    ) -> BoxStream<'static, object_store::Result<Path>> {

```

```

61 +         self.inner.delete_stream(locations)
62 +     }
63 +     fn list(&self, prefix: Option<&Path>) -> BoxStream<'static, object_store::Result<
ObjectMeta>> {
64 +         self.list_calls.fetch_add(1, Ordering::SeqCst);
65 +         if self
66 +             .failures_remaining
67 +             .fetch_update(Ordering::SeqCst, Ordering::SeqCst, |remaining| {
68 +                 remaining.checked_sub(1)
69 +             })
70 +             .is_ok()
71 +         {
72 +             Box::pin(futures::stream::once(async {
73 +                 Err(object_store::Error::Generic {
74 +                     store: "Faillists",
75 +                     source: std::io::Error::other("injected list failure").into(),
76 +                 })
77 +             })))
78 +         } else {
79 +             self.inner.list(prefix)
80 +         }
81 +     }
82 +     async fn list_with_delimiter(&self, prefix: Option<&Path>) -> object_store::Result<ListResult> {
83 +         self.inner.list_with_delimiter(prefix).await
84 +     }
85 +     async fn copy_opts(
86 +         &self,
87 +         from: &Path,
88 +         to: &Path,
89 +         opts: CopyOptions,
90 +     ) -> object_store::Result<()> {
91 +         self.inner.copy_opts(from, to, opts).await
92 +     }
93 + }
94 +
95 + async fn fixture() -> Result<(GraphRuns, Arc<Faillists>)> {
96 +     let runtime = RuntimeEnv::default();
97 +     let store = Arc::new(Faillists {
98 +         inner: InMemory::new(),
99 +         failures_remaining: AtomicUsize::new(0),
100 +         list_calls: AtomicUsize::new(0),
101 +     });
102 +     runtime.register_object_store(
103 +         &url::Url::parse("memory:///").map_err(|e| plan_datafusion_err!("{e}"))?,
104 +         store.clone(),
105 +     );
106 +     let runs = GraphRuns::new(&runtime, "memory:///graphs"?);
107 +     for _ in 0..3 {
108 +         runs.execute(Request {
109 +             verb: Some(Verb::Mkdir(Mkdir {
110 +                 root: String::new(),
111 +                 request_id: uuid::Uuid::new_v4().to_string(),
112 +             })),
113 +         })
114 +         .await?;
115 +     }
116 +     Ok((runs, store))
117 + }
118 +
119 + #[tokio::test]
120 + async fn production_cleanup_retries_a_transient_store_failure_and_attempts_every_run(
121 + ) -> Result<()> {
122 +     let (runs, store) = fixture().await?;
123 +     store.failures_remaining.store(1, Ordering::SeqCst);
124 +     // Exercise exactly the method called by session teardown: no manual retry.
125 +     runs.cleanup().await?;
126 +     // First pass attempts all three runs despite its first failure. A second
127 +     // pass revisits every run, including those already cleaned successfully.
128 +     assert_eq!(store.list_calls.load(Ordering::SeqCst), 6);
129 +     assert!(store.inner.list(None).try_next().await?.is_none());
130 +     Ok(())
131 + }
132 +
133 + #[tokio::test]
134 + async fn production_cleanup_stops_after_bounded_attempts_and_preserves_failure() -> Result<()> {

```

```

135 + let (runs, store) = fixture().await?;
136 + store.failures_remaining.store(100, Ordering::SeqCst);
137 + let error = runs
138 +     .cleanup()
139 +     .await
140 +     .err()
141 +     .ok_or_else(|| plan_datafusion_err!("cleanup unexpectedly succeeded"))?;
142 + assert!(error.to_string().contains("exhausted 3 attempts"));
143 + assert!(error.to_string().contains("injected list failure"));
144 + assert_eq!(store.list_calls.load(Ordering::SeqCst), 9);
145 + assert_eq!(
146 +     store.inner.list(None).try_collect::
```

41. [crates/sail-session/src/extensions/graph_utils/functions.rs](#)

```

1 diff --git a/crates/sail-session/src/extensions/graph_utils/functions.rs b/crates/sail-
2 session/src/extensions/graph_utils/functions.rs
3 new file mode 100644
4 index 0000000000000000000000000000000000..9caa557a5a0b940cd01754c7b735cd09e7f1ad
5 5f
6 --- /dev/null
7 +++ b/crates/sail-session/src/extensions/graph_utils/functions.rs
8 @@ -0,0 +1,98 @@
9 +use std::sync::Arc;
10 +
11 +use datafusion::arrow::array::{Array, Int64Array};
12 +use datafusion::arrow::datatypes::DataType;
13 +use datafusion_common::{Result, ScalarValue, exec_datafusion_err};
14 +use datafusion_expr::{ColumnarValue, ScalarUDF, Volatility, create_udf};
15 +use sail_common_datafusion::native_scalar::{OwnedScalar, retain_scalar};
16 +
17 +// Carry-less multiplication modulo x^64 + x^4 + x^3 + x + 1. Signed SQL
18 +// BIGINTs represent all 64 bit patterns; wrapping is polynomial arithmetic.
19 +fn multiply(mut a: u64, mut x: u64) -> u64 {
20 +    let mut result = 0;
21 +    for _ in 0..64 {
22 +        result ^= a & (0_u64.wrapping_sub(x & 1));
23 +        let high = a >> 63;
24 +        a = (a << 1) ^ (0x1b & (0_u64.wrapping_sub(high)));
25 +        x >>= 1;
26 +    }
27 +    result
28 +}
29 +
30 +pub(super) fn register() -> Result<Vec<ScalarUDF>> {
31 +    let axpb = create_udf(
32 +        "gf_axpb",
33 +        vec![DataType::Int64; 3],
34 +        DataType::Int64,
35 +        Volatility::Immutable,
36 +        Arc::new(|args| {
37 +            let arrays = ColumnarValue::values_to_arrays(args)?;
38 +            let arrays = arrays
39 +                .iter()
40 +                .map(|array| {
41 +                    array.as_any().downcast_ref:::<Int64Array>().ok_or_else(|| {
42 +                        exec_datafusion_err!("gf_axpb requires three BIGINT arguments")
43 +                    })
44 +                })
45 +                .collect:::<Result<Vec<_>>>()?;
46 +            if arrays.len() != 3 {
47 +                return Err(exec_datafusion_err!(
48 +                    "gf_axpb requires three BIGINT arguments"
49 +                ));
50 +            }
51 +            let result = Int64Array::from_iter((0..arrays[0].len()).map(|i| {
52 +                if arrays.iter().any(|a| a.is_null(i)) {
53 +                    None
54 +                } else {
55 +                    Some(

```

```

54 +             (multiply(arrays[0].value(i) as u64, arrays[1].value(i) as u6
55 + 4)
56 +             ^ arrays[2].value(i) as u64) as i64,
57 +         )
58 +     });
59 +     Ok(ColumnarValue::Array(Arc::new(result)))
60 + },
61 + );
62 + let version = create_udf(
63 +     "gf_version",
64 +     vec![],
65 +     DataType::Utf8,
66 +     Volatility::Immutable,
67 +     Arc::new(|_| {
68 +         Ok(ColumnarValue::Scalar(ScalarValue::Utf8(Some(
69 +             "sail-gf-utils/1".into(),
70 +         ))))
71 +     }),
72 + );
73 + [axpb, version]
74 +     .into_iter()
75 +     .map(|udf| {
76 +         let scalar = ScalarUDF::new_from_impl(OwnedScalar {
77 +             name: udf.name().into(),
78 +             identity: "sail-gf-utils/1:gf64-polynomial-1b".into(),
79 +             udf,
80 +             owner: Arc::new(()),
81 +         });
82 +         retain_scalar(scalar.clone())?;
83 +         Ok(scalar)
84 +     })
85 +     .collect()
86 + }
87 +
88 + #[cfg(test)]
89 + mod tests {
90 +     use super::*;
91 +     #[test]
92 +     fn gf64_vectors_and_field_properties() {
93 +         assert_eq!(multiply(1 << 63, 2), 0x1b);
94 +         assert_eq!(multiply(u64::MAX, 2), 0xffff_ffff_ffff_ffe5);
95 +         assert_eq!(multiply(0, u64::MAX), 0);
96 +         assert_eq!(multiply(1, u64::MAX), u64::MAX);
97 +         for a in [0, 1, 2, 0x83, 1 << 63, u64::MAX] {
98 +             for x in [0, 1, 2, 0xaabb_ccdd_1122_3344, u64::MAX] {
99 +                 assert_eq!(multiply(a, x), multiply(x, a));
100 +                 assert_eq!(multiply(a, x ^ 0x42), multiply(a, x) ^ multiply(a, 0x42))
101 +             }
102 +         }
103 +     }
104 + }

```

42. crates/sail-session/src/extensions/graph_utils/local.rs

```

1 diff --git a/crates/sail-session/src/extensions/graph_utils/local.rs b/crates/sail-ses
2 sion/src/extensions/graph_utils/local.rs
3 new file mode 100644
4 index 0000000000000000000000000000000000000000..4ab437ad40bf08a97e263d6556d4fbfef37fde
5 10
6 --- /dev/null
7 +++ b/crates/sail-session/src/extensions/graph_utils/local.rs
8 @@ -0,0 +1,114 @@
9 +/// A no-follow check for the explicitly trusted, exclusively managed local root.
10 +///
11 +/// ObjectStore's local listing follows symbolic links. Checking an owned URI's
12 +/// lexical prefix therefore is not sufficient before listing/deleting its tree.
13 +/// This check rejects existing links; it is not a descriptor-relative sandbox
14 +/// against an OS user racing filesystem changes between the check and operation.
15 +use std::path::{Path, PathBuf};
16 +
17 +use datafusion_common::{Result, exec_datafusion_err, exec_err, plan_datafusion_err, p
18 lan_err};
19 +use url::Url;

```

```

17 +
18 +#[derive(Debug, Clone)]
19 +pub(super) struct LocalRoot(PathBuf);
20 +
21 +impl LocalRoot {
22 +    pub(super) fn canonicalize(url: &Url) -> Result<Url, Option<Self>> {
23 +        if url.scheme() != "file" {
24 +            return Ok((url.clone(), None));
25 +        }
26 +        let path = url
27 +            .to_file_path()
28 +            .map_err(|()| plan_datafusion_err!("graph utils requires a local file URI
29 +            "));
30 +        let root = path.canonicalize().map_err(|e| {
31 +            plan_datafusion_err!(
32 +                "precreate the trusted graph staging directory before starting Sail:
33 +                {e}"
34 +            )
35 +        });
36 +        if !root.is_dir() || root.parent().is_none() {
37 +            return plan_err!("graph staging root must be an existing non-root directo
38 +            ry");
39 +        }
40 +        let url = Url::from_directory_path(&root)
41 +            .map_err(|()| plan_datafusion_err!("invalid canonical graph staging root"
42 +            ));
43 +        if url.path().trim_matches('/').split('/').any(|part| {
44 +            !part
45 +                .bytes()
46 +                .all(|b| b.is_ascii_alphanumeric() || b"-_."==>.contains(&b))
47 +        }) {
48 +            return plan_err!(
49 +                "canonical graph staging root requires unambiguous ASCII path segment
50 +                S"
51 +            );
52 +        }
53 +        Ok((url, Some(Self(root))))
54 +    }
55 +
56 +    pub(super) async fn check(&self, relative: PathBuf) -> Result<()> {
57 +        let root = self.0.clone();
58 +        tokio::task::spawn_blocking(move || {
59 +            // Recheck all root components too: replacing the allocated run or
60 +            // an ancestor with a symlink must not redirect a later request.
61 +            let mut current = PathBuf::new();
62 +            for component in root.components() {
63 +                current.push(component);
64 +                reject_link(&current)?;
65 +            }
66 +            for component in relative.components() {
67 +                current.push(component);
68 +                match std::fs::symlink_metadata(&current) {
69 +                    Ok(metadata) => reject_type(&current, &metadata)?,
70 +                    Err(error) if error.kind() == std::io::ErrorKind::NotFound => ret
71 +                    urn Ok(()),
72 +                    Err(error) => return Err(error.into()),
73 +                }
74 +            }
75 +            let metadata = std::fs::symlink_metadata(&current)?;
76 +            reject_type(&current, &metadata)?;
77 +            if !metadata.is_dir() {
78 +                return Ok(());
79 +            }
80 +            // Keep one iterator per directory depth, rather than accumulating
81 +            // every object path in memory. read_dir/file_type do not follow links.
82 +            let mut directories = vec![std::fs::read_dir(&current)?];
83 +            while let Some(directory) = directories.last_mut() {
84 +                if let Some(entry) = directory.next() {
85 +                    let entry = entry?;
86 +                    let kind = entry.file_type()?;
87 +                    if kind.is_symlink() {
88 +                        return exec_err!(
89 +                            "symbolic links are not allowed in graph staging: {}",
90 +                            entry.path().display()
91 +                        );
92 +                    }
93 +                    if kind.is_dir() {

```

```

88 +             directories.push(std::fs::read_dir(entry.path())?);
89 +         } else if !kind.is_file() {
90 +             return exec_err!(
91 +                 "non-regular file in graph staging: {}",
92 +                 entry.path().display()
93 +             );
94 +         }
95 +     } else {
96 +         directories.pop();
97 +     }
98 + }
99 + Ok(())
100 + })
101 + .await
102 + .map_err(|e| exec_datafusion_err!("graph staging path check: {e}"))?
103 + }
104 +}
105 +
106 +fn reject_link(path: &Path) -> Result<> {
107 +    reject_type(path, &std::fs::symlink_metadata(path)?)
108 +}
109 +fn reject_type(path: &Path, metadata: &std::fs::Metadata) -> Result<> {
110 +    if metadata.file_type().is_symlink() {
111 +        return exec_err!(
112 +            "symbolic links are not allowed in graph staging: {}",
113 +            path.display()
114 +        );
115 +    }
116 +    if !metadata.is_dir() && !metadata.is_file() {
117 +        return exec_err!("non-regular file in graph staging: {}", path.display());
118 +    }
119 +    Ok(())
120 +}

```

43. crates/sail-session/src/extensions/graph_utils/local_tests.rs

```

1 diff --git a/crates/sail-session/src/extensions/graph_utils/local_tests.rs b/crates/sail-session/src/extensions/graph_utils/local_tests.rs
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..b0a1a362fd16344c2ef89f37ad78a4075ed109
4 --- /dev/null
5 +++ b/crates/sail-session/src/extensions/graph_utils/local_tests.rs
6 @@ -0,0 +1,138 @@
7 +use std::os::unix::fs::symlink;
8 +use std::path::PathBuf;
9 +
10 +use datafusion::execution::runtime_env::RuntimeEnv;
11 +use datafusion_common::{Result, plan_datafusion_err};
12 +
13 +use super::proto::request::Verb;
14 +use super::proto::{Exists, Ls, Mkdir, Request, Rm};
15 +use super::storage::GraphRuns;
16 +
17 +async fn allocate(runs: &GraphRuns) -> Result<(String, String, String, PathBuf)> {
18 +    let id = uuid::Uuid::new_v4().to_string();
19 +    let rows = runs.execute(mkdir(&id)).await?;
20 +    let uri = rows[0]
21 +        .path
22 +        .clone()
23 +        .ok_or_else(|| plan_datafusion_err!("missing path"))?;
24 +    let token = rows[0]
25 +        .token
26 +        .clone()
27 +        .ok_or_else(|| plan_datafusion_err!("missing token"))?;
28 +    let path = url::Url::parse(&uri)
29 +        .map_err(|e| plan_datafusion_err!("{e}"))?
30 +        .to_file_path()
31 +        .map_err(|_| plan_datafusion_err!("invalid local receipt URI"))?;
32 +    Ok((id, uri, token, path))
33 +}
34 +fn mkdir(id: &str) -> Request {
35 +    Request {
36 +        verb: Some(Verb::Mkdir(Mkdir {
37 +            root: String::new(),

```

```

38 +         request_id: id.into(),
39 +     })),
40 + }
41 +}
42 +fn remove(path: &str, token: &str) -> Request {
43 +    Request {
44 +        verb: Some(Verb::Rm(Rm {
45 +            path: path.into(),
46 +            token: token.into(),
47 +        })),
48 +    }
49 +}
50 +fn fixture(root: &std::path::Path) -> Result<GraphRuns> {
51 +    let uri = url::Url::from_directory_path(root)
52 +    .map_err(|| plan_datafusion_err!("invalid fixture directory"))?;
53 +    GraphRuns::new(&RuntimeEnv::default(), uri.as_str())
54 +}
55 +
56 +#[tokio::test]
57 +async fn local_link_cannot_list_or_delete_outside_run_and_other_cleanup_continues() -
58 +    > Result<()> {
59 +    let directory = tempfile::tempdir()?;
60 +    let root = directory.path().join("root");
61 +    let outside = directory.path().join("outside");
62 +    std::fs::create_dir_all(&root)?;
63 +    std::fs::create_dir_all(&outside)?;
64 +    let sentinel = outside.join("keep.parquet");
65 +    std::fs::write(&sentinel, b"outside sentinel");
66 +    let runs = fixture(&root)?;
67 +    let (id, uri, token, path) = allocate(&runs).await?;
68 +    let (_, _, sibling) = allocate(&runs).await?;
69 +    std::fs::write(sibling.join("part.parquet"), b"owned");
70 +    symlink(&outside, path.join("escape"))?;
71 +    for verb in [
72 +        Verb::Exists(Exists {
73 +            path: uri.clone(),
74 +            token: token.clone(),
75 +        }),
76 +        Verb::Ls(Ls {
77 +            path: uri.clone(),
78 +            token: token.clone(),
79 +            limit: 1,
80 +        }),
81 +        Verb::Rm(Rm {
82 +            path: uri.clone(),
83 +            token: token.clone(),
84 +        }),
85 +        Verb::Rm(Rm {
86 +            path: format!("{uri}/escape/keep.parquet"),
87 +            token: token.clone(),
88 +        }),
89 +    ] {
90 +        let error = runs
91 +            .execute(Request { verb: Some(verb) })
92 +            .await
93 +            .err()
94 +            .ok_or_else(|| plan_datafusion_err!("symlink request unexpectedly succeed
95 +            ed"))?;
96 +        assert!(error.to_string().contains("symbolic links"));
97 +        assert_eq!(std::fs::read(&sentinel)?, b"outside sentinel");
98 +    }
99 +    assert!(runs.execute(mkdir(&id)).await.is_err());
100 +    assert!(runs.cleanup().await.is_err());
101 +    assert_eq!(std::fs::read(&sentinel)?, b"outside sentinel");
102 +    assert!(!sibling.join("part.parquet").exists());
103 +    assert!(!sibling.join("_sail_graph_run").exists());
104 +    // A dangling link is equally invalid and must not be mistaken for a missing path
105 +    .
106 +    std::fs::remove_file(path.join("escape"))?;
107 +    symlink(outside.join("missing"), path.join("escape"))?;
108 +    assert!(runs.cleanup().await.is_err());
109 +    std::fs::remove_file(path.join("escape"))?;
110 +    runs.cleanup().await?;
111 +    Ok(())
112 +}
113 +
114 +#[tokio::test]
115 +async fn local_run_replaced_by_link_cannot_mutate_sibling_run() -> Result<()> {

```

```

113 + let root = tempfile::tempdir()?;
114 + let runs = fixture(root.path())?;
115 + let (id, uri, token, path) = allocate(&runs).await?;
116 + let (_, _, _, sibling) = allocate(&runs).await?;
117 + let sentinel = sibling.join("keep.parquet");
118 + std::fs::write(&sentinel, b"sibling sentinel");
119 + std::fs::remove_dir_all(&path)?;
120 + symlink(&sibling, &path)?;
121 + assert!(runs.execute(remove(&uri, &token)).await.is_err());
122 + assert!(runs.execute(mkdir(&id)).await.is_err());
123 + assert_eq!(std::fs::read(&sentinel)?, b"sibling sentinel");
124 + std::fs::remove_file(&path)?;
125 + runs.cleanup().await?;
126 + Ok(())
127 +}
128 +
129 +#[tokio::test]
130 +async fn local_root_alias_is_canonicalized_before_ownership_is_assigned() -> Result<
131 +> {
132 + let directory = tempfile::tempdir()?;
133 + let root = directory.path().join("actual");
134 + std::fs::create_dir(&root)?;
135 + let alias = directory.path().join("alias");
136 + symlink(&root, &alias)?;
137 + let runs = fixture(&alias)?;
138 + let (_, _, _, allocated) = allocate(&runs).await?;
139 + let canonical = root.canonicalize()?;
140 + assert_eq!(allocated.parent(), Some(canonical.as_path()));
141 + runs.cleanup().await?;
142 + let missing = directory.path().join("not-created");
143 + assert!(fixture(&missing).is_err());
144 + Ok(())
145 +}

```

44. crates/sail-session/src/extensions/graph_utils/mod.rs

```

1 diff --git a/crates/sail-session/src/extensions/graph_utils/mod.rs b/crates/sail-session/src/extensions/graph_utils/mod.rs
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..a5a3907ee2c8ca63d57837ab4e611fc8472afd
4 --- /dev/null
5 +++ b/crates/sail-session/src/extensions/graph_utils/mod.rs
6 @@ -0,0 +1,64 @@
7 +/// Opt-in, host-owned storage utilities for relational graph clients.
8 +///
9 +/// This is a compiled-in adapter: object-store credentials/runtime never cross
10 +/// the native wheel interface. The ordinary scalar codec handles its functions.
11 +#[cfg(test)]
12 +mod cleanup_tests;
13 +mod functions;
14 +mod local;
15 +#[cfg(all(test, unix))]
16 +mod local_tests;
17 +mod plan;
18 +mod storage;
19 +#[cfg(test)]
20 +mod tests;
21 +
22 +use std::sync::Arc;
23 +
24 +use datafusion::execution::runtime_env::RuntimeEnv;
25 +use datafusion::prelude::SessionConfig;
26 +use datafusion_common::{Result, plan_err};
27 +use datafusion_expr::ScalarUDF;
28 +use sail_common_datafusion::connect_extension::ConnectExtensionRegistry;
29 +use sail_common_datafusion::driver_extension::DriverExtensionRegistry;
30 +pub(crate) use storage::GraphRuns;
31 +
32 +pub(super) mod proto {
33 +    include!(concat!(env!("OUT_DIR"), "/gf_utils.v1.rs"));
34 +}
35 +
36 +pub(super) const TYPE_URL: &str = "type.googleapis.com/gf_utils.v1.Request";
37 +

```

```

38 +pub(super) fn register(
39 +     config: &mut SessionConfig,
40 +     runtime: &Arc<RuntimeEnv>,
41 +     registry: &mut ConnectExtensionRegistry,
42 +     driver: Option<Arc<DriverExtensionRegistry>>,
43 +) -> Result<Vec<ScalarUDF>> {
44 +     let root = match std::env::var("SAIL_GRAPH_UTILS_ROOT") {
45 +         Ok(root) if !root.is_empty() => root,
46 +         Ok(_) => return plan_err!("SAIL_GRAPH_UTILS_ROOT must be a nonempty absolute
URI"),
47 +         Err(std::env::VarError::NotPresent) => return Ok(vec![]),
48 +         Err(error) => return plan_err!("SAIL_GRAPH_UTILS_ROOT: {error}"),
49 +     };
50 +     let runs = Arc::new(GraphRuns::new(runtime, &root)?);
51 +     registry.register(
52 +         TYPE_URL.into(),
53 +         true,
54 +         0,
55 +         0,
56 +         Arc::new(plan::Handler {
57 +             runs: runs.clone(),
58 +             driver,
59 +         }),
60 +     );
61 +     config.set_extension(runs);
62 +     functions::register()
63 +}
64 +
65 +pub(super) fn register_worker_functions() -> Result<()> {
66 +     // Workers need no filesystem capability or configured driver root. The
67 +     // function identity is versioned with its bit-level implementation contract.
68 +     functions::register()?;
69 +     Ok(())
70 +}

```

45. crates/sail-session/src/extensions/graph_utils/plan.rs

```

1 diff --git a/crates/sail-session/src/extensions/graph_utils/plan.rs b/crates/sail-sess
ion/src/extensions/graph_utils/plan.rs
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..20d86842778e6d186aeaed872db781276d5310
b8
4 --- /dev/null
5 +++ b/crates/sail-session/src/extensions/graph_utils/plan.rs
6 @@ -0,0 +1,231 @@
7 +use std::sync::Arc;
8 +
9 +use async_trait::async_trait;
10 +use datafusion::arrow::array::{ArrayRef, BooleanArray, Int32Array, Int64Array, String
Array};
11 +use datafusion::arrow::datatypes::{DataType, Field, Schema, SchemaRef};
12 +use datafusion::arrow::record_batch::RecordBatch;
13 +use datafusion::catalog::{Session, TableProvider};
14 +use datafusion::execution::{SendableRecordBatchStream, TaskContext};
15 +use datafusion::physical_expr::{EquivalenceProperties, Partitioning};
16 +use datafusion::physical_plan::execution_plan::{Boundedness, EmissionType};
17 +use datafusion::physical_plan::stream::RecordBatchStreamAdapter;
18 +use datafusion::physical_plan::{DisplayAs, DisplayFormatType, ExecutionPlan, PlanProp
erties};
19 +use datafusion_common::{Result, plan_datafusion_err, plan_err};
20 +use datafusion_expr::{Expr, TableType};
21 +use prost::Message;
22 +use sail_common_datafusion::connect_extension::ConnectRelationHandler;
23 +use sail_common_datafusion::driver_extension::DriverExtensionRegistry;
24 +
25 +use super::super::driver::DriverTableProvider;
26 +use super::proto::Request;
27 +use super::storage::GraphRuns;
28 +
29 +#[derive(Default)]
30 +pub(super) struct Row {
31 +     pub kind: String,
32 +     pub path: Option<String>,
33 +     pub size: Option<i64>,
34 +     pub value: Option<bool>,

```

```

35 +     pub count: Option<i64>,
36 +     pub capabilities: Option<String>,
37 +     pub token: Option<String>,
38 +     pub truncated: bool,
39 + }
40 +impl Row {
41 +     pub fn new(kind: &str) -> Self {
42 +         Self {
43 +             kind: kind.into(),
44 +             ..Self::default()
45 +         }
46 +     }
47 + }
48 +
49 +pub(super) fn schema() -> SchemaRef {
50 +     Arc::new(Schema::new(vec![
51 +         Field::new("kind", DataType::Utf8, false),
52 +         Field::new("path", DataType::Utf8, true),
53 +         Field::new("size", DataType::Int64, true),
54 +         Field::new("value", DataType::Boolean, true),
55 +         Field::new("count", DataType::Int64, true),
56 +         Field::new("capabilities", DataType::Utf8, true),
57 +         Field::new("token", DataType::Utf8, true),
58 +         Field::new("truncated", DataType::Boolean, false),
59 +         Field::new("engine", DataType::Utf8, false),
60 +         Field::new("protocol_version", DataType::Int32, false),
61 +         Field::new("lease_seconds", DataType::Int64, false),
62 +     ]))
63 + }
64 +
65 +fn batch(rows: &[Row]) -> Result<RecordBatch> {
66 +     let arrays: Vec<ArrayRef> = vec![
67 +         Arc::new(StringArray::from_iter_values(
68 +             rows.iter().map(|r| r.kind.as_str()),
69 +         )),
70 +         Arc::new(StringArray::from_iter(
71 +             rows.iter().map(|r| r.path.as_deref()),
72 +         )),
73 +         Arc::new(Int64Array::from_iter(rows.iter().map(|r| r.size))),
74 +         Arc::new(BooleanArray::from_iter(rows.iter().map(|r| r.value))),
75 +         Arc::new(Int64Array::from_iter(rows.iter().map(|r| r.count))),
76 +         Arc::new(StringArray::from_iter(
77 +             rows.iter().map(|r| r.capabilities.as_deref()),
78 +         )),
79 +         Arc::new(StringArray::from_iter(
80 +             rows.iter().map(|r| r.token.as_deref()),
81 +         )),
82 +         Arc::new(BooleanArray::from_iter(
83 +             rows.iter().map(|r| Some(r.truncated)),
84 +         )),
85 +         Arc::new(StringArray::from_iter_values(rows.iter().map(|_| "sail"))),
86 +         Arc::new(Int32Array::from(vec![1; rows.len()])),
87 +         Arc::new(Int64Array::from(vec![0; rows.len()])),
88 +     ];
89 +     Ok(RecordBatch::try_new(schema(), arrays)?)
90 + }
91 +
92 +pub(super) struct Handler {
93 +     pub runs: Arc<GraphRuns>,
94 +     pub driver: Option<Arc<DriverExtensionRegistry>>,
95 + }
96 +impl ConnectRelationHandler for Handler {
97 +     fn plan(
98 +         &self,
99 +         payload: &[u8],
100 +         inputs: Vec<Arc<dyn ExecutionPlan>>,
101 +     ) -> Result<Arc<dyn TableProvider>> {
102 +         if payload.len() > 8192 || !inputs.is_empty() {
103 +             return plan_err!("graph utils requires zero inputs and at most 8192 request bytes");
104 +         }
105 +         let request = Request::decode(payload)
106 +             .map_err(|e| plan_datafusion_err!("graph utils protobuf: {e}"))?;
107 +         GraphRuns::validate(&request)?;
108 +         let provider: Arc<dyn TableProvider> = Arc::new(Provider {
109 +             runs: self.runs.clone(),
110 +             request,
111 +         });

```

```

112 +         match &self.driver {
113 +             Some(registry) => Ok(Arc::new(DriverTableProvider {
114 +                 inner: provider,
115 +                 inputs: vec![],
116 +                 names: vec![],
117 +                 owner: "sail-gf-utils/1".into(),
118 +                 registry: registry.clone(),
119 +             })),
120 +             None => Ok(provider),
121 +         }
122 +     }
123 +}
124 +
125 +struct Provider {
126 +    runs: Arc<GraphRuns>,
127 +    request: Request,
128 +}
129 +impl std::fmt::Debug for Provider {
130 +    fn fmt(&self, f: &mut std::fmt::Formatter<'>) -> std::fmt::Result {
131 +        f.debug_struct("GraphUtilsProvider").finish_non_exhaustive()
132 +    }
133 +}
134 +#[async_trait]
135 +impl TableProvider for Provider {
136 +    fn schema(&self) -> SchemaRef {
137 +        schema()
138 +    }
139 +    fn table_type(&self) -> TableType {
140 +        TableType::Temporary
141 +    }
142 +    async fn scan(
143 +        &self,
144 +        _session: &dyn Session,
145 +        projection: Option<&Vec<usize>>,
146 +        filters: &[Expr],
147 +        _limit: Option<usize>,
148 +    ) -> Result<Arc<dyn ExecutionPlan>> {
149 +        let schema = match projection {
150 +            Some(p) => Arc::new(schema().project(p)?),
151 +            None => schema(),
152 +        };
153 +        let properties = Arc::new(PlanProperties::new(
154 +            EquivalenceProperties::new(schema),
155 +            Partitioning::UnknownPartitioning(1),
156 +            EmissionType::Final,
157 +            Boundedness::Bounded,
158 +        ));
159 +        Ok(Arc::new(UtilsExec {
160 +            runs: self.runs.clone(),
161 +            request: self.request.clone(),
162 +            projection: projection.cloned(),
163 +            properties,
164 +        }))
165 +    }
166 +}
167 +
168 +struct UtilsExec {
169 +    runs: Arc<GraphRuns>,
170 +    request: Request,
171 +    projection: Option<Vec<usize>>,
172 +    properties: Arc<PlanProperties>,
173 +}
174 +impl std::fmt::Debug for UtilsExec {
175 +    fn fmt(&self, f: &mut std::fmt::Formatter<'>) -> std::fmt::Result {
176 +        self.fmt_as(DisplayFormatType::Default, f)
177 +    }
178 +}
179 +impl DisplayAs for UtilsExec {
180 +    fn fmt_as(&self, _: DisplayFormatType, f: &mut std::fmt::Formatter<'>) -> std::f
181 +        mt::Result {
182 +        // Never print token-bearing requests in explain output or logs.
183 +        write!(f, "GraphUtilsExec: host storage, placement=driver")
184 +    }
185 +}
186 +impl ExecutionPlan for UtilsExec {
187 +    fn apply_expressions(
188 +        &self,
189 +        _f: &mut dyn FnMut(

```

```

189 +         &Arc<dyn datafusion::physical_expr::PhysicalExpr>,
190 +     ) -> Result<datafusion_common::tree_node::TreeNodeRecursion>,
191 + ) -> Result<datafusion_common::tree_node::TreeNodeRecursion> {
192 +     Ok(datafusion_common::tree_node::TreeNodeRecursion::Continue)
193 + }
194 + fn name(&self) -> &'static str {
195 +     "GraphUtilsExec"
196 + }
197 + fn properties(&self) -> &Arc<PlanProperties> {
198 +     &self.properties
199 + }
200 + fn children(&self) -> Vec<&Arc<dyn ExecutionPlan>> {
201 +     vec![]
202 + }
203 + fn with_new_children(
204 +     self: Arc<Self>,
205 +     children: Vec<Arc<dyn ExecutionPlan>>,
206 + ) -> Result<Arc<dyn ExecutionPlan>> {
207 +     if !children.is_empty() {
208 +         return plan_err!("graph utils has no children");
209 +     }
210 +     Ok(self)
211 + }
212 + fn execute(
213 +     &self,
214 +     partition: usize,
215 +     _context: Arc<TaskContext>,
216 + ) -> Result<SendableRecordBatchStream> {
217 +     if partition != 0 {
218 +         return plan_err!("graph utils supports only partition zero");
219 +     }
220 +     let (runs, request, projection) = (
221 +         self.runs.clone(),
222 +         self.request.clone(),
223 +         self.projection.clone(),
224 +     );
225 +     let stream = futures::stream::once(async move {
226 +         let batch = batch(&runs.execute(request).await)?;
227 +         match projection {
228 +             Some(p) => Ok(batch.project(&p)?),
229 +             None => Ok(batch),
230 +         }
231 +     });
232 +     Ok(Box::pin(RecordBatchStreamAdapter::new(
233 +         self.schema(),
234 +         stream,
235 +     )))
236 + }
237 +}

```

46. crates/sail-session/src/extensions/graph_utils/storage.rs

```

1 diff --git a/crates/sail-session/src/extensions/graph_utils/storage.rs b/crates/sail-s
2 session/src/extensions/graph_utils/storage.rs
3 new file mode 100644
4 index 0000000000000000000000000000000000000000..cae3975061349b909d1f3fd8ce4f804ff77844
5 d1
6 --- /dev/null
7 +++ b/crates/sail-session/src/extensions/graph_utils/storage.rs
8 @@ -0,0 +1,356 @@
9 +use std::collections::HashMap;
10 +use std::sync::Arc;
11 +
12 +use datafusion::execution::runtime_env::RuntimeEnv;
13 +use datafusion_common::{DataFusionError, Result, exec_err, plan_datafusion_err, plan_
14 +err};
15 +use futures::TryStreamExt;
16 +use object_store::path::Path;
17 +use object_store::{ObjectStore, ObjectStoreExt, PutPayload};
18 +use sail_common_datafusion::extension::SessionExtension;
19 +use sail_object_store::resolve_object_store_path;
20 +use tokio::sync::Mutex;
21 +use url::Url;
22 +
23 +use super::local::LocalRoot;

```

```

21 +use super::plan::Row;
22 +use super::proto::Request;
23 +use super::proto::request::Verb;
24 +
25 +const MARKER: &str = "_sail_graph_run";
26 +const MAX_RUNS: usize = 1024;
27 +
28 +#[derive(Debug)]
29 +struct Run {
30 +    uri: String,
31 +    prefix: Path,
32 +    token: String,
33 +    released: bool,
34 +}
35 +#[derive(Debug, Default)]
36 +struct State {
37 +    runs: HashMap<String, Run>,
38 +    closed: bool,
39 +}
40 +
41 +pub(crate) struct GraphRuns {
42 +    root: String,
43 +    prefix: Path,
44 +    store: Arc<dyn ObjectStore>,
45 +    local: Option<LocalRoot>,
46 +    state: Mutex<State>,
47 +}
48 +
49 +impl std::fmt::Debug for GraphRuns {
50 +    fn fmt(&self, f: &mut std::fmt::Formatter<'>) -> std::fmt::Result {
51 +        f.debug_struct("GraphRuns")
52 +            .field("root", &self.root)
53 +            .finish_non_exhaustive()
54 +    }
55 +}
56 +
57 +impl SessionExtension for GraphRuns {
58 +    fn name() -> &'static str {
59 +        "GraphUtilsOwnedRuns"
60 +    }
61 +}
62 +
63 +impl GraphRuns {
64 +    pub(super) fn new(runtime: &RuntimeEnv, root: &str) -> Result<Self> {
65 +        // Reject ambiguous spellings before URL parsing normalizes traversal.
66 +        if root.len() > 4096
67 +            || root.contains(['%', '\\', '?', '#'])
68 +            || root.split('/').any(|s| s == "." || s == "..")
69 +        {
70 +            return plan_err!("graph utils root contains an ambiguous path");
71 +        }
72 +        let url = Url::parse(root).map_err(|e| plan_datafusion_err!("graph utils root
: {e}"))?;
73 +        if !url.username().is_empty()
74 +            || url.password().is_some()
75 +            || url.cannot_be_a_base()
76 +            || url.path().trim_matches('/').is_empty()
77 +        {
78 +            return plan_err!("graph utils root must be an absolute URI with a non-roo
t prefix");
79 +        }
80 +        if url.path().trim_matches('/').split('/').any(|s| {
81 +            s.is_empty()
82 +            || !s
83 +                .bytes()
84 +                .all(|b| b.is_ascii_alphanumeric() || b"-._" == ".contains(&b))
85 +        }) {
86 +            return plan_err!("graph utils root requires unambiguous ASCII path segmen
ts");
87 +        }
88 +        let (url, local) = LocalRoot::canonicalize(&url)?;
89 +        let root = url.to_string().trim_end_matches('/').to_owned();
90 +        let resolved = resolve_object_store_path(runtime, &root)?;
91 +        Ok(Self {
92 +            root,
93 +            prefix: resolved.prefix().clone(),
94 +            store: resolved.store().clone(),
95 +            local,

```

```

96 +         state: Mutex::new(State::default()),
97 +     })
98 + }
99 +
100 + pub(super) fn validate(request: &Request) -> Result<()> {
101 +     let Some(verb) = &request.verb else {
102 +         return plan_err!("graph utils request has no verb");
103 +     };
104 +     match verb {
105 +         Verb::Ping(p) if p.client_version.len() > 128 => {
106 +             return plan_err!("client version too long");
107 +         }
108 +         Verb::Mkdir(m) => {
109 +             uuid::Uuid::parse_str(&m.request_id)
110 +                 .map_err(|_| plan_datafusion_err!("Mkdir.request_id must be a UUID"))?;
111 +             if m.root.len() > 4096 {
112 +                 return plan_err!("root URI too long");
113 +             }
114 +         }
115 +         Verb::Ls(l) if l.limit > 1000 => return plan_err!("Ls.limit must be at most 1000"),
116 +         _ => {}
117 +     }
118 +     Ok(())
119 + }
120 +
121 + fn owned_path(&self, run: &Run, path: &str) -> Result<Path> {
122 +     if path.len() > 4096 || path.contains(['%', '\\', '?', '#']) {
123 +         return plan_err!("invalid owned run path");
124 +     }
125 +     let path = path.trim_end_matches('/');
126 +     if path == run.uri {
127 +         return Ok(run.prefix.clone());
128 +     }
129 +     let suffix = path
130 +         .strip_prefix(&format!("{}", run.uri))
131 +         .ok_or_else(|| plan_datafusion_err!("path is outside the token's owned run"))?;
132 +     if suffix.split('/').any(|s| {
133 +         s.is_empty()
134 +         || s == "."
135 +         || s == ".."
136 +         || !s
137 +         .bytes()
138 +         .all(|b| b.is_ascii_alphanumeric() || b"-._" ==> b.contains(&b))
139 +     }) {
140 +         return plan_err!("owned run suffix must contain unambiguous ASCII path segments");
141 +     }
142 +     Ok(suffix
143 +         .split('/')
144 +         .fold(run.prefix.clone(), |path, segment| path.join(segment)))
145 + }
146 +
147 + async fn remove(&self, prefix: &Path) -> Result<i64> {
148 +     self.check_local(prefix).await?;
149 +     let mut listing = self.store.list(Some(prefix));
150 +     let mut count = 0_i64;
151 +     while let Some(entry) = listing.try_next().await? {
152 +         match self.store.delete(&entry.location).await {
153 +             Ok(()) => count += 1,
154 +             Err(object_store::Error::NotFound { .. }) => {}
155 +             Err(error) => return Err(DataFusionError::ObjectStore(Box::new(error)))
156 +         }
157 +     }
158 +     Ok(count)
159 + }
160 +
161 + async fn check_local(&self, prefix: &Path) -> Result<()> {
162 +     if let Some(local) = &self.local {
163 +         let parts = prefix.prefix_match(&self.prefix).ok_or_else(|| {
164 +             plan_datafusion_err!("local graph path is outside the staging root")
165 +         })?;
166 +         let relative = parts
167 +             .map(|part| part.as_ref().to_string())
168 +             .collect::<std::path::PathBuf>();

```

```

169 +         local.check(relative).await?;
170 +     }
171 +     Ok(())
172 + }
173 +
174 + pub(super) async fn execute(&self, request: Request) -> Result<Vec<Row>> {
175 +     Self::validate(&request)?;
176 +     let Some(verb) = request.verb else {
177 +         return exec_err!("graph utils request has no verb");
178 +     };
179 +     let mut state = self.state.lock().await;
180 +     if state.closed {
181 +         return exec_err!("graph utils session has closed");
182 +     }
183 +     match verb {
184 +         Verb::Ping(_) => Ok(vec![Row {
185 +             path: Some(self.root.clone()),
186 +             capabilities: Some(r#"["fs","owned_runs_v1","axpb"]"#.into()),
187 +             ..Row::new("pong")
188 +         }]),
189 +         Verb::Mkdir(request) => {
190 +             if !request.root.is_empty() && request.root.trim_end_matches('/') !=
self.root {
191 +                 return exec_err!("Mkdir.root differs from the configured trusted
root");
192 +             }
193 +             let key = uuid::Uuid::parse_str(&request.request_id)
194 +                 .map_err(|e| plan_datafusion_err!("request UUID: {e}"))?
195 +                 .to_string();
196 +             if !state.runs.contains_key(&key) {
197 +                 if state.runs.len() >= MAX_RUNS {
198 +                     return exec_err!("graph utils session reached the 1024 run li
mit");
199 +                 }
200 +                 let id = uuid::Uuid::new_v4().to_string();
201 +                 let run = Run {
202 +                     uri: format!("{}/{id}", self.root),
203 +                     prefix: self.prefix.clone().join(id),
204 +                     token: uuid::Uuid::new_v4().to_string(),
205 +                     released: false,
206 +                 };
207 +                 // Record ownership before awaiting storage: cancelled allocation
208 +                 // attempts remain discoverable by retry and session cleanup.
209 +                 state.runs.insert(key.clone(), run);
210 +             }
211 +             let run = state
212 +                 .runs
213 +                 .get(&key)
214 +                 .ok_or_else(|| plan_datafusion_err!("run missing"))?;
215 +             if run.released {
216 +                 return exec_err!("Mkdir request refers to a released run");
217 +             }
218 +             self.check_local(&run.prefix).await?;
219 +             self.store
220 +                 .put(
221 +                     &run.prefix.clone().join(MARKER),
222 +                     PutPayload::from_static(b"gf-utils-v1"),
223 +                 )
224 +                 .await?;
225 +             Ok(vec![Row {
226 +                 path: Some(run.uri.clone()),
227 +                 token: Some(run.token.clone()),
228 +                 ..Row::new("mkdir")
229 +             }])
230 +         }
231 +         verb => {
232 +             let (path, token) = match &verb {
233 +                 Verb::Exists(v) => (&v.path, &v.token),
234 +                 Verb::Ls(v) => (&v.path, &v.token),
235 +                 Verb::Rm(v) => (&v.path, &v.token),
236 +                 _ => return exec_err!("invalid filesystem verb"),
237 +             };
238 +             let run = state
239 +                 .runs
240 +                 .values_mut()
241 +                 .find(|r| r.token == *token)
242 +                 .ok_or_else(|| plan_datafusion_err!("unknown run token in this se
ssion"))?;

```

```

243 +         let prefix = self.owned_path(run, path)?;
244 +         if run.released && !matches!(verb, Verb::Rm(_)) {
245 +             return exec_err!("graph run has been released");
246 +         }
247 +         match verb {
248 +             Verb::Exists(request) => {
249 +                 self.check_local(&prefix).await?;
250 +                 let exists = self.store.list(Some(&prefix)).try_next().await?
.is_some();
251 +                 Ok(vec![Row {
252 +                     path: Some(request.path),
253 +                     value: Some(exists),
254 +                     ..Row::new("exists")
255 +                 }])
256 +             }
257 +             Verb::Ls(request) => {
258 +                 self.check_local(&prefix).await?;
259 +                 let limit = if request.limit == 0 {
260 +                     100
261 +                 } else {
262 +                     request.limit
263 +                 } as usize;
264 +                 let mut listing = self.store.list(Some(&prefix));
265 +                 let mut rows = vec![];
266 +                 let mut truncated = false;
267 +                 while let Some(entry) = listing.try_next().await? {
268 +                     if entry.location == run.prefix.clone().join(MARKER) {
269 +                         continue;
270 +                     }
271 +                     if rows.len() == limit {
272 +                         truncated = true;
273 +                         break;
274 +                     }
275 +                     let suffix = entry
276 +                         .location
277 +                         .as_ref()
278 +                         .strip_prefix(run.prefix.as_ref())
279 +                         .ok_or_else(|| {
280 +                             plan_datafusion_err!("store returned object outsi
de owned run")
281 +                         })?;
282 +                     rows.push(Row {
283 +                         path: Some(format!("{}", suffix)),
284 +                         size: Some(
285 +                             i64::try_from(entry.size)
286 +                             .map_err(|e| plan_datafusion_err!("object siz
e: {e}"))?,
287 +                         ),
288 +                         ..Row::new("entry")
289 +                     });
290 +                 }
291 +                 rows.push(Row {
292 +                     path: Some(request.path),
293 +                     count: Some(rows.len() as i64),
294 +                     truncated,
295 +                     ..Row::new("ls")
296 +                 });
297 +                 Ok(rows)
298 +             }
299 +             Verb::Rm(request) => {
300 +                 let count = if run.released {
301 +                     0
302 +                 } else {
303 +                     self.remove(&prefix).await?
304 +                 };
305 +                 if prefix == run.prefix {
306 +                     run.released = true;
307 +                 }
308 +                 Ok(vec![Row {
309 +                     path: Some(request.path),
310 +                     count: Some(count),
311 +                     ..Row::new("rm")
312 +                 }])
313 +             }
314 +             _ => exec_err!("invalid filesystem verb"),
315 +         }
316 +     }
317 + }

```

```

318 +     }
319 +
320 +     /// Best-effort cleanup after session teardown has requested executor/job
321 +     /// shutdown. That shutdown is not a join barrier for every detached writer;
322 +     /// interrupted writes can still require operator cleanup of late objects.
323 +     /// Transient store failures get bounded retries, with every owned namespace
324 +     /// attempted on each pass. The caller logs any error after exhaustion.
325 +     pub(crate) async fn cleanup(&self) -> Result<()> {
326 +         const ATTEMPTS: usize = 3;
327 +         for attempt in 1..=ATTEMPTS {
328 +             match self.cleanup_once().await {
329 +                 Ok(()) => return Ok(()),
330 +                 Err(error) if attempt == ATTEMPTS => {
331 +                     return exec_err!("graph run cleanup exhausted {ATTEMPTS} attempts
: {error}");
332 +                 }
333 +                 Err(error) => {
334 +                     log::warn!("graph run cleanup attempt {attempt}/{ATTEMPTS} failed
: {error}");
335 +                     tokio::time::sleep(std::time::Duration::from_millis(200)).await;
336 +                 }
337 +             }
338 +         }
339 +         Ok(())
340 +     }
341 +
342 +     async fn cleanup_once(&self) -> Result<()> {
343 +         let mut state = self.state.lock().await;
344 +         state.closed = true;
345 +         let mut errors = vec![];
346 +         for run in state.runs.values_mut() {
347 +             match self.remove(&run.prefix).await {
348 +                 Ok(()) => run.released = true,
349 +                 Err(error) => errors.push(error.to_string()),
350 +             }
351 +         }
352 +         if errors.is_empty() {
353 +             Ok(())
354 +         } else {
355 +             exec_err!(
356 +                 "graph run cleanup failed for {} namespaces: {}",
357 +                 errors.len(),
358 +                 errors.join("; ")
359 +             )
360 +         }
361 +     }
362 + }

```

47. crates/sail-session/src/extensions/graph_utils/tests.rs

```

1 diff --git a/crates/sail-session/src/extensions/graph_utils/tests.rs b/crates/sail-ses
sion/src/extensions/graph_utils/tests.rs
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..f34200e977bc382b16f0b8be18bbf56e0e696a
48
4 --- /dev/null
5 +++ b/crates/sail-session/src/extensions/graph_utils/tests.rs
6 @@ -0,0 +1,267 @@
7 +#![expect(
8 +     clippy::expect_used,
9 +     reason = "test fixtures assert their typed receipt contract"
10 +)]
11 +
12 +use std::sync::Arc;
13 +
14 +use datafusion::execution::runtime_env::RuntimeEnv;
15 +use datafusion::prelude::SessionContext;
16 +use datafusion_common::Result;
17 +use futures::TryStreamExt;
18 +use object_store::memory::InMemory;
19 +use object_store::path::Path;
20 +use object_store::{ObjectStore, ObjectStoreExt, PutPayload};
21 +use prost::Message;
22 +use sail_common_datafusion::connect_extension::ConnectRelationHandler;
23 +

```

```

24 +use super::plan::Handler;
25 +use super::proto::request::Verb;
26 +use super::proto::{Exists, Ls, Mkdir, Request, Rm};
27 +use super::storage::GraphRuns;
28 +
29 +fn request(verb: Verb) -> Request {
30 +    Request { verb: Some(verb) }
31 +}
32 +fn mkdir(id: &str) -> Request {
33 +    request(Verb::Mkdir(Mkdir {
34 +        root: String::new(),
35 +        request_id: id.into(),
36 +    }))
37 +}
38 +fn fixture() -> Result<(Arc<GraphRuns>, Arc<InMemory>)> {
39 +    let runtime = RuntimeEnv::default();
40 +    let store = Arc::new(InMemory::new());
41 +    runtime.register_object_store(
42 +        &url::Url::parse("memory:///").expect("constant URL"),
43 +        store.clone(),
44 +    );
45 +    Ok((
46 +        Arc::new(GraphRuns::new(&runtime, "memory:///graphs"?),
47 +        store,
48 +    ))
49 +}
50 +async fn allocate(runs: &GraphRuns) -> Result<(String, String)> {
51 +    let rows = runs
52 +        .execute(mkdir(&uuid::Uuid::new_v4().to_string()))
53 +        .await?;
54 +    Ok((
55 +        rows[0].path.clone().expect("path"),
56 +        rows[0].token.clone().expect("token"),
57 +    ))
58 +}
59 +
60 +#[tokio::test]
61 +async fn planning_and_schema_analysis_have_no_storage_effects() -> Result<()> {
62 +    let (runs, store) = fixture()?;
63 +    let handler = Handler { runs, driver: None };
64 +    let provider = handler.plan(
65 +        &mkdir(&uuid::Uuid::new_v4().to_string()).encode_to_vec(),
66 +        vec![],
67 +    )?;
68 +    assert_eq!(provider.schema().field(0).name(), "kind");
69 +    let session = SessionContext::new();
70 +    let plan = provider.scan(&session.state(), None, &[], None).await?;
71 +    assert!(store.list(None).try_next().await?.is_none());
72 +    let batches = datafusion::physical_plan::collect(plan, session.task_ctx()).await?;
73 +    assert_eq!(batches[0].num_rows(), 1);
74 +    assert!(store.list(None).try_next().await?.is_some());
75 +    Ok(())
76 +}
77 +
78 +#[tokio::test]
79 +async fn allocation_retry_and_release_are_session_owned_and_idempotent() -> Result<()> {
80 +    let (runs, store) = fixture()?;
81 +    let id = uuid::Uuid::new_v4().to_string();
82 +    let first = runs.execute(mkdir(&id)).await?;
83 +    let second = runs.execute(mkdir(&id)).await?;
84 +    assert_eq!(first[0].path, second[0].path);
85 +    assert_eq!(first[0].token, second[0].token);
86 +    let path = first[0].path.clone().expect("path");
87 +    let token = first[0].token.clone().expect("token");
88 +    let exists = request(Verb::Exists(Exists {
89 +        path: path.clone(),
90 +        token: token.clone(),
91 +    }));
92 +    assert_eq!(runs.execute(exists.clone()).await?[0].value, Some(true));
93 +    let remove = request(Verb::Rm(Rm { path, token }));
94 +    assert_eq!(runs.execute(remove.clone()).await?[0].count, Some(1));
95 +    assert_eq!(runs.execute(remove).await?[0].count, Some(0));
96 +    assert!(runs.execute(mkdir(&id)).await.is_err());
97 +    assert!(runs.execute(exists).await.is_err());
98 +    assert!(store.list(None).try_next().await?.is_none());
99 +    Ok(())

```

```

100 +}
101 +
102 +#[tokio::test]
103 +async fn capabilities_reject_other_runs_roots_traversal_and_sessions() -> Result<()>
104 + {
105 +     let (runs, _) = fixture()?;
106 +     let (path, token) = allocate(&runs).await?;
107 +     let (other, _) = allocate(&runs).await?;
108 +     for invalid in [
109 +         "memory:///graphs".to_string(),
110 +         other,
111 +         format!("{path}extra"),
112 +         format!("{path}/../escape"),
113 +         format!("{path}/%2e%2e/escape"),
114 +         format!("{path}/a//b"),
115 +         format!("{path}/a\\b"),
116 +         format!("{path}/stage?query"),
117 +     ] {
118 +         assert!(
119 +             runs.execute(request(Verb::Rm(Rm {
120 +                 path: invalid,
121 +                 token: token.clone()
122 +             })))
123 +             .await
124 +             .is_err()
125 +         );
126 +     }
127 +     let (other_session, _) = fixture()?;
128 +     assert!(
129 +         other_session
130 +         .execute(request(Verb::Rm(Rm { path, token })))
131 +         .await
132 +         .is_err()
133 +     );
134 +     Ok(())
135 + }
136 +
137 +#[tokio::test]
138 +async fn bounded_listing_and_session_cleanup_preserve_foreign_objects() -> Result<()>
139 + {
140 +     let (runs, store) = fixture()?;
141 +     let (path, token) = allocate(&runs).await?;
142 +     let prefix = path.strip_prefix("memory:///").expect("memory URI");
143 +     for suffix in [
144 +         "stage-1/part-1.parquet",
145 +         "stage-1/part-2.parquet",
146 +         "stage-2/part-1.parquet",
147 +         "stage-10/part-1.parquet",
148 +     ] {
149 +         store
150 +         .put(
151 +             &Path::from(format!("{prefix}/{suffix}")),
152 +             PutPayload::from_static(b"test"),
153 +         )
154 +         .await?;
155 +     }
156 +     store
157 +     .put(
158 +         &Path::from("graphs/unowned/keep"),
159 +         PutPayload::from_static(b"keep"),
160 +     )
161 +     .await?;
162 +     let rows = runs
163 +     .execute(request(Verb::Ls(Ls {
164 +         path: path.clone(),
165 +         token: token.clone(),
166 +         limit: 1,
167 +     })))
168 +     .await?;
169 +     assert_eq!(rows.len(), 2);
170 +     assert_eq!(rows[0].kind, "entry");
171 +     assert_eq!(rows[1].kind, "ls");
172 +     assert_eq!(rows[1].count, Some(1));
173 +     assert!(rows[1].truncated);
174 +     let removed = runs
175 +     .execute(request(Verb::Rm(Rm {
176 +         path: format!("{path}/stage-1"),
177 +         token: token.clone(),

```

```

176 +         })))
177 +         .await?;
178 +         assert_eq!(removed[0].count, Some(2));
179 +         assert!(
180 +             store
181 +                 .head(&Path::from(format!("{prefix}/stage-10/part-1.parquet")))
182 +                 .await
183 +                 .is_ok()
184 +         );
185 +         assert_eq!(
186 +             runs.execute(request(Verb::Exists(Exists {
187 +                 path: path.clone(),
188 +                 token
189 +             })))
190 +                 .await?[0]
191 +                 .value,
192 +             Some(true)
193 +         );
194 +         runs.cleanup().await?;
195 +         let remaining = store.list(None).try_collect::<Vec<_>>().await?;
196 +         assert_eq!(remaining.len(), 1);
197 +         assert_eq!(remaining[0].location, Path::from("graphs/unowned/keep"));
198 +         assert!(allocate(&runs).await.is_err());
199 +         Ok(())
200 +     }
201 +
202 +     #[test]
203 +     fn request_limits_and_root_spelling_are_validated() -> Result<()> {
204 +         let (runs, _) = fixture()?;
205 +         let handler = Handler { runs, driver: None };
206 +         assert!(handler.plan(&[0; 8193], vec![]).is_err());
207 +         assert!(handler.plan(&[], vec![]).is_err());
208 +         assert!(GraphRuns::validate(&mkdir("not-a-uuid")).is_err());
209 +         assert!(
210 +             GraphRuns::validate(&request(Verb::Ls(Ls {
211 +                 path: String::new(),
212 +                 token: String::new(),
213 +                 limit: 1001
214 +             })))
215 +                 .is_err()
216 +         );
217 +         for invalid in [
218 +             "/tmp/root",
219 +             "file:///a",
220 +             "file:///a/..b",
221 +             "file:///a/%2e",
222 +             "file:///a?b",
223 +             "file:///a#b",
224 +         ] {
225 +             assert!(GraphRuns::new(&RuntimeEnv::default(), invalid).is_err());
226 +         }
227 +         Ok(())
228 +     }
229 +
230 +     #[tokio::test]
231 +     async fn graph_scalar_codec_and_signed_null_semantics() -> Result<()> {
232 +         use datafusion_common::ScalarValue;
233 +         use datafusion_expr::lit;
234 +
235 +         let session = SessionContext::new();
236 +         let mut functions = vec![];
237 +         for function in super::functions::register()? {
238 +             let mut bytes = vec![];
239 +             assert!(sail_common_datafusion::native_scalar::encode_scalar(
240 +                 &function, &mut bytes
241 +             ));
242 +             let decoded =
243 +                 sail_common_datafusion::native_scalar::decode_scalar(function.name(), &bytes);
244 +             session.register_udf((*decoded).clone());
245 +             functions.push(decoded);
246 +         }
247 +         let batches = session
248 +             .read_empty()?
249 +             .select(vec![
250 +                 functions[0]
251 +                     .call(vec![lit(i64::MIN), lit(2_i64), lit(0_i64)])
252 +                     .alias("a"),

```



```

49 +         self.api_version,
50 +         self.datafusion_version,
51 +         self.arrow_version
52 +     );
53 + }
54 + if !matches!(self.placement.as_str(), "driver" | "any") {
55 +     return plan_err!(
56 +         "extension {} has unsupported placement {}",
57 +         self.name,
58 +         self.placement
59 +     );
60 + }
61 + if let Some(bytes) = self.memory_bytes
62 +     && (bytes == 0 || self.placement != "driver")
63 + {
64 +     return plan_err!(
65 +         "extension {} memory_bytes must be positive and placement must be dri
66 + ver",
67 +         self.name
68 +     );
69 + }
70 + let mut urls = HashSet::new();
71 + for relation in &self.relation_types {
72 +     if relation.type_url.is_empty()
73 +         || relation.min_inputs > relation.max_inputs
74 +         || relation.max_inputs > 16
75 +         || !urls.insert(&relation.type_url)
76 +     {
77 +         return plan_err!(
78 +             "extension {} has invalid or duplicate relation type {}",
79 +             self.name,
80 +             relation.type_url
81 +         );
82 +     }
83 + }
84 + Ok(())
85 + }
86 + }
87 + #[cfg(test)]
88 + mod tests {
89 +     use super::*;
90 +
91 +     fn valid() -> Manifest {
92 +         Manifest {
93 +             name: "fixture".into(),
94 +             version: "1".into(),
95 +             api_version: 1,
96 +             datafusion_version: "55.1.0".into(),
97 +             arrow_version: "59.3.0".into(),
98 +             placement: "driver".into(),
99 +             memory_bytes: None,
100 +             relation_types: vec![],
101 +         }
102 +     }
103 +
104 +     #[test]
105 +     fn validates_before_native_layout_access() {
106 +         assert!(valid().validate().is_ok());
107 +         let mut manifest = valid();
108 +         manifest.datafusion_version = "54.1.0".into();
109 +         assert!(
110 +             matches!(manifest.validate(), Err(error) if error.to_string().contains("f
111 + ixture") && error.to_string().contains("55.1.0") && error.to_string().contains("54.1.0
112 + "))
113 +         );
114 +     }
115 +
116 +     #[test]
117 +     fn rejects_unknown_fields_and_bad_bounds() {
118 +         assert!(serde_json::from_str::(<Manifest>(r#"{"future_layout":2}"#)).is_err());
119 +         let mut manifest = valid();
120 +         manifest.relation_types.push(RelationType {
121 +             type_url: "x".into(),
122 +             accepts_bare: false,
123 +             min_inputs: 2,
124 +             max_inputs: 1,

```

```

124 +     assert!(manifest.validate().is_err());
125 + }
126 +
127 + #[test]
128 + fn native_session_quota_requires_a_positive_driver_only_cap() {
129 +     let mut manifest = valid();
130 +     manifest.memory_bytes = Some(64);
131 +     assert!(manifest.validate().is_ok());
132 +     manifest.memory_bytes = Some(0);
133 +     assert!(manifest.validate().is_err());
134 +     manifest.memory_bytes = Some(64);
135 +     manifest.placement = "any".into();
136 +     assert!(manifest.validate().is_err());
137 + }
138 +}

```

49. crates/sail-session/src/extensions/mod.rs

```

1 diff --git a/crates/sail-session/src/extensions/mod.rs b/crates/sail-session/src/exten
2 sions/mod.rs
3 new file mode 100644
4 index 0000000000000000000000000000000000000000..4bd79a811e2f0cdf1c49c89e2a2b28e54abc9c
5 33
6 --- /dev/null
7 +++ b/crates/sail-session/src/extensions/mod.rs
8 @@ -0,0 +1,420 @@
9 +/// Experimental, exact-build native packages for local Sail sessions.
10 +///
11 +/// Python metadata is the bootstrap protocol; native objects use DataFusion's
12 +/// named capsules. This is deliberately not a promise of a stable Sail C ABI.
13 +mod driver;
14 +pub(crate) mod graph_utils;
15 +mod manifest;
16 +mod plan;
17 +mod python_owner;
18 +#[cfg(test)]
19 +mod resource_tests;
20 +
21 +use std::collections::{HashMap, HashSet};
22 +use std::sync::{Arc, Mutex, OnceLock};
23 +
24 +use datafusion::catalog::TableProvider;
25 +use datafusion::execution::runtime_env::RuntimeEnv;
26 +use datafusion::physical_plan::ExecutionPlan;
27 +use datafusion::prelude::SessionConfig;
28 +use datafusion_common::{DataFusionError, Result, plan_err};
29 +use datafusion_expr::ScalarUDF;
30 +use datafusion_ffi::execution_plan::FFI_ExecutionPlan;
31 +use datafusion_ffi::table_provider::FFI_TableProvider;
32 +use datafusion_ffi::udf::FFI_ScalarUDF;
33 +use pyo3::prelude::*;
34 +use pyo3::types::{PyBytes, PyCapsule, PyCapsuleMethods, PyDict, PyList};
35 +use sail_catalog::manager::CatalogManager;
36 +use sail_common::config::ExecutionMode;
37 +use sail_common_datafusion::connect_extension::{
38 +    ConnectExtensionRegistry, ConnectRelationHandler, HostInputExec,
39 +};
40 +use sail_common_datafusion::driver_extension::DriverExtensionRegistry;
41 +use sail_common_datafusion::native_resource::{MEMORY_LEASE_CAPSULE, NativeResourceTracker};
42 +use sail_common_datafusion::native_scalar::{OwnedScalar, retain_scalar};
43 +use sail_plan::function::is_built_in_function_name;
44 +
45 +use self::driver::{DriverTableProvider, InputPlaceholder};
46 +use self::manifest::Manifest;
47 +use self::plan::NativeTableProvider;
48 +use self::python_owner::PythonOwner;
49 +
50 +fn py_error(error: impl std::fmt::Display) -> DataFusionError {
51 +    DataFusionError::Plan(format!("native extension: {error}"))
52 +}
53 +
54 +fn package_identity(
55 +    py: Python<'>,
56 +    entry: &Bound<'_, PyAny>,
57 +)

```

```

55 +     metadata: &Bound<'_, PyAny>,
56 +) -> Result<String> {
57 +     let source = std::ffi::CString::new(include_str!("package_identity.py")).map_err(
py_error)?;
58 +     pyo3::types::PyModule::from_code(
59 +         py,
60 +         &source,
61 +         c"sail_extension_identity.py",
62 +         c"sail_extension_identity",
63 +     )
64 +     .and_then(|module| module.call_method1("identity", (entry, metadata)))
65 +     .and_then(|value| value.extract())
66 +     .map_err(py_error)
67 +}
68 +
69 +/// Loaded native code is process-lifetime; session state is never stored here.
70 +/// This also protects release callbacks in Arrow arrays retained beyond a query.
71 +fn retain_package(py: Python<'_,>, identity: String, factory: &Bound<'_, PyAny>) -> Re
sult<()> {
72 +     static PACKAGES: OnceLock<Mutex<HashMap<String, Py<PyAny>>>> = OnceLock::new();
73 +     let mut packages = PACKAGES
74 +         .get_or_init(Mutex::default)
75 +         .lock()
76 +         .map_err(py_error)?;
77 +     packages
78 +         .entry(identity)
79 +         .or_insert_with(|| factory.clone().unbind());
80 +     let _ = py;
81 +     Ok(())
82 +}
83 +
84 +struct PythonRelationHandler {
85 +     owner: Arc<PythonOwner>,
86 +     type_url: String,
87 +     identity: String,
88 +     driver_registry: Option<Arc<DriverExtensionRegistry>>,
89 +}
90 +
91 +impl ConnectRelationHandler for PythonRelationHandler {
92 +     fn plan(
93 +         &self,
94 +         payload: &[u8],
95 +         inputs: Vec<Arc<dyn ExecutionPlan>>,
96 +     ) -> Result<Arc<dyn TableProvider>> {
97 +         Python::attach(|py| {
98 +             let capsules = PyList::empty(py);
99 +             let mut driver_inputs = Vec::new();
100 +             let mut names = Vec::new();
101 +             for input in inputs {
102 +                 let input = if self.driver_registry.is_some() {
103 +                     let host = input
104 +                         .downcast_ref:::<HostInputExec>()
105 +                         .ok_or_else(|| py_error("native driver input is missing host
adapter"))?;
106 +                     let input = host.gathered_input();
107 +                     let name = format!("SailNativeInput_{}", uuid::Uuid::new_v4());
108 +                     let placeholder = Arc::new(InputPlaceholder {
109 +                         name: name.clone(),
110 +                         properties: input.properties().clone(),
111 +                     });
112 +                     driver_inputs.push(input);
113 +                     names.push(name);
114 +                     placeholder as Arc<dyn ExecutionPlan>
115 +                 } else {
116 +                     input
117 +                 };
118 +                 let ffi = FFI_ExecutionPlan::new(input, tokio::runtime::Handle::try_c
urrent().ok());
119 +                 capsules
120 +                     .append(
121 +                         PyCapsule::new_with_value(py, ffi, c"datafusion_execution_pla
n")
122 +                     )
123 +                     .map_err(py_error)?;
124 +                 .map_err(py_error)?;
125 +             }
126 +             let result = self
127 +                 .owner

```

```

128 +         .bind(py)
129 +         .map_err(py_error)?
130 +         .call_method1(
131 +             "plan_relation",
132 +             (&self.type_url, PyBytes::new(py, payload), capsules),
133 +         )
134 +         .map_err(|e| py_error(format!("{}", self.identity)))?;
135 +     let capsule = result.cast::<PyCapsule>().map_err(py_error)?;
136 +     let pointer = capsule
137 +         .pointer_checked(Some(c"datafusion_table_provider"))
138 +         .map_err(py_error)?;
139 +     // SAFETY: exact build metadata was checked before binding this handler;
140 +     // the trusted package promises the named DataFusion capsule layout. No
141 +     // Python code runs between obtaining the pointer and cloning its owner.
142 +     let provider = unsafe { pointer.cast::<FFI_TableProvider>().as_ref().clone() };
143 +     if let Some(registry) = &self.driver_registry {
144 +         return Ok(Arc::new(DriverTableProvider {
145 +             inner: Arc::<dyn TableProvider>::from(&provider),
146 +             inputs: driver_inputs,
147 +             names,
148 +             owner: self.identity.clone(),
149 +             registry: registry.clone(),
150 +         }) as Arc<dyn TableProvider>);
151 +     }
152 +     Ok(
153 +         Arc::new(NativeTableProvider::new(Arc::<dyn TableProvider>::from(
154 +             &provider,
155 +         ))) as Arc<dyn TableProvider>,
156 +     )
157 + })
158 + }
159 + }
160 +
161 + /// All installation work happens on a fresh session. Validate every component
162 + /// before mutating the session's catalog, so a failure cannot expose a partial
163 + /// extension set to a client.
164 + pub(crate) fn register_extensions(
165 +     mut config: SessionConfig,
166 +     mode: &ExecutionMode,
167 +     runtime: &Arc<RuntimeEnv>,
168 + ) -> Result<SessionConfig> {
169 +     let mut registry = ConnectExtensionRegistry::new();
170 +     if std::env::var("SAIL_EXPERIMENTAL_EXTENSIONS").as_deref() != Ok("1") {
171 +         config.set_extension(Arc::new(registry));
172 +         return Ok(config);
173 +     }
174 +     let distributed = !matches!(mode, ExecutionMode::Local);
175 +     let resources = Arc::new(NativeResourceTracker::default());
176 +     config.set_extension(resources.clone());
177 +     let driver_registry = Arc::new(DriverExtensionRegistry::default());
178 +     config.set_extension(driver_registry.clone());
179 +     let catalog = config
180 +         .get_extension::<CatalogManager>()
181 +         .ok_or_else(|| py_error("session catalog is missing"))?;
182 +     let scalars = Python::attach(|py| -> Result<Vec<ScalarUDF>> {
183 +         let kwargs = PyDict::new(py);
184 +         kwargs
185 +             .set_item("group", "pysail.extensions")
186 +             .map_err(py_error)?;
187 +         let entries = py
188 +             .import("importlib.metadata")
189 +             .and_then(|m| m.getattr("entry_points"))
190 +             .and_then(|f| f.call((), Some(&kwargs)))
191 +             .map_err(py_error)?;
192 +         let mut entries = entries
193 +             .try_iter()
194 +             .map_err(py_error)?
195 +             .map(|item| {
196 +                 let item = item?;
197 +                 let name = item.getattr("name")?.extract::<String>()?;
198 +                 Ok((name, item))
199 +             })
200 +             .collect::<PyResult<Vec<_>>>()
201 +             .map_err(py_error)?;
202 +         entries.sort_by(|a, b| a.0.cmp(&b.0));
203 +         let mut identities = HashSet::new();
204 +         let mut names = HashSet::new();

```

```

205 +     let mut scalars = Vec::new();
206 +     // A fresh host-issued incarnation, not a client-selected graph namespace.
207 +     let incarnation = uuid::Uuid::new_v4().to_string();
208 +     for (entry_name, entry) in entries {
209 +         let loaded = entry.call_method0("load").map_err(py_error)?;
210 +         let factory = if loaded.is_callable() {
211 +             loaded.call0().map_err(py_error)?
212 +         } else {
213 +             loaded
214 +         };
215 +         let metadata = factory.call_method0("manifest").map_err(py_error)?;
216 +         let json = py
217 +             .import("json")
218 +             .and_then(|m| m.call_method1("dumps", (&metadata,)))
219 +             .and_then(|s| s.extract::<String>())
220 +             .map_err(py_error)?;
221 +         let manifest: Manifest = serde_json::from_str(&json).map_err(py_error)?;
222 +         manifest.validate()?;
223 +         if distributed && manifest.placement != "driver" && !manifest.relation_ty
pes.is_empty()
224 +         {
225 +             return plan_err!(
226 +                 "extension {} requires a worker relation codec; only driver-resid
ent relations are supported",
227 +                 manifest.name
228 +             );
229 +         }
230 +         if !identities.insert(manifest.name.to_ascii_lowercase()) {
231 +             return plan_err!(
232 +                 "duplicate native extension name: {} (entry point {entry_name})",
233 +                 manifest.name
234 +             );
235 +         }
236 +         let identity = package_identity(py, &entry, &metadata)?;
237 +         retain_package(py, identity.clone(), &factory)?;
238 +         let bound = if let Some(bytes) = manifest.memory_bytes {
239 +             let lease = resources.reserve(&runtime.memory_pool, &identity, bytes)
?;
240 +             let capsule =
241 +                 PyCapsule::new_with_value(py, lease, MEMORY_LEASE_CAPSULE).map_er
r(py_error)?;
242 +             factory.call_method1("bind_with_resources", (&incarnation, bytes, cap
sule))
243 +         } else {
244 +             factory.call_method1("bind", (&incarnation,))
245 +         };
246 +         .map_err(py_error)?;
247 +         let owner = Arc::new(PythonOwner::new(bound.unbind()));
248 +         let functions = owner
249 +             .bind(py)
250 +             .map_err(py_error)?
251 +             .call_method0("scalar_udfs")
252 +             .map_err(py_error)?;
253 +         for function in functions.try_iter().map_err(py_error)? {
254 +             let function = function.map_err(py_error)?;
255 +             if manifest.placement == "driver" {
256 +                 return plan_err!(
257 +                     "driver-only extension {} cannot export scalar functions",
258 +                     manifest.name
259 +                 );
260 +             }
261 +             let capsule = function
262 +                 .call_method0("__datafusion_scalar_udf__")
263 +                 .map_err(py_error)?;
264 +             let capsule = capsule.cast::<PyCapsule>().map_err(py_error)?;
265 +             let pointer = capsule
266 +                 .pointer_checked(Some(c"datafusion_scalar_udf"))
267 +                 .map_err(py_error)?;
268 +             // SAFETY: see the provider import above; clone while capsule is live
269 +             let ffi = unsafe { pointer.cast::<FFI_ScalarUDF>().as_ref().clone() }
;
270 +             let udf = ScalarUDF::new_from_shared_impl((&ffi).into());
271 +             let mut aliases = vec![udf.name().to_ascii_lowercase()];
272 +             aliases.extend(udf.aliases().iter().map(|s| s.to_ascii_lowercase()));
273 +             aliases.sort();
274 +             aliases.dedup();

```

```

275 + // Retain both the bound session and the exporting object.
276 + let owner: Arc<dyn std::any::Any + Send + Sync> = Arc::new(PythonOwner::new(
277 +     (owner.bind(py).map_err(py_error)?, function)
278 +     .into_pyobject(py)
279 +     .map_err(py_error)?
280 +     .into_any()
281 +     .unbind(),
282 + ));
283 + for name in aliases {
284 +     if is_built_in_function_name(&name)
285 +     || catalog.get_function(&name).map_err(py_error)?.is_some()
286 +     || !names.insert(name.clone())
287 +     {
288 +         return plan_err!(
289 +             "extension {} function name collision: {name}",
290 +             manifest.name
291 +         );
292 +     }
293 +     let scalar = ScalarUDF::new_from_impl(OwnedScalar {
294 +         name,
295 +         identity: identity.clone(),
296 +         udf: udf.clone(),
297 +         owner: Arc::clone(&owner),
298 +     });
299 +     retain_scalar(scalar.clone())?;
300 +     scalars.push(scalar);
301 + }
302 + }
303 + for relation in manifest.relation_types {
304 +     registry.register(
305 +         relation.type_url.clone(),
306 +         relation.accepts_bare,
307 +         relation.min_inputs,
308 +         relation.max_inputs,
309 +         Arc::new(PythonRelationHandler {
310 +             owner: Arc::clone(&owner),
311 +             type_url: relation.type_url,
312 +             identity: identity.clone(),
313 +             driver_registry: distributed.then(|| driver_registry.clone())
314 +         },
315 +     )?;
316 + }
317 + log::info!("bound native extension {identity} to session incarnation {incarnation}");
318 + }
319 + Ok(scalars)
320 + }?;
321 + let graph_scalars = graph_utils::register(
322 +     &mut config,
323 +     runtime,
324 +     &mut registry,
325 +     distributed.then_some(driver_registry),
326 + )?;
327 + for udf in scalars.into_iter().chain(graph_scalars) {
328 +     if catalog
329 +     .get_function(udf.name())
330 +     .map_err(py_error)?
331 +     .is_some()
332 +     {
333 +         return plan_err!("extension function name collision: {}", udf.name());
334 +     }
335 +     catalog.register_function(udf).map_err(py_error)?;
336 + }
337 + config.set_extension(Arc::new(registry));
338 + Ok(config)
339 + }
340 +
341 + /// Load scalar implementations on an execution worker before it decodes a
342 + /// distributed physical plan. The worker does not install relation handlers or
343 + /// mutate the driver catalog; the codec resolves native scalar descriptors from
344 + /// the process-local registry populated here.
345 + pub(crate) fn load_worker_extensions() -> Result<> {
346 +     graph_utils::register_worker_functions()?;
347 +     Python::attach(|py| {
348 +         let kwargs = PyDict::new(py);
349 +         kwargs

```

```

350 +         .set_item("group", "pysail.extensions")
351 +         .map_err(py_error)?;
352 +     let entries = py
353 +         .import("importlib.metadata")
354 +         .and_then(|m| m.getattr("entry_points"))
355 +         .and_then(|f| f.call((), Some(&kwargs)))
356 +         .map_err(py_error)?;
357 +     for entry in entries.try_iter().map_err(py_error)? {
358 +         let entry = entry.map_err(py_error)?;
359 +         let loaded = entry.call_method0("load").map_err(py_error)?;
360 +         let factory = if loaded.is_callable() {
361 +             loaded.call0().map_err(py_error)?
362 +         } else {
363 +             loaded
364 +         };
365 +         let metadata = factory.call_method0("manifest").map_err(py_error)?;
366 +         let json = py
367 +             .import("json")
368 +             .and_then(|m| m.call_method1("dumps", (&metadata,)))
369 +             .and_then(|s| s.extract::<String>())
370 +             .map_err(py_error)?;
371 +         let manifest: Manifest = serde_json::from_str(&json).map_err(py_error)?;
372 +         manifest.validate()?;
373 +         if manifest.placement == "driver" {
374 +             continue;
375 +         }
376 +         let identity = package_identity(py, &entry, &metadata)?;
377 +         retain_package(py, identity.clone(), &factory)?;
378 +         log::info!(
379 +             "worker loaded native extension {identity}, pid={}",
380 +             std::process::id()
381 +         );
382 +         let owner = factory
383 +             .call_method1("bind", (format!("worker-{{identity}}"),))
384 +             .map_err(py_error)?;
385 +         for function in owner
386 +             .call_method0("scalar_udfs")
387 +             .map_err(py_error)?
388 +             .try_iter()
389 +             .map_err(py_error)?
390 +         {
391 +             let function = function.map_err(py_error)?;
392 +             let capsule = function
393 +                 .call_method0("__datafusion_scalar_udf__")
394 +                 .map_err(py_error)?;
395 +             let capsule = capsule.cast::<PyCapsule>().map_err(py_error)?;
396 +             let pointer = capsule
397 +                 .pointer_checked(Some(c"datafusion_scalar_udf"))
398 +                 .map_err(py_error)?;
399 +             // SAFETY: the validated capsule owns an FFI_ScalarUDF, and the
400 +             // Python owner/function are retained below for its full lifetime.
401 +             let ffi = unsafe { pointer.cast::<FFI_ScalarUDF>().as_ref().clone() };
402 +             ;
403 +             let udf = ScalarUDF::new_from_shared_impl((&ffi).into());
404 +             let owner: Arc<dyn std::any::Any + Send + Sync> = Arc::new(
405 +                 (owner.clone(), function)
406 +                     .into_pyobject(py)
407 +                     .map_err(py_error)?
408 +                     .into_any()
409 +                     .unbind(),
410 +             );
411 +             let mut names = vec![udf.name().to_ascii_lowercase()];
412 +             names.extend(udf.aliases().iter().map(|name| name.to_ascii_lowercase(
413 +             )));
414 +             names.sort_unstable();
415 +             names.dedup();
416 +             for name in names {
417 +                 retain_scalar(ScalarUDF::new_from_impl(OwnedScalar {
418 +                     name,
419 +                     identity: identity.clone(),
420 +                     udf: udf.clone(),
421 +                     owner: Arc::clone(&owner),
422 +                 }))?;
423 +             }
424 +         }
425 +     }
426 +     Ok(())
427 + }

```

50. crates/sail-session/src/extensions/package_identity.py

```

1 diff --git a/crates/sail-session/src/extensions/package_identity.py b/crates/sail-session/src/extensions/package_identity.py
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..f2e13a1b887ac5fa06dcc0fe59b5f2e7d931de04
4 --- /dev/null
5 +++ b/crates/sail-session/src/extensions/package_identity.py
6 @@ -0,0 +1,26 @@
7 +"""Content identity for installed native wheels and immutable manifest options."""
8 +import hashlib
9 +import json
10 +
11 +
12 +def identity(entry, manifest):
13 +    digest = hashlib.sha256()
14 +    digest.update(json.dumps(manifest, sort_keys=True, separators=(",", ":")).encode())
15 +    files = entry.dist.files
16 +    if not files:
17 +        raise ValueError("native extension needs an installed wheel file manifest")
18 +    count = 0
19 +    for relative in sorted(files, key=str):
20 +        name = str(relative)
21 +        if ".dist-info/" in name or "__pycache__/" in name or name.endswith(".pyc"):
22 +            continue
23 +        path = entry.dist.locate_file(relative)
24 +        if not path.is_file():
25 +            raise ValueError(f"native extension package file missing: {relative}")
26 +        digest.update(name.encode() + b"\0")
27 +        with path.open("rb") as stream:
28 +            digest.update(hashlib.file_digest(stream, "sha256").digest())
29 +        count += 1
30 +    if count == 0:
31 +        raise ValueError("native extension wheel contains no package files")
32 +    return f"{manifest['name']}@{manifest['version']}:{digest.hexdigest()}"

```

51. crates/sail-session/src/extensions/plan.rs

```

1 diff --git a/crates/sail-session/src/extensions/plan.rs b/crates/sail-session/src/exte
nsions/plan.rs
2 new file mode 100644
3 index 00000000000000000000000000000000..b189616806e6169a538e07668f9d098fef4f19
9f
4 --- /dev/null
5 +++ b/crates/sail-session/src/extensions/plan.rs
6 @@ -0,0 +1,126 @@
7 +///! Local physical-plan boundary for an already-planned native relation.
8 +///!
9 +///! DataFusion's FFI does not preserve all input requirements, and foreign child
10 +///! replacement can export host-added nodes without a host Tokio runtime. Do not
11 +///! let the outer host optimizer rewrite this mixed-library execution region.
12 +
13 +use std::fmt::{Formatter, Result as FmtResult};
14 +use std::sync::Arc;
15 +
16 +use arrow_schema::SchemaRef;
17 +use async_trait::async_trait;
18 +use datafusion::catalog::{Session, TableProvider};
19 +use datafusion::execution::{SendableRecordBatchStream, TaskContext};
20 +use datafusion::physical_expr::PhysicalExpr;
21 +use datafusion::physical_plan::{DisplayAs, DisplayFormatType, ExecutionPlan, PlanProp
erties};
22 +use datafusion_common::tree_node::TreeNodeRecursion;
23 +use datafusion_common::{Result, Statistics, plan_err};
24 +use datafusion_expr::{Expr, TableProviderFilterPushDown, TableType};
25 +
26 +#[derive(Debug)]
27 +pub(super) struct NativeTableProvider {
28 +    inner: Arc<dyn TableProvider>,

```

```

29 +}
30 +
31 +impl NativeTableProvider {
32 +   pub fn new(inner: Arc<dyn TableProvider>) -> Self {
33 +       Self { inner }
34 +   }
35 +}
36 +
37 +#[async_trait]
38 +impl TableProvider for NativeTableProvider {
39 +   fn schema(&self) -> SchemaRef {
40 +       self.inner.schema()
41 +   }
42 +
43 +   fn table_type(&self) -> TableType {
44 +       self.inner.table_type()
45 +   }
46 +
47 +   fn supports_filters_pushdown(
48 +       &self,
49 +       filters: &[&Expr],
50 +   ) -> Result<Vec<TableProviderFilterPushDown>> {
51 +       self.inner.supports_filters_pushdown(filters)
52 +   }
53 +
54 +   fn statistics(&self) -> Option<Statistics> {
55 +       self.inner.statistics()
56 +   }
57 +
58 +   async fn scan(
59 +       &self,
60 +       session: &dyn Session,
61 +       projection: Option<&Vec<usize>>,
62 +       filters: &[Expr],
63 +       limit: Option<usize>,
64 +   ) -> Result<Arc<dyn ExecutionPlan>> {
65 +       let inner = self.inner.scan(session, projection, filters, limit).await?;
66 +       Ok(Arc::new(NativeRelationExec {
67 +           inner,
68 +           // Keep provider-owned native resources alive with its execution.
69 +           _provider: Arc::clone(&self.inner),
70 +       }))
71 +   }
72 +}
73 +
74 +#[derive(Debug)]
75 +struct NativeRelationExec {
76 +   inner: Arc<dyn ExecutionPlan>,
77 +   _provider: Arc<dyn TableProvider>,
78 +}
79 +
80 +impl DisplayAs for NativeRelationExec {
81 +   fn fmt_as(&self, _t: DisplayFormatType, f: &mut Formatter<'_>) -> FmtResult {
82 +       write!(
83 +           f,
84 +           "NativeRelationExec: local, opaque_region=true, native={}",
85 +           self.inner.name()
86 +       )
87 +   }
88 +}
89 +
90 +impl ExecutionPlan for NativeRelationExec {
91 +   fn name(&self) -> &'static str {
92 +       "NativeRelationExec"
93 +   }
94 +
95 +   fn properties(&self) -> &Arc<PlanProperties> {
96 +       self.inner.properties()
97 +   }
98 +
99 +   fn children(&self) -> Vec<&Arc<dyn ExecutionPlan>> {
100 +       // Host inputs were optimized before export. The package owns the
101 +       // complete region below this leaf, including its input distribution.
102 +       vec![]
103 +   }
104 +
105 +   fn apply_expressions(
106 +       &self,

```

```

107 +     _f: &mut dyn FnMut(&Arc<dyn PhysicalExpr>) -> Result<TreeNodeRecursion>,
108 + ) -> Result<TreeNodeRecursion> {
109 +     Ok(TreeNodeRecursion::Continue)
110 + }
111 +
112 + fn with_new_children(
113 +     self: Arc<Self>,
114 +     children: Vec<Arc<dyn ExecutionPlan>>,
115 + ) -> Result<Arc<dyn ExecutionPlan>> {
116 +     if !children.is_empty() {
117 +         return plan_err!("NativeRelationExec is an opaque leaf and does not accep
118 + t children");
119 +     }
120 +     Ok(self)
121 + }
122 +
123 + fn execute(
124 +     &self,
125 +     partition: usize,
126 +     context: Arc<TaskContext>,
127 + ) -> Result<SendableRecordBatchStream> {
128 +     self.inner.execute(partition, context)
129 + }
130 +
131 +#[cfg(test)]
132 +mod tests;

```

52. crates/sail-session/src/extensions/plan/tests.rs

```

1 diff --git a/crates/sail-session/src/extensions/plan/tests.rs b/crates/sail-session/src/
2 extensions/plan/tests.rs
3 new file mode 100644
4 index 0000000000000000000000000000000000000000..bbfffd2f512e83c4c7fbd74b7d13e7ce700c96
5 a2
6 --- /dev/null
7 +++ b/crates/sail-session/src/extensions/plan/tests.rs
8 @@ -0,0 +1,90 @@
9 +use datafusion::arrow::array::Int64Array;
10 +use datafusion::arrow::datatypes::{DataType, Field, Schema};
11 +use datafusion::arrow::record_batch::RecordBatch;
12 +use datafusion::physical_optimizer::PhysicalOptimizerRule;
13 +use datafusion::physical_optimizer::ensure_requirements::EnsureRequirements;
14 +use datafusion::physical_plan::coalesce_partitions::CoalescePartitionsExec;
15 +use datafusion::physical_plan::common::collect;
16 +use datafusion::physical_plan::displayable;
17 +use datafusion::prelude::{SessionConfig, SessionContext};
18 +use datafusion_datasource::memory::MemorySourceConfig;
19 +use datafusion_ffi::execution_plan::{FFI_ExecutionPlan, ForeignExecutionPlan};
20 +use sail_common_datafusion::connect_extension::HostInputExec;
21 +
22 +use super::*;
23 +
24 +#[derive(Debug)]
25 +struct FixedProvider(Arc<dyn ExecutionPlan>);
26 +
27 +#[async_trait]
28 +impl TableProvider for FixedProvider {
29 +    fn schema(&self) -> SchemaRef {
30 +        self.0.schema()
31 +    }
32 +    fn table_type(&self) -> TableType {
33 +        TableType::Temporary
34 +    }
35 +    async fn scan(
36 +        &self,
37 +        _session: &dyn Session,
38 +        _projection: Option<&Vec<usize>>,
39 +        _filters: &[Expr],
40 +        _limit: Option<usize>,
41 +    ) -> Result<Arc<dyn ExecutionPlan>> {
42 +        Ok(Arc::clone(&self.0))
43 +    }
44 +}

```

```

44 +#[tokio::test]
45 +async fn native_region_blocks_host_repartition_insertion_inside_ffi_plan() -> Result<
46 +    ()> {
47 +    // Keep the two-row fixture above the optimizer's one-batch threshold;
48 +    // otherwise its exact in-memory statistics suppress repartitioning.
49 +    let ctx = SessionContext::new_with_config(
50 +        SessionConfig::new()
51 +            .with_target_partitions(4)
52 +            .with_batch_size(1),
53 +    );
54 +    let state = ctx.state();
55 +    let schema = Arc::new(Schema::new(vec![Field::new("id", DataType::Int64, false)]));
56 +    let batch = RecordBatch::try_new(schema.clone(), vec![Arc::new(Int64Array::from(vec![1, 2]))]);
57 +    let source = MemorySourceConfig::try_new_exec(&[batch], schema, None)?;
58 +    let input = Arc::new(HostInputExec::new(
59 +        source,
60 +        ctx.task_ctx(),
61 +        tokio::runtime::Handle::current(),
62 +    ));
63 +    // Force the actual foreign adapter even in this one-library fixture. Its
64 +    // default input requirements permit host repartitioning of its local child.
65 +    let ffi = FFI_ExecutionPlan::new(
66 +        Arc::new(CoalescePartitionsExec::new(input)),
67 +        Some(tokio::runtime::Handle::current()),
68 +    );
69 +    let foreign: Arc<dyn ExecutionPlan> = Arc::new(ForeignExecutionPlan::try_from(ffi)?);
70 +    assert!(foreign.children().is_empty());
71 +    let optimizer = EnsureRequirements::new();
72 +    let rewritten = optimizer.optimize(Arc::clone(&foreign), state.config_options())?;
73 +    assert!(
74 +        displayable(rewritten.as_ref())
75 +            .indent(true)
76 +            .to_string()
77 +            .contains("RepartitionExec"),
78 +        "fixture must reproduce host insertion within an exposed foreign graph"
79 +    );
80 +    let provider = NativeTableProvider::new(Arc::new(FixedProvider(Arc::clone(&foreign))));
81 +    let plan = provider.scan(&state, None, &[], None).await?;
82 +    assert_eq!(plan.schema(), foreign.schema());
83 +    assert!(Arc::ptr_eq(plan.properties(), foreign.properties()));
84 +    let plan = optimizer.optimize(plan, state.config_options())?;
85 +    assert_eq!(plan.name(), "NativeRelationExec");
86 +    assert!(plan.children().is_empty());
87 +    #[expect(deprecated)]
88 +    let unchanged = Arc::clone(&plan).with_new_children(vec![])?;
89 +    #[expect(deprecated)]
90 +    let rejected = Arc::clone(&plan).with_new_children(vec![foreign]);
91 +    assert!(rejected.is_err());
92 +    assert!(Arc::ptr_eq(&plan, &unchanged));
93 +    let batches = collect(plan.execute(0, ctx.task_ctx())?).await?;
94 +    assert_eq!(batches.iter().map(RecordBatch::num_rows).sum::<usize>(), 2);
95 +    Ok(())
96 +}

```

53. crates/sail-session/src/extensions/python_owner.rs

```

1 diff --git a/crates/sail-session/src/extensions/python_owner.rs b/crates/sail-session/
2 src/extensions/python_owner.rs
3 new file mode 100644
4 index 0000000000000000000000000000000000000000..21651b1340b18817c6828c5cf1d492eaaa9945
5 60
6 --- /dev/null
7 +++ b/crates/sail-session/src/extensions/python_owner.rs
8 @@ -0,0 +1,66 @@
9 +use pyo3::prelude::*;
10 +
11 +/// Finalize session-owned Python objects when the last native owner is dropped.
12 +/// PyO3 otherwise queues a decref outside the GIL until a later Python entry,
13 +/// which can retain native session quotas indefinitely on an idle server.

```

```

12 +pub(super) struct PythonOwner(Option<Py<PyAny>>);
13 +
14 +impl PythonOwner {
15 +     pub(super) fn new(value: Py<PyAny>) -> Self {
16 +         Self(Some(value))
17 +     }
18 +
19 +     pub(super) fn bind<'py>(&self, py: Python<'py>) -> PyResult<&Bound<'py, PyAny>> {
20 +         self.0.as_ref().map(|value| value.bind(py)).ok_or_else(|| {
21 +             pyo3::exceptions::PyRuntimeError::new_err("native Python owner was already
y dropped")
22 +         })
23 +     }
24 +}
25 +
26 +impl Drop for PythonOwner {
27 +     fn drop(&mut self) {
28 +         if let Some(value) = self.0.take() {
29 +             Python::attach(|_| drop(value));
30 +         }
31 +     }
32 +}
33 +
34 +#[cfg(test)]
35 +mod tests {
36 +     use std::sync::Arc;
37 +     use std::sync::atomic::{AtomicUsize, Ordering};
38 +
39 +     use pyo3::types::PyCapsule;
40 +
41 +     use super::*;
42 +
43 +     struct Released(Arc<AtomicUsize>);
44 +
45 +     impl Drop for Released {
46 +         fn drop(&mut self) {
47 +             self.0.fetch_add(1, Ordering::SeqCst);
48 +         }
49 +     }
50 +
51 +     #[test]
52 +     fn last_owner_releases_without_a_later_python_request() -> PyResult<()> {
53 +         Python::initialize();
54 +         let released = Arc::new(AtomicUsize::new(0));
55 +         let owner = Python::attach(|py| -> PyResult<_> {
56 +             Ok(Arc::new(PythonOwner::new(
57 +                 PyCapsule::new_with_value(py, Released(released.clone()), c"resource_
test")?
58 +                     .into_any()
59 +                     .unbind(),
60 +                 )))
61 +         })?;
62 +         let output_owner = owner.clone();
63 +         drop(owner);
64 +         assert_eq!(released.load(Ordering::SeqCst), 0);
65 +         // No further request, Python attachment, or GC occurs after this drop.
66 +         std::thread::spawn(move || drop(output_owner))
67 +             .join()
68 +             .map_err(|_| pyo3::exceptions::PyRuntimeError::new_err("owner thread panicked"))?;
69 +         assert_eq!(released.load(Ordering::SeqCst), 1);
70 +         Ok(())
71 +     }
72 +}

```

54. crates/sail-session/src/extensions/resource_tests.rs

```

1 diff --git a/crates/sail-session/src/extensions/resource_tests.rs b/crates/sail-session/
n/src/extensions/resource_tests.rs
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..a755517229c96ceba2b0be23932dacf0cc2d0f
10
4 --- /dev/null
5 +++ b/crates/sail-session/src/extensions/resource_tests.rs
6 @@ -0,0 +1,188 @@

```

```

7 +use std::fmt::Formatter;
8 +use std::io::Write;
9 +use std::sync::Arc;
10 +
11 +use datafusion::arrow::datatypes::Schema;
12 +use datafusion::arrow::record_batch::RecordBatch;
13 +use datafusion::execution::disk_manager::{DiskManagerBuilder, DiskManagerMode};
14 +use datafusion::execution::memory_pool::{GreedyMemoryPool, MemoryConsumer, MemoryPool};
15 +use datafusion::execution::runtime_env::RuntimeEnvBuilder;
16 +use datafusion::execution::{SendableRecordBatchStream, TaskContext};
17 +use datafusion::physical_expr::PhysicalExpr;
18 +use datafusion::physical_plan::common::collect;
19 +use datafusion::physical_plan::empty::EmptyExec;
20 +use datafusion::physical_plan::stream::RecordBatchStreamAdapter;
21 +use datafusion::physical_plan::{DisplayAs, DisplayFormatType, ExecutionPlan, PlanProperties};
22 +use datafusion::prelude::{SessionConfig, SessionContext};
23 +use datafusion_common::tree_node::TreeNodeRecursion;
24 +use datafusion_common::{Result, plan_err};
25 +use datafusion_ffi::execution_plan::{FFI_ExecutionPlan, ForeignExecutionPlan};
26 +use sail_common_datafusion::connect_extension::HostInputExec;
27 +use sail_common_datafusion::native_resource::reserve_native_quota;
28 +
29 +/// A participating host input whose fixed reservation gives the pressure test
30 +/// an exact boundary independent of allocator sizes or DataFusion heuristics.
31 +#[derive(Debug)]
32 +struct ReservingInput {
33 +    bytes: usize,
34 +    spill_bytes: usize,
35 +    properties: Arc<PlanProperties>,
36 +}
37 +
38 +impl DisplayAs for ReservingInput {
39 +    fn fmt_as(&self, _: DisplayFormatType, f: &mut Formatter<'>_) -> std::fmt::Result {
40 +        write!(f, "ReservingInput: bytes={}", self.bytes)
41 +    }
42 +}
43 +
44 +impl ExecutionPlan for ReservingInput {
45 +    fn name(&self) -> &str {
46 +        "ReservingInput"
47 +    }
48 +    fn properties(&self) -> &Arc<PlanProperties> {
49 +        &self.properties
50 +    }
51 +    fn children(&self) -> Vec<&Arc<dyn ExecutionPlan>> {
52 +        vec![]
53 +    }
54 +    fn apply_expressions(
55 +        &self,
56 +        _: &mut dyn FnMut(&Arc<dyn PhysicalExpr>) -> Result<TreeNodeRecursion>,
57 +    ) -> Result<TreeNodeRecursion> {
58 +        Ok(TreeNodeRecursion::Continue)
59 +    }
60 +    fn with_new_children(
61 +        self: Arc<Self>,
62 +        children: Vec<Arc<dyn ExecutionPlan>>,
63 +    ) -> Result<Arc<dyn ExecutionPlan>> {
64 +        if !children.is_empty() {
65 +            return plan_err!("reserving input has no children");
66 +        }
67 +        Ok(self)
68 +    }
69 +    fn execute(&self, _: usize, context: Arc<TaskContext>) -> Result<SendableRecordBatchStream> {
70 +        let reservation =
71 +            MemoryConsumer::new("foreign parent host input").register(context.memory_pool());
72 +        reservation.try_grow(self.bytes)?;
73 +        let spill = if self.spill_bytes > 0 {
74 +            let file = context
75 +                .runtime_env()
76 +                .disk_manager
77 +                .create_tmp_file("foreign input spill")?;
78 +            let mut writer = file.open_writer()?;
79 +            writer.write_all(&vec![0; self.spill_bytes])?;

```

```

80 +         writer.finish()?;
81 +         Some(file)
82 +     } else {
83 +         None
84 +     };
85 +     let schema = self.schema();
86 +     Ok(Box::pin(RecordBatchStreamAdapter::new(
87 +         schema.clone(),
88 +         futures::stream::once(async move {
89 +             let _reservation = reservation;
90 +             let _spill = spill;
91 +             Ok(RecordBatch::new_empty(schema))
92 +         })),
93 +     )))
94 + }
95 +}
96 +
97 +#[tokio::test]
98 +async fn foreign_input_refuses_host_pressure_instead_of_using_an_unbounded_default()
99 +-> Result<()> {
100 +    let pool: Arc<dyn MemoryPool> = Arc::new(GreedyMemoryPool::new(128));
101 +    let runtime = Arc::new(
102 +        RuntimeEnvBuilder::default()
103 +        .with_memory_pool(pool.clone())
104 +        .build()?,
105 +    );
106 +    let host = SessionContext::new_with_config_rt(SessionConfig::new(), runtime);
107 +    let quota = reserve_native_quota(&pool, "nutmeg-test", 96)?;
108 +    let input: Arc<dyn ExecutionPlan> = Arc::new(ReservingInput {
109 +        bytes: 33,
110 +        spill_bytes: 0,
111 +        properties: EmptyExec::new(Arc::new(Schema::empty()))
112 +        .properties()
113 +        .clone(),
114 +    });
115 +    let foreign_context = Arc::new(TaskContext::default());
116 +    // Control proves that losing the host's pool would silently admit the work.
117 +    collect(input.execute(0, foreign_context.clone())?.await?);
118 +    let adapter = Arc::new(HostInputExec::new(
119 +        input,
120 +        host.task_ctx(),
121 +        tokio::runtime::Handle::current(),
122 +    ));
123 +    let ffi = FFI_ExecutionPlan::new(adapter, Some(tokio::runtime::Handle::current()));
124 +    // Force the real foreign adapter, bypassing its same-library shortcut.
125 +    let foreign: Arc<dyn ExecutionPlan> = Arc::new(ForeignExecutionPlan::try_from(ffi)?);
126 +    let result = match foreign.execute(0, foreign_context.clone()) {
127 +        Ok(stream) => collect(stream).await.map(|_| ()),
128 +        Err(error) => Err(error),
129 +    };
130 +    let message = match result {
131 +        Ok(()) => return plan_err!("96 native + 33 input unexpectedly fitted the 128-
byte pool"),
132 +        Err(error) => error.to_string(),
133 +    };
134 +    assert!(message.contains("Resources exhausted"), "{message}");
135 +    assert!(message.contains("foreign parent host input"), "{message}");
136 +    assert_eq!(
137 +        pool.reserved(),
138 +        96,
139 +        "refused host input must leak no reservation"
140 +    );
141 +    drop(quota);
142 +    collect(foreign.execute(0, foreign_context)?.await?);
143 +    assert_eq!(pool.reserved(), 0);
144 +    Ok(())
145 +}
146 +
147 +#[tokio::test]
148 +async fn foreign_input_preserves_disabled_and_exhausted_host_spill_policies() -> Resu
lt<()> {
149 +    for (mode, expected) in [
150 +        (DiskManagerMode::Disabled, "DiskManager is disabled"),
151 +        (
152 +            DiskManagerMode::OsTmpDirectory,
153 +            "spilling process has exceeded the allowable limit",

```

```

153 +     ),
154 +   ] {
155 +     let runtime = Arc::new(
156 +       RuntimeEnvBuilder::default()
157 +         .with_disk_manager_builder(
158 +           DiskManagerBuilder::default()
159 +             .with_mode(mode)
160 +             .with_max_temp_directory_size(1),
161 +         )
162 +         .build()?,
163 +     );
164 +     let host = SessionContext::new_with_config_rt(SessionConfig::new(), runtime.clone());
165 +     let input: Arc<dyn ExecutionPlan> = Arc::new(ReservingInput {
166 +       bytes: 0,
167 +       spill_bytes: 2,
168 +       properties: EmptyExec::new(Arc::new(Schema::empty()))
169 +         .properties()
170 +         .clone(),
171 +     });
172 +     let foreign_context = Arc::new(TaskContext::default());
173 +     // Losing the disk policy would permit this exact two-byte spill.
174 +     collect(input.execute(0, foreign_context.clone())?.await?);
175 +     let adapted = Arc::new(HostInputExec::new(
176 +       input,
177 +       host.task_ctx(),
178 +       tokio::runtime::Handle::current(),
179 +     ));
180 +     let ffi = FFI_ExecutionPlan::new(adapted, Some(tokio::runtime::Handle::current()));
181 +     let foreign = ForeignExecutionPlan::try_from(ffi)?;
182 +     let result = match foreign.execute(0, foreign_context) {
183 +       Ok(stream) => collect(stream).await.map(|_| ()),
184 +       Err(error) => Err(error),
185 +     };
186 +     let message = match result {
187 +       Ok(()) => return plan_err!("foreign input ignored its host spill policy")
188 +       ,
189 +       Err(error) => error.to_string(),
190 +     };
191 +     assert!(message.contains(expected), "{message}");
192 +     assert_eq!(runtime.disk_manager.used_disk_space(), 0);
193 +   }
194 + }

```

55. crates/sail-session/src/lib.rs

```

1 diff --git a/crates/sail-session/src/lib.rs b/crates/sail-session/src/lib.rs
2 index fb388c54b86705fb9c2566fddf051765a85aac48..25fd67d0e5b9427b869085bd5958b9d3b855a0
3 5e 100644
4 --- a/crates/sail-session/src/lib.rs
5 +++ b/crates/sail-session/src/lib.rs
6 @@ -1,5 +1,6 @@
7 pub mod catalog;
8 pub mod error;
9 +mod extensions;
10 pub mod formats;
11 pub mod optimizer;
12 pub mod planner;

```

56. crates/sail-session/src/runtime.rs

```

1 diff --git a/crates/sail-session/src/runtime.rs b/crates/sail-session/src/runtime.rs
2 index d0004dc4d763b98a05da438072cf7828b8d662c8..1807e9bed7656ea78eab75ddaa49886cc29221
3 1a 100644
4 --- a/crates/sail-session/src/runtime.rs
5 +++ b/crates/sail-session/src/runtime.rs
6 @@ -1,22 +1,19 @@
7 use std::sync::Arc;
8
9 +mod memory;
10 use datafusion::execution::DiskManager;

```

57. [crates/sail-session/src/runtime/memory.rs](#)

100

```

7 +/// Explicit admission domains. Configuration equality never implies shared ownership
8 +
9 +use std::sync::Arc;
10 +
11 +use datafusion::execution::memory_pool::{
12 +     FairSpillPool, GreedyMemoryPool, MemoryPool, UnboundedMemoryPool,
13 + };
14 +use sail_common::config::{FairMemoryPoolConfig, GreedyMemoryPoolConfig, MemoryPoolCon
15 +     fig};
16 +
17 +/// One explicitly owned admission domain. Clone this value to share admission;
18 +/// constructing another domain, even with identical settings, creates isolation.
19 +/// The standard server owns one per session manager and injects it into its
20 +/// sessions and in-process workers. Separate worker processes own their own.
21 +#[derive(Clone, Debug)]
22 +pub struct MemoryResourceDomain {
23 +    pool: Arc<dyn MemoryPool>,
24 +}
25 +
26 +impl MemoryResourceDomain {
27 +    pub fn new(config: &MemoryPoolConfig) -> Self {
28 +        Self {
29 +            pool: new_pool(config),
30 +        }
31 +    }
32 +
33 +    pub fn pool(&self) -> Arc<dyn MemoryPool> {
34 +        self.pool.clone()
35 +    }
36 +}
37 +
38 +pub(super) fn new_pool(config: &MemoryPoolConfig) -> Arc<dyn MemoryPool> {
39 +    match config {
40 +        MemoryPoolConfig::Unbounded => Arc::new(UnboundedMemoryPool::default()),
41 +        MemoryPoolConfig::Greedy(GreedyMemoryPoolConfig { max_size }) => {
42 +            Arc::new(GreedyMemoryPool::new(*max_size))
43 +        }
44 +        MemoryPoolConfig::Fair(FairMemoryPoolConfig { max_size }) => {
45 +            Arc::new(FairSpillPool::new(*max_size))
46 +        }
47 +    }
48 +}
49 +
50 +#[cfg(test)]
51 +mod tests {
52 +    use datafusion::execution::memory_pool::MemoryConsumer;
53 +    use sail_common_datafusion::native_resource::reserve_native_quota;
54 +
55 +    use super::*;
56 +
57 +    #[test]
58 +    fn explicit_clones_contend_but_equal_config_domains_are_isolated()
59 +    -> datafusion_common::Result<()> {
60 +        for config in [
61 +            MemoryPoolConfig::Greedy(GreedyMemoryPoolConfig { max_size: 113 }),
62 +            MemoryPoolConfig::Fair(FairMemoryPoolConfig { max_size: 113 }),
63 +        ] {
64 +            let domain = MemoryResourceDomain::new(&config);
65 +            let worker = domain.clone();
66 +            let unrelated = MemoryResourceDomain::new(&config);
67 +            assert!(Arc::ptr_eq(&domain.pool(), &worker.pool()));
68 +            assert!(!Arc::ptr_eq(&domain.pool(), &unrelated.pool()));
69 +            let a = reserve_native_quota(&domain.pool(), "session-a", 64)?;
70 +            let b = reserve_native_quota(&domain.pool(), "session-b", 32)?;
71 +            let query = MemoryConsumer::new("worker join").register(&worker.pool());
72 +            assert!(query.try_grow(18).is_err());
73 +            query.try_grow(17)?;
74 +            let other = reserve_native_quota(&unrelated.pool(), "other-server", 113)?;
75 +
76 +            assert_eq!(domain.pool().reserved(), 113);
77 +            drop(domain);
78 +            // A lease and an explicitly shared worker keep admission alive.
79 +            drop(a);
80 +            assert_eq!(worker.pool().reserved(), 49);
81 +            assert_eq!(unrelated.pool().reserved(), 113);
82 +            query.try_grow(64)?;
83 +            drop(b);
84 +            drop(query);

```

```

82 +         assert_eq!(worker.pool().reserved(), 0);
83 +         drop(other);
84 +         assert_eq!(unrelated.pool().reserved(), 0);
85 +     }
86 +     Ok(())
87 + }
88 + #[tokio::test]
89 + async fn runtime_factories_share_only_the_injected_domain() -> datafusion_common::
:Result<()> {
90 +     use sail_common::config::AppConfig;
91 +     use sail_common::runtime::RuntimeHandle;
92 +
93 +     use crate::runtime::RuntimeEnvFactory;
94 +
95 +     let mut config = AppConfig::load()
96 +         .map_err(|e| datafusion_common::DataFusionError::External(Box::new(e)))?;
97 +     config.runtime.memory_pool =
98 +         MemoryPoolConfig::Greedy(GreedyMemoryPoolConfig { max_size: 113 });
99 +     let domain = MemoryResourceDomain::new(&config.runtime.memory_pool);
100 +     let other = MemoryResourceDomain::new(&config.runtime.memory_pool);
101 +     let handle = tokio::runtime::Handle::current();
102 +     let runtime = RuntimeHandle::new(handle.clone(), handle);
103 +     let config = Arc::new(config);
104 +     let mut server = RuntimeEnvFactory::new(config.clone(), runtime.clone());
105 +     let mut worker = RuntimeEnvFactory::new(config, runtime);
106 +     let a = server.create(Some(&domain), Ok)?;
107 +     let b = server.create(Some(&domain), Ok)?;
108 +     let w = worker.create(Some(&domain), Ok)?;
109 +     let unrelated = server.create(Some(&other), Ok)?;
110 +     let standalone = server.create(None, Ok)?;
111 +     let standalone2 = server.create(None, Ok)?;
112 +     assert!(Arc::ptr_eq(&a.memory_pool, &b.memory_pool));
113 +     assert!(Arc::ptr_eq(&a.memory_pool, &w.memory_pool));
114 +     assert!(!Arc::ptr_eq(&a.memory_pool, &unrelated.memory_pool));
115 +     assert!(!Arc::ptr_eq(&a.memory_pool, &standalone.memory_pool));
116 +     assert!(!Arc::ptr_eq(
117 +         &standalone.memory_pool,
118 +         &standalone2.memory_pool
119 +     ));
120 +     let native = reserve_native_quota(&a.memory_pool, "native", 100)?;
121 +     let query = MemoryConsumer::new("worker").register(&w.memory_pool);
122 +     assert!(query.try_grow(14).is_err());
123 +     query.try_grow(13)?;
124 +     let independent = reserve_native_quota(&unrelated.memory_pool, "independent",
113)?;
125 +     drop(native);
126 +     drop(query);
127 +     drop(independent);
128 +     assert_eq!(a.memory_pool.reserved(), 0);
129 +     assert_eq!(unrelated.memory_pool.reserved(), 0);
130 +     Ok(())
131 + }
132 +}

```

58. crates/sail-session/src/session_factory/job_runner.rs

```

1 diff --git a/crates/sail-session/src/session_factory/job_runner.rs b/crates/sail-sessi
on/src/session_factory/job_runner.rs
2 index 616b0855ba3c31c716fe5275ee959d823aaf2013..85038a84ce4301cf7f8d38b0b0e2315ca5ba5c
fd 100644
3 --- a/crates/sail-session/src/session_factory/job_runner.rs
4 +++ b/crates/sail-session/src/session_factory/job_runner.rs
5 @@ -13,6 +13,7 @@ use sail_execution::worker_manager::{
6     use sail_telemetry::events::SystemEventReporter;
7
8     use crate::error::{SessionError, SessionResult};
9 +use crate::runtime::MemoryResourceDomain;
10 use crate::session_factory::{SessionFactory, WorkerSessionFactory};
11
12 pub struct SessionJobRunner {
13 @@ -42,6 +43,7 @@ impl SessionJobRunner {
14 }
15
16 pub struct SessionJobRunnerInfo {
17 +     pub resource_domain: Option<MemoryResourceDomain>,

```

```

18     pub session_id: String,
19     pub driver_id: DriverId,
20     pub driver_server_port: Option<u16>,
21 @@ -103,8 +105,16 @@ impl SessionJobRunnerFactory for ServerSessionJobRunnerFactory {
22         info.session_id,
23     )),
24     ExecutionMode::LocalCluster => {
25 +         if std::env::var("SAIL_EXPERIMENTAL_PROCESS_WORKERS").as_deref() == 0
26 +         k("1") {
27 +             return self.create_cluster_runner(
28 +                 system,
29 +                 info,
30 +                 Box::new(sail_execution::worker_manager::ProcessWorkerManager
31 + ::default()),
32 +             );
33 +         }
34 +         let worker_session =
35 +             WorkerSessionFactory::new(self.config.clone(), self.runtime.clone
36 + ())
37 +             .with_resource_domain(info.resource_domain.clone())
38 +             .create()?;
39         self.create_cluster_runner(
40             system,

```

59. crates/sail-session/src/session_factory/server.rs

```

1 diff --git a/crates/sail-session/src/session_factory/server.rs b/crates/sail-session/s
2 rc/session_factory/server.rs
3 index 2828250fd36a6115dade54f9d70f177ea559722e..c187fd8f023d518937ccde24067ded62020907
4 b7 100644
5 --- a/crates/sail-session/src/session_factory/server.rs
6 +++ b/crates/sail-session/src/session_factory/server.rs
7 @@ -26,11 +26,12 @@ use crate::catalog::create_catalog_manager;
8 use crate::formats::create_data_source_registry;
9 use crate::optimizer::{default_analyzer_rules, default_optimizer_rules};
10 use crate::planner::new_query_planner;
11 -use crate::runtime::RuntimeEnvFactory;
12 +use crate::runtime::{MemoryResourceDomain, RuntimeEnvFactory};
13 use crate::session_factory::SessionFactory;
14 use crate::session_manager::SessionManagerActor;
15
16 pub struct ServerSessionInfo {
17 + pub resource_domain: Option<MemoryResourceDomain>,
18     pub session_id: String,
19     pub user_id: String,
20     pub session_manager: ActorHandle<SessionManagerActor>,
21 @@ -130,15 +131,17 @@ impl ServerSessionFactory {
22     self.apply_execution_config(&mut config)?;
23     self.apply_execution_parquet_config(&mut config)?;
24     self.apply_optimizer_config(&mut config)?;
25 - let config = self.mutator.mutate_config(config, info)?;
26 - Ok(config)
27 + self.mutator.mutate_config(config, info)
28 }
29
30 fn create_session_state(&mut self, info: &mut ServerSessionInfo) -> Result<Sessio
31 nState> {
32     let config = self.create_session_config(info)?;
33     let runtime = self
34         .runtime_env
35         .create(|builder| self.mutator.mutate_runtime_env(builder, info))?;
36 - .create(info.resource_domain.as_ref(), |builder| {
37 +     self.mutator.mutate_runtime_env(builder, info)
38 + });
39 + let config = crate::extensions::register_extensions(config, &self.config.mode
40 , &runtime)?;
41 // We do not add default features to the session state,
42// since we manage data sources and functions ourselves.
43 let builder = SessionStateBuilder::new()

```

60. crates/sail-session/src/session_factory/worker.rs

```
1 diff --git a/crates/sail-session/src/session_factory/worker.rs b/crates/sail-session/s
rc/session_factory/worker.rs
2 index 9ad40ec869dd566bcf6565f70481b3a2c26a877f..311cd38d61fc379e85ff7062519bebb84a30d7
17 100644
3 --- a/crates/sail-session/src/session_factory/worker.rs
4 +++ b/crates/sail-session/src/session_factory/worker.rs
5 @@ -9,10 +9,11 @@ use sail_common::runtime::RuntimeHandle;
6 use sail_common_datafusion::session::repartition::RepartitionBufferConfig;
7 use sail_delta_take::session_extension::DeltaTableCache;
8
9 -use crate::runtime::RuntimeEnvFactory;
10 +use crate::runtime::{MemoryResourceDomain, RuntimeEnvFactory};
11 use crate::session_factory::SessionFactory;
12
13 pub struct WorkerSessionFactory {
14 + resource_domain: Option<MemoryResourceDomain>,
15 runtime_env: RuntimeEnvFactory,
16 batch_size: usize,
17 repartition_buffer_size: usize,
18 @@ -22,18 +23,28 @@ impl WorkerSessionFactory {
19 pub fn new(config: Arc<AppConfig>, runtime: RuntimeHandle) -> Self {
20 let batch_size = config.execution.batch_size;
21 let repartition_buffer_size = config.cluster.task_stream_buffer;
22 + let resource_domain = (std::env::var("SAIL_EXPERIMENTAL_EXTENSIONS").as_deref
23 () == Ok("1"))
24 + .then(|| MemoryResourceDomain::new(&config.runtime.memory_pool));
25 let runtime_env = RuntimeEnvFactory::new(config, runtime.clone());
26 Self {
27 + resource_domain,
28 runtime_env,
29 batch_size,
30 repartition_buffer_size,
31 }
32 + pub fn with_resource_domain(mut self, domain: Option<MemoryResourceDomain>) -> Se
33 lf {
34 + self.resource_domain = domain;
35 self
36 }
37
38 impl SessionFactory<> for WorkerSessionFactory {
39 fn create(&mut self, info: ()) -> Result<SessionContext> {
40 let runtime = self.runtime_env.create(Ok)?;
41 + if std::env::var("SAIL_EXPERIMENTAL_EXTENSIONS").as_deref() == Ok("1") {
42 + crate::extensions::load_worker_extensions()?;
43 + }
44 + let runtime = self.runtime_env.create(self.resource_domain.as_ref(), Ok)?;
45 // We still add default features for the worker session
46 // since we need built-in functions to be available for the codec
47 // when decoding the execution plan.
```

61. crates/sail-session/src/session_manager/actor/core.rs

```
1 diff --git a/crates/sail-session/src/session_manager/actor/core.rs b/crates/sail-sessi
on/src/session_manager/actor/core.rs
2 index f3fb136c8a15130426467877c34b273f99c7e264..85c12e1429dca1b2d3ce04ae025cdc75c3a207
ef 100644
3 --- a/crates/sail-session/src/session_manager/actor/core.rs
4 +++ b/crates/sail-session/src/session_manager/actor/core.rs
5 @@ -3,12 +3,16 @@ use std::sync::Arc;
6 use indexmap::IndexMap;
7 use log::{info, warn};
8 use sail_common::actor::{Actor, ActorAction, ActorContext, ActorHandle};
9 +use sail_common_datafusion::extension::SessionExtensionAccessor;
10 +use sail_common_datafusion::native_resource::NativeResourceTracker;
11 +use sail_common_datafusion::session::lifecycle::SessionLifecycle;
12 use sail_execution::driver::{DriverHandle, DriverRegistryAccessor};
13 use sail_execution::error::{ExecutionError, ExecutionResult};
14 use sail_execution::{DriverId, IdGenerator};
15 use sail_system_store::SystemEvent;
16
17 use crate::session_manager::actor::SessionManagerActor;
```

```

18 +use crate::session_manager::session::ServerSessionState;
19 use crate::session_manager::{
20     SessionManagerComponents, SessionManagerMessage, SessionManagerOptions,
21 };
22 @@ -53,6 +57,7 @@ impl Actor for SessionManagerActor {
23     session_factory,
24     job_runner_factory,
25     sessions: IndexMap::new(),
26 +    cleanup: Default::default(),
27     drivers: Default::default(),
28     driver_gateway,
29     driver_id_generator: IdGenerator::new(),
30 @@ -118,6 +123,26 @@ impl Actor for SessionManagerActor {
31 }
32
33 async fn stop(mut self, ctx: &mut ActorContext<Self>) {
34 +    let native_resources = self
35 +        .sessions
36 +        .values()
37 +        .filter_map(|session| {
38 +            let ServerSessionState::Running { context, .. } = &session.state else
39 +            {
40 +                return None;
41 +            };
42 +            context.extension:::<NativeResourceTracker>().ok()
43 +        })
44 +        .collect:::<Vec<_>>();
45 +    // A local job has no driver to stop its client-owned stream. Protocol
46 +    // operations must terminate before contexts are dropped or gRPC drains.
47 +    for (session_id, session) in &self.sessions {
48 +        if let ServerSessionState::Running { context, .. } = &session.state
49 +        && let Ok(lifecycle) = context.extension:::<SessionLifecycle>()
50 +        && let Err(error) = lifecycle.stop().await
51 +        {
52 +            warn!("failed to stop session resources for {session_id}: {error}");
53 +        }
54 +    }
55 +    // Keep the gateway available while drivers stop. Graceful gateway shutdown w
56 +    // active task stream connections, which are owned by the drivers.
57 +    let drivers = self.drivers.drain().collect:::<Vec<_>>();
58 @@ -127,6 +152,17 @@ impl Actor for SessionManagerActor {
59 }
60 ctx.children_mut().join().await;
61 self.sessions.clear();
62 +    // Stream cancellation signals detached native producers; their final
63 +    // buffers may be released afterwards. Do not let process exit race that
64 +    // release or silently reclaim their quota through OS teardown.
65 +    for resources in native_resources {
66 +        resources.wait_for_release().await;
67 +    }
68 +    // A Deleted session has already left self.sessions' live contexts, but
69 +    // its final native producer may still be releasing retained outputs.
70 +    // Unlike idle-probe timers, this work must finish before actor teardown.
71 +    self.cleanup.finish().await;
72 +    if let Some(mut driver_gateway) = self.driver_gateway {
73 +        driver_gateway.stop().await;
74 +        info!("driver server has stopped");

```

62. crates/sail-session/src/session_manager/actor/handler.rs

```

1 diff --git a/crates/sail-session/src/session_manager/actor/handler.rs b/crates/sail-s
2 ssion/src/session_manager/actor/handler.rs
3 index 2061e7c874a0a4814e102d02b23d03cf25b74f44..68c4b8dfcd4e512542a07a2fc84fd091b9f098
4 c3 100644
5 --- a/crates/sail-session/src/session_manager/actor/handler.rs
6 +++ b/crates/sail-session/src/session_manager/actor/handler.rs
7 @@ -7,8 +7,10 @@ use sail_cache::remote_checkpoint::RemoteCheckpointRegistry;
8 use sail_common::actor::{ActorAction, ActorContext};
9 use sail_common::telemetry::SpanAttribute;
10 use sail_common_datafusion::extension::SessionExtensionAccessor;
11 +use sail_common_datafusion::native_resource::NativeResourceTracker;
12 use sail_common_datafusion::session::activity::ActivityTracker;
13 use sail_common_datafusion::session::job::JobService;

```

```

12 +use sail_common_datafusion::session::lifecycle::SessionLifecycle;
13 use sail_execution::DriverId;
14 use sail_execution::driver::DriverHandle;
15 use sail_execution::error::ExecutionResult;
16 @@ -20,6 +22,7 @@ use crate::error::{SessionError, SessionResult};
17 use crate::session_factory::{ServerSessionInfo, SessionJobRunnerInfo};
18 use crate::session_manager::SessionManagerMessage;
19 use crate::session_manager::actor::SessionManagerActor;
20 +use crate::session_manager::cleanup::SessionCleanup;
21 use crate::session_manager::session::{ServerSession, ServerSessionState};
22
23 impl SessionManagerActor {
24 @@ -80,6 +83,7 @@ impl SessionManagerActor {
25     let runner = match self.job_runner_factory.create(
26         ctx.children_mut(),
27         SessionJobRunnerInfo {
28 +         resource_domain: self.options.resource_domain.clone(),
29         session_id: session_id.clone(),
30         driver_id,
31         driver_server_port: self.driver_gateway.as_ref().map(|x| x.port()),
32 @@ -118,6 +122,7 @@ impl SessionManagerActor {
33     return ActorAction::Continue;
34 }
35 let info = ServerSessionInfo {
36 + resource_domain: self.options.resource_domain.clone(),
37 session_id: session_id.clone(),
38 user_id: user_id.clone(),
39 session_manager: ctx.handle().clone(),
40 @@ -268,7 +273,7 @@ impl SessionManagerActor {
41
42 pub(super) fn handle_probe_idle_session(
43     &mut self,
44 - ctx: &mut ActorContext<Self>,
45 + _ctx: &mut ActorContext<Self>,
46 session_id: String,
47 instant: Instant,
48 ) -> ActorAction {
49 @@ -279,7 +284,7 @@ impl SessionManagerActor {
50     && tracker.active_at().is_ok_and(|x| x <= instant)
51     {
52         info!("removing idle session {session_id}");
53 - Self::delete_session(ctx, session_id.clone(), context);
54 + Self::delete_session(&mut self.cleanup, session_id.clone(), context);
55         if let Some(driver_id) = *driver_id {
56             self.drivers.remove(driver_id);
57         }
58 @@ -296,7 +301,7 @@ impl SessionManagerActor {
59
60 pub(super) fn handle_delete_session(
61     &mut self,
62 - ctx: &mut ActorContext<Self>,
63 + _ctx: &mut ActorContext<Self>,
64 session_id: String,
65 result: oneshot::Sender<SessionResult<>>>,
66 ) -> ActorAction {
67 @@ -304,7 +309,7 @@ impl SessionManagerActor {
68     let output = if let Some(session) = session {
69         if let ServerSessionState::Running { context, driver_id } = &mut session.
70 state {
71             info!("removing session {session_id}");
72 - Self::delete_session(ctx, session_id.clone(), context);
73 + Self::delete_session(&mut self.cleanup, session_id.clone(), context);
74             if let Some(driver_id) = *driver_id {
75                 self.drivers.remove(driver_id);
76             }
77 @@ -370,16 +375,32 @@ impl SessionManagerActor {
78     ActorAction::Continue
79 }
80 - fn delete_session(ctx: &mut ActorContext<Self>, session_id: String, context: &Ses
81 sionContext) {
82 + fn delete_session(cleanup: &mut SessionCleanup, session_id: String, context: &Ses
83 sionContext) {
84     let Ok(service) = context.extension:::<JobService>() else {
85         warn!("job service not found for session {session_id}");
86         return;
87     };

```

```

86         let checkpoint_registry = context.extension::<RemoteCheckpointRegistry>().ok(
87     );
88     + let lifecycle = context.extension::<SessionLifecycle>().ok();
89     + let native_resources = context.extension::<NativeResourceTracker>().ok();
90     + let graph_runs = context
91     +     .extension::<crate::extensions::graph_utils::GraphRuns>()
92     +     .ok();
93     let runtime_env = context.runtime_env();
94     - ctx.spawn(async move {
95     -     // Stop tasks before deleting the namespace so late attempts cannot recre
96     ate objects.
97     + cleanup.spawn(async move {
98     +     if let Some(lifecycle) = lifecycle
99     +     && let Err(error) = lifecycle.stop().await
100    +     {
101    +         warn!("failed to stop session resources for {session_id}: {error}");
102    +     }
103    +     // Request executor/job shutdown before cleanup. Detached data-source
104    +     // writers are not all joined here, so storage cleanup is best effort.
105    +     service.runner().stop().await;
106    +     if let Some(graph_runs) = graph_runs
107    +     && let Err(error) = graph_runs.cleanup().await
108    +     {
109    +         warn!("failed to clean graph runs for session {session_id}: {error}")
110    +     };
111    +     }
112    +     if let Some(checkpoint_registry) = checkpoint_registry
113    +     && let Err(error) = checkpoint_registry
114    +     .cleanup_session(runtime_env.as_ref())
115    +     @@ -387,6 +408,9 @@ impl SessionManagerActor {
116    +     {
117    +         warn!("failed to clean checkpoints for session {session_id}: {error}"
118    +     );
119    +     }
120    +     if let Some(resources) = native_resources {
121    +         resources.wait_for_release().await;
122    +     }
123    +     });
124    }
125    }

```

63. crates/sail-session/src/session_manager/actor/mod.rs

```

1  diff --git a/crates/sail-session/src/session_manager/actor/mod.rs b/crates/sail-sessio
2  n/src/session_manager/actor/mod.rs
3  index 9061fc33ca14da714d6ce9e0497f69d04150488b..ec75defa80b6162a501a7533e241754d4fddac
4  3c 100644
5  --- a/crates/sail-session/src/session_manager/actor/mod.rs
6  +++ b/crates/sail-session/src/session_manager/actor/mod.rs
7  @@ -11,6 +11,7 @@ use sail_execution::{DriverId, IdGenerator};
8  use sail_telemetry::events::SystemEventReporter;
9  use crate::session_factory::{ServerSessionInfo, SessionFactory, SessionJobRunnerFacto
10 ry};
11 +use crate::session_manager::cleanup::SessionCleanup;
12 use crate::session_manager::session::ServerSession;
13
14 pub struct SessionManagerActor {
15     @@ -18,6 +19,7 @@ pub struct SessionManagerActor {
16     session_factory: Box<dyn SessionFactory<ServerSessionInfo>>,
17     job_runner_factory: Box<dyn SessionJobRunnerFactory>,
18     sessions: IndexMap<String, ServerSession>,
19     + cleanup: SessionCleanup,
20     drivers: DriverRegistry,
21     driver_gateway: Option<DriverGateway>,
22     driver_id_generator: IdGenerator<DriverId>,

```

64. crates/sail-session/src/session_manager/actor/options.rs

```

1  diff --git a/crates/sail-session/src/session_manager/actor/options.rs b/crates/sail-se
2  ssion/src/session_manager/actor/options.rs
3  index 5956d741b427a32820dba4c34b6a3ae148cd4e1e..44aa034ee76d61c67a487cfd40ab090de5980f
4  72 100644

```

```

3 --- a/crates/sail-session/src/session_manager/actor/options.rs
4 +++ b/crates/sail-session/src/session_manager/actor/options.rs
5 @@ -4,10 +4,12 @@ use sail_common::runtime::RuntimeHandle;
6 use sail_execution::driver::DriverGateway;
7 use sail_telemetry::events::SystemEventReporter;
8
9 +use crate::runtime::MemoryResourceDomain;
10 use crate::session_factory::{ServerSessionInfo, SessionFactory, SessionJobRunnerFacto
    ry};
11
12 #[readonly::make]
13 pub struct SessionManagerOptions {
14 + pub resource_domain: Option<MemoryResourceDomain>,
15 pub session_timeout: Duration,
16 pub runtime: RuntimeHandle,
17 /// The application configuration options as key-value pairs,
18 @@ -25,12 +27,18 @@ pub struct SessionManagerComponents {
19 impl SessionManagerOptions {
20 pub fn new(runtime: RuntimeHandle) -> Self {
21     Self {
22 + resource_domain: None,
23 session_timeout: Duration::MAX,
24 runtime,
25 options: Vec::new(),
26     }
27 }
28
29 + pub fn with_resource_domain(mut self, domain: MemoryResourceDomain) -> Self {
30 + self.resource_domain = Some(domain);
31 + self
32 + }
33 +
34 pub fn with_session_timeout(mut self, timeout: Duration) -> Self {
35     self.session_timeout = timeout;
36     self

```

65. crates/sail-session/src/session_manager/cleanup.rs

```

1 diff --git a/crates/sail-session/src/session_manager/cleanup.rs b/crates/sail-session/
  src/session_manager/cleanup.rs
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..1aba8815d832cd5cd1f7ef77f64834622a1df0
  e5
4 --- /dev/null
5 +++ b/crates/sail-session/src/session_manager/cleanup.rs
6 @@ -0,0 +1,85 @@
7 +use std::future::Future;
8 +
9 +use log::warn;
10 +use tokio::task::JoinSet;
11 +
12 +/// Session deletion is acknowledged before its asynchronous cleanup completes.
13 +/// Keep those tasks separate from abortable actor timers, and drain them before
14 +/// graceful shutdown releases the actor and its already-deleted sessions.
15 +#[derive(Default)]
16 +pub(super) struct SessionCleanup {
17 +    tasks: JoinSet<()>,
18 +}
19 +
20 +impl SessionCleanup {
21 +    pub(super) fn spawn(&mut self, task: impl Future<Output = ()> + Send + 'static) {
22 +        while let Some(result) = self.tasks.try_join_next() {
23 +            Self::report(result);
24 +        }
25 +        self.tasks.spawn(task);
26 +    }
27 +
28 +    pub(super) async fn finish(&mut self) {
29 +        while let Some(result) = self.tasks.join_next().await {
30 +            Self::report(result);
31 +        }
32 +    }
33 +
34 +    fn report(result: Result<(), tokio::task::JoinError>) {
35 +        if let Err(error) = result {

```

```

36 +         warn!("session cleanup task failed: {error}");
37 +     }
38 + }
39 +}
40 +
41 +#[cfg(test)]
42 +mod tests {
43 +    use std::sync::Arc;
44 +    use std::sync::atomic::{AtomicBool, Ordering};
45 +
46 +    use datafusion::execution::memory_pool::{GreedyMemoryPool, MemoryPool};
47 +    use sail_common_datafusion::native_resource::NativeResourceTracker;
48 +    use tokio::sync::oneshot;
49 +
50 +    use super::*;
51 +
52 +    #[tokio::test]
53 +    async fn immediate_shutdown_waits_for_held_cleanup_and_final_native_release()
54 +    -> Result<(), Box<dyn std::error::Error> {
55 +        let pool: Arc<dyn MemoryPool> = Arc::new(GreedyMemoryPool::new(64));
56 +        let tracker = Arc::new(NativeResourceTracker::default());
57 +        let output = tracker.reserve(&pool, "deleted-session-output", 64)?;
58 +        let completed = Arc::new(AtomicBool::new(false));
59 +        let (entered, entered_rx) = oneshot::channel();
60 +        let (released, released_rx) = oneshot::channel();
61 +        let producer = std::thread::spawn(move || -> Result<(), oneshot::error::RecvE
62 +    rror> {
63 +        // Hold the last output independently of the Tokio scheduler. Its
64 +        // admission cannot disappear because an actor dropped its tasks.
65 +        released_rx.blocking_recv()?;
66 +        drop(output);
67 +        Ok(())
68 +    });
69 +    let mut cleanup = SessionCleanup::default();
70 +    let ended = completed.clone();
71 +    cleanup.spawn(async move {
72 +        assert!(entered.send(()).is_ok());
73 +        tracker.wait_for_release().await;
74 +        ended.store(true, Ordering::SeqCst);
75 +    });
76 +    entered_rx.await?;
77 +    let mut shutdown = Box::pin(cleanup.finish());
78 +    assert!(futures::poll!(shutdown.as_mut()).is_pending());
79 +    assert_eq!(pool.reserved(), 64);
80 +    assert!(!completed.load(Ordering::SeqCst));
81 +    released
82 +        .send(())
83 +        .map_err(|_| "native producer exited before release")?;
84 +    shutdown.await;
85 +    producer
86 +        .join()
87 +        .map_err(|_| "native producer thread panicked")??;
88 +    assert!(completed.load(Ordering::SeqCst));
89 +    assert_eq!(pool.reserved(), 0);
90 +    Ok(())
91 +}

```

66. crates/sail-session/src/session_manager/mod.rs

```

1 diff --git a/crates/sail-session/src/session_manager/mod.rs b/crates/sail-session/src/
2 session_manager/mod.rs
3 index 0ccaleee1a30ec0aae28fd112fc0f634055a1390..b6ce85481ed07878d131f65252ec10b40015bb
4 45 100644
5 --- a/crates/sail-session/src/session_manager/mod.rs
6 +++ b/crates/sail-session/src/session_manager/mod.rs
7 @@ -1,4 +1,5 @@
8 mod actor;
9 +mod cleanup;
10 mod session;
11
12 use std::fmt;
13 @@ -114,6 +115,13 @@ pub async fn create_session_manager(
14     .raw()
15     .map_err(|e| SessionError::internal(e.to_string()))?,

```

```

14         );
15 +     let options = if std::env::var("SAIL_EXPERIMENTAL_EXTENSIONS").as_deref() == Ok("
16 1") {
17 +         options.with_resource_domain(crate::runtime::MemoryResourceDomain::new(
18 +             &config.runtime.memory_pool,
19 +         ))
20 +     } else {
21 +         options
22 +     };
23     let components = SessionManagerComponents {
24         session_factory,
25         job_runner_factory,

```

67. crates/sail-spark-connect/Cargo.toml

```
1 diff --git a/crates/sail-spark-connect/Cargo.toml b/crates/sail-spark-connect/Cargo.to
2 ml
3 index 2b288f5ea39fbb38338379a6a1e055088a6eb5cc..0b79022cbb0a11582dfbaf3477728dc600703
4 f5 100644
5 --- a/crates/sail-spark-connect/Cargo.toml
6 +++ b/crates/sail-spark-connect/Cargo.toml
7 @@ -7,6 +7,7 @@ edition = { workspace = true }
8   workspace = true
9
10  [dependencies]
11 +stacker = { workspace = true }
12   sail-cache = { path = "../sail-cache" }
13   sail-common = { path = "../sail-common" }
14   sail-common-datafusion = { path = "../sail-common-datafusion" }
```

68. [crates/sail-spark-connect/proto/sail/extension/v1/extension.proto](https://crates.io/crates/sail-spark-connect/proto/sail/extension/v1/extension.proto)

```

1 diff --git a/crates/sail-spark-connect/proto/sail/extension.v1/extension.proto b/crate
s/sail-spark-connect/proto/sail/extension.v1/extension.proto
2 new file mode 100644
3 index 0000000000000000000000000000000000..86db5179a4a70c68fa80483e5abbb1f309ec8e
0c
4 --- /dev/null
5 +++ b/crates/sail-spark-connect/proto/sail/extension.v1/extension.proto
6 @@ -0,0 +1,20 @@
7 +// Experimental local-mode protocol. This schema is a wire contract, not a native ABI
8 .
9 +syntax = "proto3";
10 +package sail.extension.v1;
11 +
12 +import "spark/connect/base.proto";
13 +import "spark/connect/expressions.proto";
14 +
15 +// Pack into Relation.extension with type URL:
16 +// type.googleapis.com/sail.extension.v1.SailExtensionRequest
17 +message SailExtensionRequest {
18 +    string payload_type_url = 1;
19 +    bytes payload = 2;
20 +    // Only Plan.root is accepted. Names are restored to the input DataFrame's
21 +    // user-facing names before the native handler receives a physical plan.
22 +    repeated spark.connect.Plan inputs = 3;
23 +    // Reserved by the proposal; this PoC rejects any occurrence of this field.
24 +    repeated spark.connect.Expression input_expressions = 4;
25 +    // Required value: 1. An omitted proto3_value (0) is rejected.
26 +    uint32 envelope_version = 5;
27 +}

```

69. [crates/sail-spark-connect/src/entrypoint.rs](https://github.com/crates-io/sail-spark-connect/blob/main/src/entrypoint.rs)

```
1 diff --git a/crates/sail-spark-connect/src/entrypoint.rs b/crates/sail-spark-connect/s
rc/entrypoint.rs
2 index b95cb4dfce456921f362fa51260e9f3f342cff04..4d8fa4187ed492c9805c2593f1f3f22e460a50
68 100644
3 --- a/crates/sail-spark-connect/src/entrypoint.rs
4 +++ b/crates/sail-spark-connect/src/entrypoint.rs
```

```

5 @@ -50,6 +50,7 @@ where
6     let session_manager =
7         create_spark_session_manager_with_factory(config, runtime, &mut system, sessi
on_factory_fn)
8         .await?;
9 +     let mut shutdown_result = None;
10     let result = {
11         let server = SparkConnectServer::new(session_manager.clone());
12         let service = SparkConnectServiceServer::new(server)
13 @@ -63,11 +64,20 @@ where
14         ServerBuilder::new("sail_spark_connect", Default::default())
15             .add_service(service, Some(crate::spark::connect::FILE_DESCRIPTOR_SET))
16             .await
17 -         .serve(listener, signal)
18 +         .serve(listener, async {
19 +             signal.await;
20 +             // Tonic's graceful drain waits for active responses. Stop their
21 +             // session-owned producers first so unbounded work cannot keep
22 +             // the server and native reservations alive during shutdown.
23 +             shutdown_result = Some(session_manager.shutdown().await);
24 +         })
25         .await
26         .map_err(|e| std::io::Error::other(e.to_string()));
27     };
28 - session_manager.shutdown().await?;
29 + match shutdown_result {
30 +     Some(result) => result?,
31 +     None => session_manager.shutdown().await?,
32 + }
33 system.join().await;
34 result.map_err(Into::into)
35 }

```

70. crates/sail-spark-connect/src/error.rs

```

1 diff --git a/crates/sail-spark-connect/src/error.rs b/crates/sail-spark-connect/src/er
ror.rs
2 index acfe4e8737530758f1f9d27f9b9bb922cf9bd498..955776f93976032484a4e5bc8cc343e86b5ed2
9b 100644
3 --- a/crates/sail-spark-connect/src/error.rs
4 +++ b/crates/sail-spark-connect/src/error.rs
5 @@ -49,6 +49,8 @@ pub enum SparkError {
6     AnalysisError(String),
7     #[error("parse error: {0}")]
8     ParseError(String),
9 +     #[error("operation interrupted: {0}")]
10 +     OperationInterrupted(String),
11 }
12
13 impl SparkError {
14 @@ -460,6 +462,9 @@ impl From<SparkError> for Status {
15     }
16     SparkError::AnalysisError(s) => SparkThrowable::AnalysisException(s).into
17     (),
18     SparkError::ParseError(s) => SparkThrowable::ParseException(s).into(),
19 +     e @ SparkError::OperationInterrupted(_) => {
20 +         SparkThrowable::QueryExecutionException(e.to_string()).into()
21 +     }
22 +     e @ SparkError::SendError(_) => {
23         Status::cancelled(truncate_grpc_message(&e.to_string()))
24     }
25 }

```

71. crates/sail-spark-connect/src/executor.rs

```

1 diff --git a/crates/sail-spark-connect/src/executor.rs b/crates/sail-spark-connect/src
/executor.rs
2 index 0ebc2ad21b6394f862abaaeeb76b9d74d235c1e3..82fb2debf15da7c8027252d623cd75bc0c19e0
bb 100644
3 --- a/crates/sail-spark-connect/src/executor.rs
4 +++ b/crates/sail-spark-connect/src/executor.rs
5 @@ -122,6 +122,9 @@ enum ExecutorState {
6     span: Span,
7 },

```

```

8     Pausing,
9 +    // Retain only the terminal identity so a reattaching client receives the
10 +    // interruption, without retaining the plan, native buffers, or task context.
11 +    Interrupted,
12     Failed(SparkError),
13 }
14
15 @@ -401,6 +404,12 @@ impl Executor {
16     *state = x;
17     return Err(SparkError::internal("task is being paused"));
18 }
19 +    ExecutorState::Interrupted => {
20 +        *state = ExecutorState::Interrupted;
21 +        return Err(SparkError::OperationInterrupted(
22 +            self.metadata.operation_id.clone(),
23 +        ));
24 +    }
25 };
26 let (tx, rx) = mpsc::channel(1);
27 let (notifier, listener) = oneshot::channel();
28 @@ -445,16 +454,42 @@ impl Executor {
29     ExecutorTaskResult::Completed => ExecutorState::Idle,
30     ExecutorTaskResult::Failed(e) => ExecutorState::Failed(e),
31 };
32 -    *(self.state.lock()) = state;
33 +    let mut current = self.state.lock();
34 +    // An interrupt may have terminalized the operation while the paused
35 +    // task was being joined. Never restore that task's context afterwards.
36 +    if matches!(*current, ExecutorState::Pausing) {
37 +        *current = state;
38 +    }
39     Ok(())
40 }
41
42 +    pub(crate) async fn interrupt(&self) -> SparkResult<bool> {
43 +        let previous = {
44 +            let mut state = self.state.lock();
45 +            mem::replace(state.deref_mut(), ExecutorState::Interrupted)
46 +        };
47 +        match previous {
48 +            ExecutorState::Interrupted => Ok(false),
49 +            ExecutorState::Running { task, .. } => {
50 +                let _ = task.notifier.send();
51 +                // Drop the returned task context before acknowledging. Native
52 +                // blocking producers observe their stream cancellation next.
53 +                drop(task.handle.await?);
54 +                Ok(true)
55 +            }
56 +            _ => Ok(true),
57 +        }
58 +    }
59 +
60     pub(crate) fn release(&self, response_id: String) -> SparkResult<()> {
61         let state = self.state.lock();
62         let buffer = match state.deref() {
63             ExecutorState::Running { task, span: _ } => &task.buffer,
64             ExecutorState::Pending { context, span: _ } => &context.buffer,
65             ExecutorState::Idle | ExecutorState::Failed(_) | ExecutorState::Pausing =
66 > {
67 +                ExecutorState::Idle
68 +                | ExecutorState::Failed(_)
69 +                | ExecutorState::Pausing
70 +                | ExecutorState::Interrupted => {
71                     return Ok(());
72                 }
73             };
74         }
75     }
76
77 +#[cfg(test)]
78 +mod tests;
79 +
80     pub(crate) fn to_arrow_batch(batch: &RecordBatch) -> SparkResult<ArrowBatch> {
81         let mut output = ArrowBatch::default();
82         {

```

72. crates/sail-spark-connect/src/executor/tests.rs

```
1 diff --git a/crates/sail-spark-connect/src/executor/tests.rs b/crates/sail-spark-connect/src/executor/tests.rs
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..fe8c666df2b81e3267bb9563cb3041832c81b2f9
4 --- /dev/null
5 +++ b/crates/sail-spark-connect/src/executor/tests.rs
6 @@ -0,0 +1,126 @@
7 +use std::sync::atomic::{AtomicUsize, Ordering};
8 +use std::task::{Context, Poll};
9 +
10 +use datafusion::arrow::datatypes::Schema;
11 +use datafusion::execution::RecordBatchStream;
12 +use sail_common_datafusion::session::lifecycle::SessionResource;
13 +
14 +use super::*;
15 +use crate::session::{SparkSession, SparkSessionOptions};
16 +
17 +struct PendingStream(Arc<AtomicUsize>);
18 +
19 +impl Stream for PendingStream {
20 +    type Item = datafusion::common::Result<RecordBatch>;
21 +
22 +    fn poll_next(self: Pin<&mut Self>, _: &mut Context<'_>) -> Poll<Option<Self::Item>> {
23 +        Poll::Pending
24 +    }
25 +}
26 +
27 +impl RecordBatchStream for PendingStream {
28 +    fn schema(&self) -> SchemaRef {
29 +        Arc::new(Schema::empty())
30 +    }
31 +}
32 +
33 +impl Drop for PendingStream {
34 +    fn drop(&mut self) {
35 +        self.0.fetch_add(1, Ordering::SeqCst);
36 +    }
37 +}
38 +
39 +fn executor() -> (Executor, Arc<AtomicUsize>) {
40 +    let dropped = Arc::new(AtomicUsize::new(0));
41 +    let executor = Executor::new(
42 +        ExecutorMetadata {
43 +            operation_id: "interrupted-operation".into(),
44 +            tags: vec![],
45 +            reattachable: true,
46 +        },
47 +        Box::pin(PendingStream(Arc::clone(&dropped))),
48 +        Duration::from_secs(3600),
49 +        ExecutorMode::Query,
50 +    );
51 +    (executor, dropped)
52 +}
53 +
54 +fn assert_terminal(executor: &Executor, dropped: &AtomicUsize) {
55 +    assert_eq!(dropped.load(Ordering::SeqCst), 1);
56 +    // Repeated reattach cannot consume the error and accidentally restart work.
57 +    for _ in 0..2 {
58 +        assert!(matches!(
59 +            executor.start(),
60 +            Err(SparkError::OperationInterrupted(id)) if id == "interrupted-operation"
61 +        ));
62 +    }
63 +}
64 +
65 +#[tokio::test]
66 +async fn interrupt_drops_pending_and_running_contexts_but_preserves_terminal_identity() {
67 +    SparkResult<> {
68 +        for started in [false, true] {
69 +            let (executor, dropped) = executor();
70 +            let mut output = if started {
71 +                Some(executor.start())?
```

```

72 +         } else {
73 +             None
74 +         };
75 +         if let Some(output) = &mut output {
76 +             assert!(matches!(
77 +                 output.next().await,
78 +                 Some(Ok(ExecutorOutput {
79 +                     batch: ExecutorBatch::Schema(_),
80 +                     ..
81 +                 })))
82 +             );
83 +         }
84 +         assert!(executor.interrupt().await?);
85 +         assert!(!executor.interrupt().await?);
86 +         assert_terminal(&executor, &dropped);
87 +     }
88 +     Ok(())
89 + }
90 +
91 + #[tokio::test]
92 + async fn concurrent_pause_cannot_restore_an_interrupted_context() -> SparkResult<()>
93 + {
94 +     let (executor, dropped) = executor();
95 +     let mut output = executor.start()?;
96 +     assert!(output.next().await.is_some());
97 +     let mut pause = Box::pin(executor.pause_if_running());
98 +     // This current-thread runtime cannot join the executor task until yielding.
99 +     // Hold the exact Pausing state without a sleep or a clock race.
100 +    assert!(futures::poll!(pause.as_mut()).is_pending());
101 +    assert!(executor.interrupt().await?);
102 +    pause.await?;
103 +    assert_terminal(&executor, &dropped);
104 +    Ok(())
105 + }
106 +
107 + #[tokio::test]
108 + async fn session_stop_drops_owned_plans_and_rejects_in_flight_planning_completion()
109 + -> SparkResult<()> {
110 +     pyo3::Python::initialize();
111 +     let session = SparkSession::try_new(
112 +         "teardown-session".into(),
113 +         "test".into(),
114 +         SparkSessionOptions {
115 +             execution_heartbeat_interval: Duration::from_secs(3600),
116 +         },
117 +     )?;
118 +     let (pending, dropped) = executor();
119 +     session.add_executor(pending)?;
120 +     session.stop().await?;
121 +     assert_eq!(dropped.load(Ordering::SeqCst), 1);
122 +     assert!(session.get_executor("interrupted-operation").is_none());
123 +     // A request may have begun planning before teardown. Its completed plan
124 +     // must be dropped instead of resurrecting operations in the deleted session.
125 +     let (late, dropped) = executor();
126 +     assert!(matches!(
127 +         session.add_executor(late),
128 +         Err(SparkError::OperationInterrupted(_))
129 +     ));
130 +     assert_eq!(dropped.load(Ordering::SeqCst), 1);
131 +     session.stop().await?;
132 +     Ok(())
133 + }

```

73. crates/sail-spark-connect/src/proto/extension.rs

```

1 diff --git a/crates/sail-spark-connect/src/proto/extension.rs b/crates/sail-spark-connect/src/proto/extension.rs
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..99ac3123341ceca22f7f951e929dd77059e9e1bc
4 --- /dev/null
5 +++ b/crates/sail-spark-connect/src/proto/extension.rs
6 @@ -0,0 +1,173 @@
7 +/// Wire decoding for the experimental local Sail relation extension envelope.
8 +

```

```

9 +use std::cell::Cell;
10 +
11 +use pbjson_types::Any;
12 +use prost::Message;
13 +use sail_common::spec;
14 +
15 +use crate::error::{SparkError, SparkResult};
16 +use crate::spark::connect::Plan;
17 +
18 +mod wire;
19 +
20 +pub(crate) const ENVELOPE_TYPE_URL: &str =
21 +    "type.googleapis.com/sail.extension.v1.SailExtensionRequest";
22 +const MAX_ENVELOPE_BYTES: usize = 8 * 1024 * 1024;
23 +const MAX_PAYLOAD_BYTES: usize = 1024 * 1024;
24 +const MAX_INPUTS: usize = 16;
25 +const MAX_INPUT_DEPTH: usize = 64;
26 +const MAX_TYPE_URL_BYTES: usize = 512;
27 +const CONVERSION_STACK_BYTES: usize = 2 * 1024 * 1024;
28 +
29 +pub(crate) fn with_conversion_stack<T>(f: impl FnOnce() -> T) -> T {
30 +    // RelationNode conversion has a large enum-dispatch frame. Grow before
31 +    // entering it, including for an accepted nested extension on a small stack.
32 +    stacker::maybe_grow(64 * 1024, CONVERSION_STACK_BYTES, f)
33 +}
34 +
35 +// Embedded protobuf messages and bytes share their length-delimited wire format.
36 +// Keeping input Plans as bytes lets us validate arity and unsupported expression
37 +// fields before recursively decoding any input. The canonical .proto is checked
38 +// in under proto/sail/extension/v1/extension.proto.
39 +#[derive(Clone, PartialEq, Message)]
40 +struct SailExtensionRequest {
41 +    #[prost(string, tag = "1")]
42 +    payload_type_url: String,
43 +    #[prost(bytes = "vec", tag = "2")]
44 +    payload: Vec<u8>,
45 +    #[prost(bytes = "vec", repeated, tag = "3")]
46 +    inputs: Vec<Vec<u8>>,
47 +    #[prost(bytes = "vec", repeated, tag = "4")]
48 +    input_expressions: Vec<Vec<u8>>,
49 +    #[prost(uint32, tag = "5")]
50 +    envelope_version: u32,
51 +}
52 +
53 +thread_local! {
54 +    // Conversion is synchronous. This budget remains live while converting
55 +    // nested Any payloads, whose byte fields otherwise reset prost's recursion
56 +    // budget on every separate decode. Ordinary non-extension plans are unchanged.
57 +    static EXTENSION_INPUT_DEPTH: Cell<usize> = const { Cell::new(0) };
58 +}
59 +
60 +pub(crate) struct RelationConversionGuard {
61 +    active: bool,
62 +}
63 +
64 +impl RelationConversionGuard {
65 +    pub(crate) fn enter() -> SparkResult<Self> {
66 +        Self::enter_scope(false)
67 +    }
68 +
69 +    fn enter_scope(extension: bool) -> SparkResult<Self> {
70 +        EXTENSION_INPUT_DEPTH.with(|depth| {
71 +            let current = depth.get();
72 +            let active = extension || current != 0;
73 +            if active {
74 +                if current >= MAX_INPUT_DEPTH {
75 +                    return Err(SparkError::invalid(format!(
76 +                        "Sail extension input nesting exceeds {MAX_INPUT_DEPTH} level
77 +                        "
78 +                    )));
79 +                }
80 +                depth.set(current + 1);
81 +            }
82 +            Ok(Self { active })
83 +        })
84 +    }
85 +}

```

```

86 +impl Drop for RelationConversionGuard {
87 +    fn drop(&mut self) {
88 +        if self.active {
89 +            EXTENSION_INPUT_DEPTH.with(|depth| depth.set(depth.get() - 1));
90 +        }
91 +    }
92 +}
93 +
94 +fn validate_type_url(type_url: &str) -> SparkResult<()> {
95 +    if type_url.is_empty() || type_url.len() > MAX_TYPE_URL_BYTES {
96 +        return Err(SparkError::invalid(format!(
97 +            "extension type URL must contain between 1 and {MAX_TYPE_URL_BYTES} bytes
98 +            ")));
99 +    }
100 +    Ok(())
101 +}
102 +
103 +/// Inspect tags without allocating input vectors or decoding nested messages.
104 +fn preflight_envelope(mut bytes: &[u8]) -> SparkResult<()> {
105 +    let mut input_count = 0;
106 +    while !bytes.is_empty() {
107 +        let (tag, _) = wire::next_field(&mut bytes)?;
108 +        if tag == 3 {
109 +            input_count += 1;
110 +            if input_count > MAX_INPUTS {
111 +                return Err(SparkError::invalid(format!(
112 +                    "Sail extension accepts at most {MAX_INPUTS} input plans"
113 +                )));
114 +            }
115 +        }
116 +        if tag == 4 {
117 +            return Err(SparkError::unsupported("Sail extension input expressions"));
118 +        }
119 +    }
120 +    Ok(())
121 +}
122 +
123 +pub(crate) fn convert_relation_extension(extension: Any) -> SparkResult<spec::QueryNode> {
124 +    validate_type_url(&extension.type_url)?;
125 +    if extension.type_url != ENVELOPE_TYPE_URL {
126 +        if extension.value.len() > MAX_PAYLOAD_BYTES {
127 +            return Err(SparkError::invalid(
128 +                "Connect extension payload exceeds 1 MiB",
129 +            ));
130 +        }
131 +        return Ok(spec::QueryNode::Extension {
132 +            payload_type_url: extension.type_url,
133 +            payload: extension.value.to_vec(),
134 +            inputs: vec![],
135 +            is_envelope: false,
136 +        });
137 +    }
138 +    if extension.value.len() > MAX_ENVELOPE_BYTES {
139 +        return Err(SparkError::invalid("Sail extension envelope exceeds 8 MiB"));
140 +    }
141 +    preflight_envelope(&extension.value)?;
142 +    let envelope = SailExtensionRequest::decode(extension.value.as_ref())?;
143 +    if envelope.envelope_version != 1 {
144 +        return Err(SparkError::invalid(format!(
145 +            "unsupported Sail extension envelope version {}; expected 1",
146 +            envelope.envelope_version
147 +        )));
148 +    }
149 +    validate_type_url(&envelope.payload_type_url)?;
150 +    if envelope.payload.len() > MAX_PAYLOAD_BYTES {
151 +        return Err(SparkError::invalid(
152 +            "Connect extension payload exceeds 1 MiB",
153 +        ));
154 +    }
155 +    let _depth = RelationConversionGuard::enter_scope(true)?;
156 +    let inputs = envelope
157 +        .inputs
158 +        .into_iter()
159 +        .map(|input| {
160 +            let remaining = EXTENSION_INPUT_DEPTH.with(|depth| MAX_INPUT_DEPTH - dept
161 +            h.get());

```

```

161 +         wire::validate_plan(&input, remaining)?;
162 +         // The generated decoder has no recursive call-site hook. Its raw
163 +         // message depth is already bounded above, so give this bounded
164 +         // decode a known stack even when called from a small initial one.
165 +         stacker::grow(CONVERSION_STACK_BYTES, || {
166 +             Plan::decode(input.as_slice())?.try_into()
167 +         })
168 +     })
169 +     .collect::<SparkResult<Vec<spec::QueryPlan>>>()?);
170 +     Ok(spec::QueryNode::Extension {
171 +         payload_type_url: envelope.payload_type_url,
172 +         payload: envelope.payload,
173 +         inputs,
174 +         is_envelope: true,
175 +     })
176 + }
177 +
178 +#[cfg(test)]
179 +mod tests;

```

74. crates/sail-spark-connect/src/proto/extension/tests.rs

```

1 diff --git a/crates/sail-spark-connect/src/proto/extension/tests.rs b/crates/sail-spark-connect/src/proto/extension/tests.rs
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..5b3286838001803d55a26e0db589e2dce2480e
4 ca
5 --- /dev/null
6 +++ b/crates/sail-spark-connect/src/proto/extension/tests.rs
7 @@ -0,0 +1,230 @@
8 +use super::*;
9 +use crate::spark::connect::{Command, Range, Relation, RelationCommon, plan, relation}
10 +;
11 +
12 +fn request(inputs: Vec<Vec<u8>>) -> SailExtensionRequest {
13 +    SailExtensionRequest {
14 +        payload_type_url: "type.test/Nutmeg".into(),
15 +        payload: b"payload".to_vec(),
16 +        inputs,
17 +        input_expressions: vec![],
18 +        envelope_version: 1,
19 +    }
20 +}
21 +
22 +fn pack(request: SailExtensionRequest) -> Any {
23 +    Any {
24 +        type_url: ENVELOPE_TYPE_URL.into(),
25 +        value: request.encode_to_vec().into(),
26 +    }
27 +}
28 +
29 +fn range_plan() -> Plan {
30 +    Plan {
31 +        op_type: Some(plan::OpType::Root(Relation {
32 +            common: Some(RelationCommon {
33 +                plan_id: Some(91),
34 +                ..Default::default()
35 +            }),
36 +            rel_type: Some(relation::RelType::Range(Range {
37 +                start: Some(0),
38 +                end: 5,
39 +                step: 1,
40 +                num_partitions: Some(4),
41 +            })),
42 +        })),
43 +    }
44 +}
45 +
46 +#[test]
47 +fn envelope_retains_input_plans_payload_and_plan_ids() -> SparkResult<()> {
48 +    let node = convert_relation_extension(pack(request(vec![range_plan().encode_to_vec()])))?;
49 +    match node {
50 +        spec::QueryNode::Extension {
51 +            payload_type_url,

```

```

50 +         payload,
51 +         inputs,
52 +         is_envelope,
53 +     } => {
54 +         assert_eq!(payload_type_url, "type.test/Nutmeg");
55 +         assert_eq!(payload, b"payload");
56 +         assert!(is_envelope);
57 +         assert_eq!(inputs.len(), 1);
58 +         assert_eq!(inputs[0].plan_id, Some(91));
59 +         assert!(
60 +             matches!(&inputs[0].node, spec::QueryNode::Range(range) if range.num_
partitions == Some(4))
61 +         );
62 +     }
63 +     other => return Err(SparkError::internal(format!("unexpected node {other:?}"))),
64 + }
65 + Ok(())
66 +}
67 +
68 +#[test]
69 +fn bare_any_keeps_its_type_url_and_has_no_inputs() -> SparkResult<> {
70 +    let node = convert_relation_extension(Any {
71 +        type_url: "type.test/Read".into(),
72 +        value: vec![1, 2, 3].into(),
73 +    })?;
74 +    assert!(
75 +        matches!(node, spec::QueryNode::Extension { payload_type_url, inputs, is_enve
lope: false, .. }
76 +            if payload_type_url == "type.test/Read" && inputs.is_empty())
77 +    );
78 +    Ok(())
79 +}
80 +
81 +#[test]
82 +fn rejects_version_expressions_nonquery_inputs_and_malformed_payloads() {
83 +    let mut version = request(vec![]);
84 +    version.envelope_version = 0;
85 +    assert!(
86 +        matches!(convert_relation_extension(pack(version)), Err(e) if e.to_string().c
ontains("expected 1"))
87 +    );
88 +    let mut expressions = request(vec![vec![255]]);
89 +    expressions.input_expressions.push(vec![255]);
90 +    assert!(
91 +        matches!(convert_relation_extension(pack(expressions)), Err(e) if e.to_string
().contains("input expressions"))
92 +    );
93 +    let command = Plan {
94 +        op_type: Some(plan::OpType::Command(Command::default())),
95 +    };
96 +    assert!(
97 +        matches!(convert_relation_extension(pack(request(vec![command.encode_to_vec()
]))), Err(e) if e.to_string().contains("relation expected"))
98 +    );
99 +    assert!(convert_relation_extension(pack(request(vec![vec![255]]))).is_err());
100 +}
101 +
102 +#[test]
103 +fn limits_input_count_before_decoding_any_child() {
104 +    // Each child is invalid protobuf; the count error must be returned first.
105 +    let envelope = request(vec![vec![255]; MAX_INPUTS + 1]);
106 +    assert!(
107 +        matches!(convert_relation_extension(pack(envelope)), Err(e) if e.to_string().
contains("at most 16"))
108 +    );
109 +}
110 +
111 +#[test]
112 +fn limits_payload_and_envelope_bytes() {
113 +    let bare = Any {
114 +        type_url: "type.test/Read".into(),
115 +        value: vec![0; MAX_PAYLOAD_BYTES + 1].into(),
116 +    };
117 +    assert!(
118 +        matches!(convert_relation_extension(bare), Err(e) if e.to_string().contains("
exceeds 1 MiB"))

```

```

119 + );
120 + let envelope = Any {
121 +     type_url: ENVELOPE_TYPE_URL.into(),
122 +     value: vec![0; MAX_ENVELOPE_BYTES + 1].into(),
123 + };
124 + assert!(
125 +     matches!(convert_relation_extension(envelope), Err(e) if e.to_string().contains("exceeds 8 MiB"))
126 + );
127 +}
128 +
129 +#[test]
130 +fn nested_any_does_not_reset_recursion_budget_and_failure_releases_guard() -> SparkResult<()> {
131 +    let mut input = range_plan();
132 +    for _ in 0..1000 {
133 +        input = Plan {
134 +            op_type: Some(plan::OpType::Root(Relation {
135 +                common: None,
136 +                rel_type: Some(relation::RelType::Extension(pack(request(vec![
137 +                    input.encode_to_vec(),
138 +                ])))),
139 +            })),
140 +        };
141 +    }
142 +    let result = spec::QueryPlan::try_from(input);
143 +    assert!(matches!(result, Err(e) if e.to_string().contains("nesting exceeds")));
144 +    convert_relation_extension(pack(request(vec![range_plan().encode_to_vec()])))?;
145 +    Ok(())
146 +}
147 +
148 +#[test]
149 +fn accepts_near_limit_nested_envelopes_on_small_initial_stack() -> SparkResult<()> {
150 +    let mut input = range_plan();
151 +    // Each nested envelope traverses four protobuf message edges. Fifteen
152 +    // layers plus the final Plan/Relation/Range stay just below the 64 limit.
153 +    for _ in 0..15 {
154 +        input = Plan {
155 +            op_type: Some(plan::OpType::Root(Relation {
156 +                common: None,
157 +                rel_type: Some(relation::RelType::Extension(pack(request(vec![
158 +                    input.encode_to_vec(),
159 +                ])))),
160 +            })),
161 +        };
162 +    }
163 +    let envelope = pack(request(vec![input.encode_to_vec()]));
164 +    let thread = std::thread::Builder::new()
165 +        .stack_size(128 * 1024)
166 +        .spawn(move || convert_relation_extension(envelope).is_ok())?;
167 +    assert!(
168 +        thread
169 +            .join()
170 +            .map_err(|_| SparkError::internal("accepted input conversion thread panicked"))?
171 +    );
172 +    Ok(())
173 +}
174 +
175 +fn length_delimited(tag: u32, payload: &[u8]) -> Vec<u8> {
176 +    use prost::encoding::{WireType, encode_key, encode_varint};
177 +    let mut output = vec![];
178 +    encode_key(tag, WireType::LengthDelimited, &mut output);
179 +    encode_varint(payload.len() as u64, &mut output);
180 +    output.extend_from_slice(payload);
181 +    output
182 +}
183 +
184 +#[test]
185 +fn rejects_thousand_nested_projects_before_prost_decode_on_small_stack() -> SparkResult<()> {
186 +    // Construct wire bytes iteratively: Relation.project = 3, Project.input = 1.
187 +    // Never construct a recursive Rust message whose own Drop could overflow.
188 +    let mut relation = vec![];
189 +    for _ in 0..1000 {
190 +        relation = length_delimited(3, &length_delimited(1, &relation));
191 +    }
192 +    let plan = length_delimited(1, &relation);

```

```

193 +     let envelope = pack(request(vec![plan]));
194 +     let thread = std::thread::Builder::new()
195 +         .stack_size(128 * 1024)
196 +         .spawn(move || {
197 +             matches!(convert_relation_extension(envelope), Err(e) if e.to_string().contains("nesting exceeds"))
198 +         })?;
199 +     assert!(
200 +         thread
201 +             .join()
202 +             .map_err(|_| SparkError::internal("input validation thread panicked"))?
203 +     );
204 +     Ok(())
205 + }
206 +
207 + #[test]
208 + fn rejects_unknown_group_fields_without_recursive_skip() {
209 +     let groups = vec![0x0b; 1000]; // Field 1, start-group wire type.
210 +     let envelope = Any {
211 +         type_url: ENVELOPE_TYPE_URL.into(),
212 +         value: groups.clone().into(),
213 +     };
214 +     assert!(
215 +         matches!(convert_relation_extension(envelope), Err(e) if e.to_string().contains("groups are not supported"))
216 +     );
217 +     assert!(
218 +         matches!(wire::validate_plan(&groups, MAX_INPUT_DEPTH), Err(e) if e.to_string().contains("groups are not supported"))
219 +     );
220 + }
221 +
222 + #[test]
223 + fn preflight_treats_arrow_data_as_opaque_bytes() -> SparkResult<()> {
224 +     use crate::spark::connect::LocalRelation;
225 +     let input = Plan {
226 +         op_type: Some(plan::OpType::Root(Relation {
227 +             common: None,
228 +             rel_type: Some(relation::RelType::LocalRelation(LocalRelation {
229 +                 data: Some(vec![0x0b; 1000]),
230 +                 schema: None,
231 +             })),
232 +         })),
233 +     };
234 +     wire::validate_plan(&input.encode_to_vec(), MAX_INPUT_DEPTH)?;
235 +     Ok(())
236 + }

```

75. crates/sail-spark-connect/src/proto/extension/wire.rs

```

1 diff --git a/crates/sail-spark-connect/src/proto/extension/wire.rs b/crates/sail-spark-connect/src/proto/extension/wire.rs
2 new file mode 100644
3 index 0000000000000000000000000000000000000000..7d3685497ce2ff0cf33b6d28126ff96f3b3c241b
4 --- /dev/null
5 +++ b/crates/sail-spark-connect/src/proto/extension/wire.rs
6 @@ -0,0 +1,150 @@
7 +/// Bounded raw protobuf traversal before prost decodes embedded input plans.
8 +///
9 +/// The workspace enables prost's `no-recursion-limit`, so its generated decoder
10 +/// cannot protect this boundary. Visit only descriptor-declared message fields;
11 +/// arbitrary strings, Arrow IPC and extension-specific payloads remain opaque.
12 +/// Sail envelopes inside Any.value are traversed before any recursive decoding.
13 +
14 +use std::collections::BTreeMap;
15 +use std::sync::LazyLock;
16 +
17 +use prost::Message;
18 +use prost::encoding::{WireType, decode_key, decode_varint};
19 +use prost_types::field_descriptor_proto::Type;
20 +use prost_types::{DescriptorProto, FileDescriptorSet};
21 +
22 +use crate::error::{SparkError, SparkResult};
23 +use crate::spark::connect::FILE_DESCRIPTOR_SET;

```

```

24 +
25 +const ENVELOPE_MESSAGE: &str = ".sail.extension.v1.SailExtensionRequest";
26 +
27 +type MessageFields = BTreeMap<String, BTreeMap<u32, String>>;
28 +
29 +static MESSAGE_FIELDS: LazyLock<Result<MessageFields, String>> = LazyLock::new(|| {
30 + // This is build-generated, trusted data, never a descriptor from a request.
31 + let descriptors = FileDescriptorSet::decode(FILE_DESCRIPTOR_SET).map_err(|e| e.to_string())?;
32 + let mut messages = BTreeMap::new();
33 + for file in descriptors.file {
34 + let prefix = format!("{}.{}", file.package.as_deref().unwrap_or_default());
35 + for message in file.message_type {
36 + register_message(&mut messages, &prefix, message)?;
37 + }
38 + }
39 + messages.insert(
40 + ENVELOPE_MESSAGE.into(),
41 + BTreeMap::from([(3, ".spark.connect.Plan".into())]),
42 + );
43 + Ok(messages)
44 +});
45 +
46 +fn register_message(
47 + messages: &mut MessageFields,
48 + prefix: &str,
49 + message: DescriptorProto,
50 +) -> Result<(), String> {
51 + let name = message.name.ok_or("protobuf descriptor without a name")?;
52 + let name = format!("{prefix}.{name}");
53 + let mut fields = BTreeMap::new();
54 + for field in message.field {
55 + if field.r#type == Some(Type::Message as i32) {
56 + let number = field.number.ok_or("protobuf field without a number")?;
57 + let number = u32::try_from(number).map_err(|e| e.to_string())?;
58 + let target = field
59 + .type_name
60 + .ok_or("protobuf message field without a type")?;
61 + fields.insert(number, target);
62 + }
63 + }
64 + messages.insert(name.clone(), fields);
65 + for nested in message.nested_type {
66 + register_message(messages, &name, nested)?;
67 + }
68 + Ok(())
69 +}
70 +
71 +/// Read one field without recursion or allocation. Groups are not part of the
72 +/// proto3 extension contract and are rejected even when their tag is unknown.
73 +pub(super) fn next_field<'a>(bytes: &mut &'a [u8]) -> SparkResult<(u32, WireType, &'a [u8])> {
74 + let (tag, wire_type) = decode_key(bytes)?;
75 + let length = match wire_type {
76 + WireType::Varint => {
77 + decode_varint(bytes)?;
78 + 0
79 + }
80 + WireType::ThirtyTwoBit => 4,
81 + WireType::SixtyFourBit => 8,
82 + WireType::LengthDelimited => usize::try_from(decode_varint(bytes)?).
83 + .map_err(|_| SparkError::invalid("extension protobuf field length overflow"))?,
84 + WireType::StartGroup | WireType::EndGroup => {
85 + return Err(SparkError::invalid(
86 + "protobuf groups are not supported in extension input plans",
87 + ));
88 + }
89 + };
90 + if length > bytes.len() {
91 + return Err(SparkError::invalid("truncated extension protobuf field"));
92 + }
93 + let (value, rest) = bytes.split_at(length);
94 + *bytes = rest;
95 + Ok((tag, wire_type, value))
96 +}
97 +
98 +pub(super) fn validate_plan(bytes: &[u8], max_depth: usize) -> SparkResult<()> {

```

```

99 +     let messages = MESSAGE_FIELDS
100 +         .as_ref()
101 +         .map_err(|e| SparkError::internal(format!("invalid Spark protocol descriptor:
{e}")))?;
102 +     // Depth-first iteration retains at most one unfinished message per level,
103 +     // rather than allocating an entry for every repeated field in the request.
104 +     let mut stack = vec![(".spark.connect.Plan", bytes, 0usize)];
105 +     while let Some((message, mut remaining, depth)) = stack.pop() {
106 +         if depth >= max_depth {
107 +             return Err(SparkError::invalid(format!(
108 +                 "Sail extension input nesting exceeds the remaining {max_depth} proto
buf levels"
109 +             )));
110 +         }
111 +         if message == ".google.protobuf.Any" {
112 +             let mut type_url = &[][..];
113 +             let mut value = &[][..];
114 +             while !remaining.is_empty() {
115 +                 let (tag, wire_type, field) = next_field(&mut remaining)?;
116 +                 if matches!(tag, 1 | 2) && wire_type != WireType::LengthDelimited {
117 +                     return Err(SparkError::invalid(
118 +                         "invalid wire type for extension Any field",
119 +                     ));
120 +                 }
121 +                 match tag {
122 +                     1 => type_url = field,
123 +                     2 => value = field,
124 +                     _ => {}
125 +                 }
126 +             }
127 +             if type_url == super::ENVELOPE_TYPE_URL.as_bytes() {
128 +                 if value.len() > super::MAX_ENVELOPE_BYTES {
129 +                     return Err(SparkError::invalid("Sail extension envelope exceeds 8
MiB"));
130 +                 }
131 +                 super::preflight_envelope(value)?;
132 +                 stack.push((ENVELOPE_MESSAGE, value, depth + 1));
133 +             }
134 +             continue;
135 +         }
136 +         let fields = messages.get(message).ok_or_else(|| {
137 +             SparkError::internal(format!("missing Spark protocol descriptor for {mess
age}")
138 +         ))?;
139 +         while !remaining.is_empty() {
140 +             let (tag, wire_type, value) = next_field(&mut remaining)?;
141 +             if let Some(child) = fields.get(&tag) {
142 +                 if wire_type != WireType::LengthDelimited {
143 +                     return Err(SparkError::invalid(format!(
144 +                         "invalid protobuf wire type for message field {message}:{tag}
"
145 +                     )));
146 +                 }
147 +                 if !remaining.is_empty() {
148 +                     stack.push((message, remaining, depth));
149 +                 }
150 +                 stack.push((child.as_str(), value, depth + 1));
151 +                 break;
152 +             }
153 +         }
154 +     }
155 +     Ok(())
156 + }

```

76. crates/sail-spark-connect/src/proto/mod.rs

```

1 diff --git a/crates/sail-spark-connect/src/proto/mod.rs b/crates/sail-spark-connect/src
c/proto/mod.rs
2 index 1dc12e61045d0fb638b08ceddc9c9c05b1e57e65..2d5e6371388750f86218d641c2bd546505e4c5
ab 100644
3 --- a/crates/sail-spark-connect/src/proto/mod.rs
4 +++ b/crates/sail-spark-connect/src/proto/mod.rs
5 @@ -4,6 +4,7 @@ pub(crate) mod data_type;
6  pub(crate) mod data_type_arrow;
7  pub(crate) mod data_type_json;

```

```

8 pub(crate) mod expression;
9 +pub(crate) mod extension;
10 pub(crate) mod function;
11 pub(crate) mod literal;
12 pub(crate) mod plan;

```

77. crates/sail-spark-connect/src/proto/plan.rs

```

1 diff --git a/crates/sail-spark-connect/src/proto/plan.rs b/crates/sail-spark-connect/s
rc/proto/plan.rs
2 index f7d41776c81541130ea0d47ed6063282fdd9e79d..fe8dadf5f43b17216d689774d4569c56372098
b6 100644
3 --- a/crates/sail-spark-connect/src/proto/plan.rs
4 +++ b/crates/sail-spark-connect/src/proto/plan.rs
5 @@ -102,9 +102,10 @@ impl TryFrom<Relation> for spec::Plan {
6
7     /// Converts a relation to a plan, somehow SQL text is parsed here.
8     fn try_from(relation: Relation) -> SparkResult<spec::Plan> {
9 +     let depth = super::extension::RelationConversionGuard::enter();
10     let Relation { common, rel_type } = relation;
11     let rel_type = rel_type.required("relation type");
12 -     let node: RelationNode = rel_type.try_into();
13 +     let node: RelationNode = super::extension::with_conversion_stack(|| rel_type.
try_into());
14     let metadata: RelationMetadata = common.into();
15     match node {
16         RelationNode::Query(query) => Ok(spec::Plan::Query(spec::QueryPlan {
17 @@ -123,9 +124,10 @@ impl TryFrom<Relation> for spec::QueryPlan {
18         type Error = SparkError;
19
20     fn try_from(relation: Relation) -> SparkResult<spec::QueryPlan> {
21 +     let depth = super::extension::RelationConversionGuard::enter();
22     let Relation { common, rel_type } = relation;
23     let rel_type = rel_type.required("relation type");
24 -     let node: RelationNode = rel_type.try_into();
25 +     let node: RelationNode = super::extension::with_conversion_stack(|| rel_type.
try_into());
26     let metadata: RelationMetadata = common.into();
27     Ok(spec::QueryPlan {
28         node: node.try_into_query(),
29 @@ -138,9 +140,10 @@ impl TryFrom<Relation> for spec::CommandPlan {
30     type Error = SparkError;
31
32     fn try_from(relation: Relation) -> SparkResult<spec::CommandPlan> {
33 +     let depth = super::extension::RelationConversionGuard::enter();
34     let Relation { common, rel_type } = relation;
35     let rel_type = rel_type.required("relation type");
36 -     let node: RelationNode = rel_type.try_into();
37 +     let node: RelationNode = super::extension::with_conversion_stack(|| rel_type.
try_into());
38     let metadata: RelationMetadata = common.into();
39     Ok(spec::CommandPlan {
40         node: node.try_into_command(),
41 @@ -1336,7 +1339,9 @@ impl TryFrom<RelType> for RelationNode {
42     RelType::RelationChanges(_) => Err(SparkError::unsupported("relation chan
ges")),
43     RelType::NearestByJoin(_) => Err(SparkError::unsupported("nearest-by join
")),
44     RelType::MLRelation(_) => Err(SparkError::unsupported("ML relation")),
45 -     RelType::Extension(_) => Err(SparkError::unsupported("extension relation"
)),
46 +     RelType::Extension(extension) => Ok(RelationNode::Query(
47 +         super::extension::convert_relation_extension(extension)?,
48 +     )),
49     RelType::Unknown(_) => Err(SparkError::unsupported("unknown relation")),
50 }
51 }

```

78. crates/sail-spark-connect/src/server.rs

```

1 diff --git a/crates/sail-spark-connect/src/server.rs b/crates/sail-spark-connect/src/s
erver.rs

```

```

2 index 3032f8d1bf5f6077cf4beb06ef63bf66d3cdbea2..9455ed6e7bbce1fc355891459759c41097a72b
f5 100644
3 --- a/crates/sail-spark-connect/src/server.rs
4 +++ b/crates/sail-spark-connect/src/server.rs
5 @@ -125,7 +125,12 @@ impl SparkConnectService for SparkConnectServer {
6     request: Request<ExecutePlanRequest>,
7     ) -> Result<Response<Self::ExecutePlanStream>, Status> {
8         let request = request.into_inner();
9         debug!("{request:?}");
10 +        // Extension payloads can carry session-owned capabilities. Log routing
11 +        // identifiers, never opaque plan bytes or client argument values.
12 +        debug!(
13 +            "ExecutePlan session_id={:?} operation_id={:?}",
14 +            request.session_id, request.operation_id
15 +        );
16         let session_id = request.session_id;
17         let user_id = request.user_context.map(|u| u.user_id).unwrap_or_default();
18         let metadata = ExecutorMetadata {
19 @@ -166,7 +171,7 @@ impl SparkConnectService for SparkConnectServer {
20         use crate::spark::connect::analyze_plan_response;
21
22         let request = request.into_inner();
23 -        debug!("{request:?}");
24 +        debug!("AnalyzePlan session_id={}", request.session_id);
25         let session_id = request.session_id.clone();
26         let user_id = request.user_context.map(|u| u.user_id).unwrap_or_default();
27         let ctx = self

```

79. crates/sail-spark-connect/src/service/plan_analyzer.rs

```

1 diff --git a/crates/sail-spark-connect/src/service/plan_analyzer.rs b/crates/sail-spark-
k-connect/src/service/plan_analyzer.rs
2 index a15ac9c01405e131e11407e412ba5651fcf951bb..5a25fd0232b86031aa4c21e031b01ac5e47e54
15 100644
3 --- a/crates/sail-spark-connect/src/service/plan_analyzer.rs
4 +++ b/crates/sail-spark-connect/src/service/plan_analyzer.rs
5 @@ -251,6 +251,7 @@ fn is_streaming_query_plan(plan: &spec::QueryPlan) -> bool {
6     fn is_streaming_query_node(node: &spec::QueryNode) -> bool {
7         match node {
8             spec::QueryNode::Read { is_streaming, .. } => *is_streaming,
9 +            spec::QueryNode::Extension { inputs, .. } => inputs.iter().any(is_streaming_q
uery_plan),
10         // leaf nodes with no query plan inputs
11         spec::QueryNode::LocalRelation { .. }
12         | spec::QueryNode::CachedLocalRelation { .. }

```

80. crates/sail-spark-connect/src/service/plan_executor.rs

```

1 diff --git a/crates/sail-spark-connect/src/service/plan_executor.rs b/crates/sail-spar
k-connect/src/service/plan_executor.rs
2 index 1c42e4eb2c46eaf2376003ff034ca0385c1657b2..d0e8d103079b01e8f32fe23721d5ca53919005
59 100644
3 --- a/crates/sail-spark-connect/src/service/plan_executor.rs
4 +++ b/crates/sail-spark-connect/src/service/plan_executor.rs
5 @@ -566,9 +566,10 @@ pub(crate) async fn handle_execute_remove_cached_remote_relation_
command(
6     pub(crate) async fn handle_interrupt_all(ctx: &SessionContext) -> SparkResult<Vec<Str
ing>> {
7         let spark = ctx.extension:::<SparkSession>()?;
8         let mut results = vec![];
9         for executor in spark.remove_all_executors()? {
10             executor.pause_if_running().await?;
11             results.push(executor.metadata.operation_id.clone());
12 +        for executor in spark.all_executors()? {
13 +            if executor.interrupt().await? {
14 +                results.push(executor.metadata.operation_id.clone());
15 +            }
16         }
17         Ok(results)
18     }
19 @@ -579,9 +580,10 @@ pub(crate) async fn handle_interrupt_tag(
20     ) -> SparkResult<Vec<String>> {
21         let spark = ctx.extension:::<SparkSession>()?;

```

```

22     let mut results = vec![];
23 -   for executor in spark.remove_executors_by_tag(tag.as_str())? {
24 -     executor.pause_if_running().await?;
25 -     results.push(executor.metadata.operation_id.clone());
26 +   for executor in spark.executors_by_tag(tag.as_str())? {
27 +     if executor.interrupt().await? {
28 +       results.push(executor.metadata.operation_id.clone());
29 +     }
30   }
31   Ok(results)
32 }
33 @@ -591,10 +593,13 @@ pub(crate) async fn handle_interrupt_operation_id(
34     operation_id: String,
35 ) -> SparkResult<Vec<String>> {
36     let spark = ctx.extension:::<SparkSession>()?;
37 -   match spark.remove_executor(operation_id.as_str())? {
38 +   match spark.get_executor(operation_id.as_str())? {
39       Some(executor) => {
40 -       executor.pause_if_running().await?;
41 -       Ok(vec![executor.metadata.operation_id.clone()])
42 +       if executor.interrupt().await? {
43 +         Ok(vec![executor.metadata.operation_id.clone()])
44 +       } else {
45 +         Ok(vec![])
46 +       }
47     }
48     None => Ok(vec![]),
49 }

```

81. crates/sail-spark-connect/src/session.rs

```

1 diff --git a/crates/sail-spark-connect/src/session.rs b/crates/sail-spark-connect/src/
2 session.rs
3 index 277060435d503c3b72bd9534d694dc57f0545948..c2e98ff3d8133874e04bc57e5d0a1fd6f06a07
4 a4 100644
5 --- a/crates/sail-spark-connect/src/session.rs
6 +++ b/crates/sail-spark-connect/src/session.rs
7 @@ -7,6 +7,7 @@ use datafusion::execution::SendableRecordBatchStream;
8 use datafusion::logical_expr::StringifiedPlan;
9 use sail_common::utils::datetime::get_system_timezone;
10 use sail_common_datafusion::extension::SessionExtension;
11 +use sail_common_datafusion::session::lifecycle::SessionResource;
12 use sail_plan::config::PlanConfig;
13
14 use crate::config::{ConfigKeyValue, SparkRuntimeConfig};
15 @@ -49,6 +50,34 @@ impl SessionExtension for SparkSession {
16 }
17
18 +#[tonic::async_trait]
19 +impl SessionResource for SparkSession {
20 +    async fn stop(&self) -> datafusion::common::Result<()> {
21 +        let mut executors = {
22 +            let mut state = self.state.lock().map_err(|error| {
23 +                datafusion::common::DataFusionError::Execution(error.to_string())
24 +            })?;
25 +            state.stopped = true;
26 +            // Dropping streaming query signals also cancels their producers.
27 +            state.streaming_queries = StreamingQueryManager::new();
28 +            state
29 +                .executors
30 +                .drain()
31 +                .map(|(_, executor)| executor)
32 +                .collect:::<Vec<_>>()
33 +        };
34 +        let mut failure = None;
35 +        for executor in executors {
36 +            if let Err(error) = executor.interrupt().await {
37 +                failure = Some(datafusion::common::DataFusionError::Execution(
38 +                    error.to_string(),
39 +                ));
40 +            }
41 +        }
42 +        failure.map_or(Ok(()), Err)
43 +    }
44 }

```

```

43 +}
44 +
45 impl SparkSession {
46     pub(crate) fn try_new(
47         session_id: String,
48 @@ -172,6 +201,9 @@ impl SparkSession {
49     pub(crate) fn add_executor(&self, executor: Executor) -> SparkResult<()> {
50         let mut state = self.state.lock()?;
51         let id = executor.metadata.operation_id.clone();
52 +         if state.stopped {
53 +             return Err(SparkError::OperationInterrupted(id));
54 +         }
55         state.executors.insert(id, Arc::new(executor));
56         Ok(())
57     }
58 @@ -189,31 +221,19 @@ impl SparkSession {
59     .map(|(_, executor)| executor))
60 }
61
62 - pub(crate) fn remove_all_executors(&self) -> SparkResult<Vec<Arc<Executor>>> {
63 -     let mut state = self.state.lock()?;
64 -     let mut out = Vec::new();
65 -     for (_, executor) in state.executors.drain() {
66 -         out.push(executor);
67 -     }
68 -     Ok(out)
69 + pub(crate) fn all_executors(&self) -> SparkResult<Vec<Arc<Executor>>> {
70 +     let state = self.state.lock()?;
71 +     Ok(state.executors.values().cloned().collect())
72 }
73
74 - pub(crate) fn remove_executors_by_tag(&self, tag: &str) -> SparkResult<Vec<Arc<Ex
75 ecutor>>> {
76 -     let mut state = self.state.lock()?;
77 -     let tag = tag.to_string();
78 -     let mut ids = Vec::new();
79 -     let mut removed = Vec::new();
80 -     for (key, executor) in &state.executors {
81 -         if executor.metadata.tags.contains(&tag) {
82 -             ids.push(key.clone());
83 -         }
84 -     }
85 -     for key in ids.iter() {
86 -         if let Some(executor) = state.executors.remove(key) {
87 -             removed.push(executor);
88 -         }
89 -     }
90 + pub(crate) fn executors_by_tag(&self, tag: &str) -> SparkResult<Vec<Arc<Executor>
91 >> {
92 +     let state = self.state.lock()?;
93 +     Ok(state
94 +         .executors
95 +         .values()
96 +         .filter(|executor| executor.metadata.tags.iter().any(|value| value == tag
97 +         ))
98 +         .cloned()
99 +         .collect())
100 }
101
102 pub(crate) fn start_streaming_query(
103 @@ -234,6 +254,9 @@ impl SparkSession {
104     run_id: uuid::Uuid::new_v4().to_string(),
105 );
106 let mut state = self.state.lock()?;
107 + if state.stopped {
108 +     return Err(SparkError::OperationInterrupted(self.session_id.clone()));
109 + }
110 let query = StreamingQuery::new(name, info, stream);
111 state.streaming_queries.add_query(id.clone(), query);
112 Ok(id)
113 @@ -306,6 +329,7 @@ impl SparkSession {
114 }
115
116 struct SparkSessionState {
117 +     stopped: bool,
118     config: SparkRuntimeConfig,
119     executors: HashMap<String, Arc<Executor>>,

```

```

118     streaming_queries: StreamingQueryManager,
119 @@ -314,6 +338,7 @@ struct SparkSessionState {
120 impl SparkSessionState {
121     fn try_new() -> SparkResult<Self> {
122         Ok(Self {
123 +         stopped: false,
124             config: SparkRuntimeConfig::try_new()?,
125             executors: HashMap::new(),
126             streaming_queries: StreamingQueryManager::new(),

```

82. crates/sail-spark-connect/src/session_manager.rs

```

1 diff --git a/crates/sail-spark-connect/src/session_manager.rs b/crates/sail-spark-connect/src/session_manager.rs
2 index caec1c289280e86b86c1c65d3e1b846a8a4a4b..818ee7f5115e1bac30ce573dc6a24d40cc01af
3 95 100644
4 --- a/crates/sail-spark-connect/src/session_manager.rs
5 +++ b/crates/sail-spark-connect/src/session_manager.rs
6 @@ -9,6 +9,7 @@ use sail_common::actor::ActorSystem;
7 use sail_common::config::AppConfig;
8 use sail_common::runtime::RuntimeHandle;
9 use sail_common_datafusion::catalog::display::DefaultCatalogDisplay;
10 +use sail_common_datafusion::session::lifecycle::SessionLifecycle;
11 use sail_common_datafusion::session::plan::PlanService;
12 use sail_plan::catalog::SparkCatalogObjectDisplay;
13 use sail_plan::formatter::SparkPlanFormatter;
14 @@ -48,19 +49,22 @@ impl ServerSessionMutator for SparkSessionMutator {
15     Box::new(DefaultCatalogDisplay::<SparkCatalogObjectDisplay>::default()),
16     Box::new(SparkPlanFormatter),
17 );
18 - let spark = SparkSession::try_new(
19 -     info.session_id.clone(),
20 -     info.user_id.clone(),
21 -     SparkSessionOptions {
22 -         execution_heartbeat_interval: Duration::from_secs(
23 -             self.config.spark.execution_heartbeat_interval_secs,
24 -         ),
25 -     },
26 - );
27 - .map_err(|e| internal_datafusion_err!("{e}"))?;
28 + let spark = Arc::new(
29 +     SparkSession::try_new(
30 +         info.session_id.clone(),
31 +         info.user_id.clone(),
32 +         SparkSessionOptions {
33 +             execution_heartbeat_interval: Duration::from_secs(
34 +                 self.config.spark.execution_heartbeat_interval_secs,
35 +             ),
36 +         },
37 +     )
38 +     .map_err(|e| internal_datafusion_err!("{e}"))?,
39 + );
40 Ok(config
41     .with_extension(Arc::new(plan_service))
42     .with_extension(Arc::new(spark)))
43 + .with_extension(Arc::new(SessionLifecycle::new(spark.clone())))
44 + .with_extension(spark))
45 }
46 fn mutate_state(

```